2026 Python 數據分析與 AI 基礎：從資料思維到機器學習入門

24 小時線上課程完整計畫

作者：桑尼資料科學 | Data Sunnie

版本：2.0

2026 Python 數據分析與 AI 基礎：從資料思維到機器學習入門

24 小時線上課程完整計畫

壹、課程核心主張

本課程以 Python 為核心工具，從資料思維出發，帶領初學者建立完整的數據分析能力鏈：從理解資料結構、清洗轉換、視覺化探

一句話定位：2026 年學 Python 做數據分析，不只是學一門語言，而是在替未來的機器學習、深度學習、統計建模、資料工程與 AI 應用，先搭一條共通底盤。

貳、課程定位 — 五重身份

#	身份	說明
1	數據分析入門課	建立從讀檔到洞察的完整工作流
2	資料思維啟蒙課	把資料看成結構，把分析看成流程
3	統計直覺建立課	讓分析不只停在「看起來像」，而是「夠不夠可
4	機器學習前導課	打通特徵、訓練、評估的基本框架
5	AI 職涯地圖課	畫出統計 / ML / DL / DE / LLM 五條路線

參、適合對象

- 想進資料分析 / AI / 數據產品 / BI / DS / DE 的初學者
- 會一點程式，但不會用 Python 做分析的人
- 未來想接 ML、DL、LLM、MLOps、資料工程的人
- Excel 使用者，想升級到可重現的分析工作流

不適合： 已有 2 年以上 Python 資料分析經驗的人。

肆、為什麼 2026 還要學 Python？— 四大理由

1. 它仍是資料與 AI 的共通語言

Python 在 JetBrains/PSF 2024 調查中持續維持高能見度，跨研究、教學、產品原型、資料工程、AI 應用的共同選擇。

2. 生態不是單點強，而是整條鏈都能接

Jupyter Notebook -> NumPy -> pandas -> scikit-learn -> PyTorch -> PySpark，這種「從學習到產業」的連續性，仍然很難被單一工具取代。

3. 近五年它不是原地踏步，而是在升級

pandas 2.0（Arrow backend）、NumPy 2.0（首個 major release）、PyTorch 2.0（編譯優化）、Python 3.13（free-threading 路線）。

4. 資料分析已經不是報表工作，而是 AI 前處理工程

Anaconda 2024 報告：AI adoption 持續升高，但真正拉開差距的是資料處理能力。

伍、課程要交付的五大能力

#	能力	具體表現
1	把資料看成結構	理解欄位、型態、缺失值、分布、維度，不再只 Excel
2	把分析看成流程	從讀檔、清洗、轉換、探索、視覺化到建模，是
3	用圖表發現模式	選對圖、問對問題、從觀察到洞察到建議
4	用統計做出判斷	不只描述，還能判斷差異是否可信、效果是否多
5	知道下一步去哪	清楚統計 / ML / DL / DE / LLM 五條路線的分岔

陸、24 小時模組總覽

模組	主題	時數	關鍵交付
M0	開場：為什麼是 Python，為什麼是現在	3h	時代感、生態
M1	資料思維與 Notebook 工作流	3h	資料結構觀、實作
M2	Python 分析基礎	3h	分析師需要的
M3	NumPy 與陣列思維	3h	ndarray、broa
M4	pandas 與資料清理	3h	DataFrame、
M5	視覺化與 EDA	3h	Matplotlib/Se 流程
M6	統計思維與假設檢定	3h	分布、信賴區 Test
M7	ML 前導與學習路線	3h	scikit-learn、
合計		24h	

柒、教學敘事節奏 — 六幕結構

第一幕：看見全貌（M0）

你不是來學語法，你是來學用 Python 理解世界、處理資料、準備進入 AI 的入口。

第二幕：建立資料思維（M1）

資料有結構、有品質問題、有分析生命週期。先看懂資料，才能開始分析。

第三幕：掌握表達工具（M2）

用 Python 表達資料問題。變數、容器、迴圈、函式——只教分析真的會用到的。

第四幕：進入數值與表格世界（M3 + M4）

NumPy 給你陣列思維，pandas 給你表格操作力。這是資料分析的雙引擎。

第五幕：從表格到洞察（M5 + M6）

視覺化讓你看見模式，統計讓你判斷模式是否可信。從描述升級到判斷。

第六幕：走向 AI（M7）

機器學習不是魔法，是函數逼近。用 scikit-learn 體驗建模流程，然後看清未來的五條路。

捌、投影片風格原則

原則 1：每頁只做一件事

不要一頁塞三個概念。一頁只回答一個問題。

原則 2：每個章節要有高層視角頁

安靜但有力的金句頁： - "Data is the raw material; Python is the operating language." - "Analysis turns records into decisions." - "Statistics turns observations into confidence." - "Machine learning

turns patterns into prediction."

原則 3：節奏要有鬆緊

1. 先拉高視角
2. 再落到工具
3. 中段給成就感
4. 後段再升維
5. 結尾給道路圖

玖、可引用的經典里程碑

事件	意義	建議插入模組
pandas 2.0	資料框架走向 Arrow / 效能 / 生產級	M4
NumPy 2.0	Scientific Python 底層大版本重整	M3
scikit-learn 1.0	傳統 ML 工具鏈成熟穩定	M7
PyTorch 2.0	深度學習從寫模型走向編譯優化	M7
Jupyter Notebook	互動分析標配介面	M1
Polars / DuckDB 興起	依規模選工具，不再只有 pandas	M4 / M7

拾、課程結束後的五條學習路線

路線 A：統計分析深化

機率分布 -> 抽樣理論 -> 假設檢定 -> 回歸分析 -> A/B Test

路線 B：機器學習

scikit-learn -> 特徵工程 -> 模型評估 -> 交叉驗證 -> 集成方法

路線 C：深度學習

線性代數 -> 張量觀念 -> PyTorch -> MLP / CNN / RNN / Transformer

路線 D：大數據 / 資料工程

SQL -> PySpark -> ETL -> Arrow -> DuckDB / Polars -> workflow orchestration

路線 E：AI 應用 / LLM 工程

向量資料處理 -> embeddings -> prompt workflow -> RAG -> agent pipeline

---

拾壹、進階課程銜接

想更深入軟體工程、系統設計與架構思維的學生，可以接續本系列的第二門課：

「系統設計與架構思維：從軟體工程到 AI 時代的架構師之路」（24h）

涵蓋：軟體工程第一原理、OOP 與程式結構、進階 Python 工程、計算機組織、作業系統、系統設計、架構模式與決策。

兩門課加起來，就是一個完整的「從零到工程師」的 48 小時學習路徑。

拾貳、口訣記憶

> 先看資料，再寫程式；先探索，再分析；先理解，再建模。

拾參、五歲小孩也能懂的版本

把這門課想成做菜：

- M0 是先看食譜，知道今天要做什麼菜
- M1 是認識食材，知道什麼是新鮮的、什麼是壞掉的
- M2 是學會用刀子和鍋子
- M3 是學會精準量測，不再用「一點點」估計
- M4 是學會洗菜、切菜、備料
- M5 是學會擺盤，讓別人也能欣賞你的成果
- M6 是學會試味道，判斷調味夠不夠
- M7 是看看大廚怎麼做，然後決定自己想走哪條路

所以先學 Python 做資料分析，不是在繞路。是在鋪路。

M0：開場 — 為什麼是 Python，為什麼是現在

課程定位：24 小時「Python 資料分析與 AI 入門」第 0 模組 時長：3 小時（含工作坊 60 分鐘）
先修需求：無，但建議已安裝 Python 3.11+

模組定位聲明

這不是一堂 Python 語法課。

這是你進入資料世界的第一道門。語法只是通行證，真正重要的是：你能不能用 Python 思維把資料變成洞察、把數字變成決策、把問題變成可執行的分析。

本模組的任務：建立全局視野，讓你在學完第一行 `import pandas as pd` 之前，就知道自己在哪裡、要去哪裡、為什麼要去那裡。

> "Data is the raw material; Python is the operating language."

> "Good analysis = clean data + right tools + clear questions"

模組學習目標

完成本模組後，學員能夠：

1. 說明 Python 在 2026 年資料科學與 AI 應用領域的主導地位及其背後原因
2. 繪製 Python 資料分析生態系核心工具地圖（從 Jupyter 到 scikit-learn）
3. 說出課程 8 個模組的主要里程碑與內容脈絡
4. 完成本機開發環境建置，並在 Jupyter 中執行第一個資料探索練習
5. 建立「工具是手段，問題求解才是目的」的學習心態

投影片大綱總覽

#	投影片標題	核心訊息	預估時間
S01	你為什麼在這裡	重新定義這門課的性質	10 min
S02	2026 年的 Python 數據報告	用調查數據說話，建立信心	15 min
S03	生態系地圖：從 Jupyter 到 scikit-learn	讓工具有位置感，不再茫然	20 min
S04	里程碑版本：這幾年發生了什麼	技術演進的歷史感	15 min
S05	AI 產品的解剖圖	看清全局：data + code + insights + action	15 min
S06	課程 8 模組路線圖	知道自己在哪條路上、去哪裡	15 min
S07	學習方法論	主動學習 vs. 被動聽課	10 min
S08	你的票，通向未來	收尾的情感錨點	5 min
W01–W04	工作坊：環境建置 + 生態系初探	動手做，讓知識落地	60 min

總計：約 3 小時

詳細投影片規格

S01：你為什麼在這裡

核心訊息：這不是語法課，是進入資料與 AI 世界的入場券。

講師要點： - 開門見山：「如果你只是要學 Python 語法，YouTube 10 小時教學已經夠了。你來這裡是因為你想做更大的事。」 - 點出 2026 年的現實：工作市場中「只會 Python」和「懂 Python 生態系 + 能交付資料分析成果」的價值差距已非常顯著 - 定義課程的性質：這是一門「從資料素養到機器學習入門邊界」的旅程，Python 是語言，目的地是資料驅動的思維與行動能力 - 明確學員終點：24 小時後，你應該能獨立完成一個完整的資料分析流程，並對機器學習的應用場景有全景理解 - 設立心理契約：「這堂課不會讓你成為專家，但會讓你知道專家在解決什麼問題，以及如何繼續往下走」 - 課程定位說明：本課程專注於資料分析與 AI 入門（單一主線）。若你之後想深入軟體工程與系統設計，可接續「系統設計與架構思維」課程

視覺建議： - 全螢幕黑底白字的開場引言：「Data is the raw material; Python is the operating language.」 - 緊接一張對比圖：左側「只學語法的學習者」（停在 for loop 層），右側「資料分析從業者」（能從數據中提取洞察並交付成果），用高度差視覺化技能天花板

轉場：「先告訴我為什麼值得花 24 小時在這裡 — 來看數據怎麼說。」

S01-B：分析成熟度階梯 — 你現在在哪裡，課程帶你到哪裡

核心訊息：資料分析不是只有一種，Gartner 分析成熟度模型把它分成四層。這門課帶你從第一層走到第三層的門口。

講師要點： - Gartner 分析成熟度四階梯： - Level 1 描述性分析（Descriptive）：「發生了什麼？」— 報表、dashboard、KPI 彙總。大部分 Excel 工作停在這裡。 - Level 2 診斷性分析（Diagnostic）：「為什麼發生？」— EDA、下鑽分析、根因探索。本課程 M5/M6 的核心。 - Level 3 預測性分析（Predictive）：「會發生什麼？」— 統計建模、機器學習。本課程 M7 的前導。 - Level 4 規範性分析（Prescriptive）：「該怎麼做？」— 最佳化、決策引擎。屬於進階課程。 - Excel vs Python 效能對比（讓學生直觀感受差距）： - 100 萬筆資料篩選：Excel 3-5 分鐘 / Python pandas

約 3 秒 - 10 張交叉分析表：Excel 手動操作 30 分鐘 / Python groupby + pivot 10 行程式碼 - 重複執行同一份分析：Excel 每次重做 / Python 一行指令重跑整個流程 - 課程目標定位：本課程的 M1-M4 建立 Level 1 能力，M5-M6 建立 Level 2 能力，M7 讓你站在 Level 3 的入口看到全貌 - 關鍵訊息：大部分企業的資料團隊仍卡在 Level 1-2，能做到 Level 2 並看懂 Level 3 已經非常有價值

視覺建議： - 左側：四階梯圖，從下到上標示 Descriptive -> Diagnostic -> Predictive -> Prescriptive，旁邊標注對應的核心問題 - 右側：Excel vs Python 的三組效能對比數字，用色塊對比

轉場：「知道目標在哪裡了，來看 Python 在這個世界的地位。」

S02：2026 年的 Python 數據報告

核心訊息：Python 的主導地位有數據支撐，不是信仰。

講師要點： - JetBrains/PSF 2024 Python Developer Survey：Python 連續多年為最常用語言，資料科學、ML/AI 使用率穩定在前三名，年增率不減 - Stack Overflow Developer Survey 2025：Python 在「最受歡迎語言」與「AI/ML 最常使用語言」雙榜並列第一 - Anaconda State of Data Science 2024：企業採用 Python 作為首要資料平台的比率達 70%+，Jupyter Notebook 仍是最普遍的交互式工作環境 - 一個關鍵洞察：Python 的增長已超越「學習熱潮」，進入「基礎設施鎖定」階段 — 大型企業的 ML pipeline 幾乎全部基於 Python 生態 - 對學員的意義：你選擇的不只是一個流行語言，你選擇的是當前 AI 產業的「作業語言」

視覺建議： - 三份報告的 logo 並排，搭配各自最關鍵的一個數字（如 48% 用 Python 做 ML、#1 最受歡迎等） - 折線圖：2019–2025 Python 在 Stack Overflow 調查中的語言排名趨勢 - 標註框：「里程碑：Python 3.13 + PEP 703 (No-GIL) — 2024 年正式進入標準庫」

里程碑插入點： - 引用 JetBrains/PSF Python Developer Survey 2024（jetbrains.com/lp/devecosystem-2024/python） - 引用 Stack Overflow Developer Survey 2025 - 引用 Anaconda State of Data Science Report 2024

轉場：「數字說服了你。現在讓我們看清楚這個生態系長什麼樣子。」

S03：生態系地圖：從 Jupyter 到 scikit-learn

核心訊息：工具不是孤島，每個工具都有它的位置和職責。這門課帶你走過核心的每一層。

講師要點： - 介紹分層生態系架構，分為四層：互動環境層、資料操作層、分析與視覺化層、模型層 - 互動環境層：Jupyter Notebook / JupyterLab / VS Code — 你思考和實驗的地方；這也是職場資料科學家最常用的工作介面 - 資料操作層：NumPy（數值核心，所有矩陣運算的基礎）、pandas（表格資料操作的業界標準） - 分析與視覺化層：Matplotlib（底層繪圖引擎）、Seaborn（統計圖表的高階介面）、Plotly（互動式圖表） - 模型層：scikit-learn（傳統 ML 的完整工具箱）、Hugging Face（LLM 應用的入口）、PyTorch 概念預覽 - 延伸生態說明：Polars（下一代高效能 DataFrame）、Apache Spark / PySpark（大資料處理）屬於進階路徑 - 強調：這張地圖就是本課程的「目錄」，M1 到 M7 的每個模組都對應地圖上的一個區域

視覺建議： - 核心視覺：一張分層架構圖，用顏色區分四層，每層列出 2-4 個工具名稱加圖標 - 特別標記：哪些工具本課程會深入教（實心圓）、哪些只做概念介紹（空心圓） - 工具關係箭頭：NumPy → pandas → sklearn，NumPy → Matplotlib → Seaborn - 在圖旁加一個標記：「本課程覆蓋範圍」括號，讓學員清楚知道邊界

延伸討論（5 分鐘）： - 請學員思考：在你目前的工作或生活中，哪種資料最常出現？表格？圖片？文字？ - 對應到生態系地圖，讓學員開始建立個人的工具優先序

轉場：「這些工具不是憑空冒出來的。讓我們看它們這幾年發生了什麼巨變。」

S04：里程碑版本：這幾年發生了什麼

核心訊息：Python 生態正在劇烈演進，你學的是 2025 年的版本，不是 2018 年的。

講師要點： - pandas 2.0（2023 年）：底層切換至 Apache Arrow 記憶體格式，Copy-on-Write 語意改變，效能大幅提升 — 很多舊教程的程式碼在 2.0 下有 `FutureWarning`，要注意 - NumPy 2.0（2024 年）：第一個重大版本，API 清理，dtype 行為更一致，部分舊程式碼需要修改 - scikit-learn 1.4–1.5（2023–2024 年）：Pipeline 介面改進、更好的 metadata routing、新增多個轉換器，讓 ML 工作流更簡潔 - Python 3.12/3.13 + PEP 703（2024 年）：實驗性 Free-threaded（No-GIL）模式，3.13 正式版含 JIT 編譯器實驗旗標 - Jupyter / JupyterLab 4.x（2023–2024 年）：效能大幅改善、更好的 real-time 協作、extension 生態更穩定 - 對學員的意義：「你不是在學一個靜態工具，你在追一個快速行進的生態。學會怎麼看 changelog 比背 API 更重要。」

視覺建議： - 時間軸（2022–2026），標記各大版本釋出日期 - 每個里程碑用一行黃底文字框強調最重要的一個變化 - 最後加一個「預告框」：pandas 3.0 / NumPy 2.1+ / scikit-learn 2.0 預計方向

里程碑插入點： - pandas 2.0 Migration Guide（pandas.pydata.org/docs/whatsnew/v2.0.0.html） - NumPy 2.0 Release Notes - scikit-learn 1.4 What's New（scikit-learn.org/stable/whats_new/v1.4.html） - PEP 703 — Making the Global Interpreter Lock Optional

轉場：「知道工具在哪裡、發展到哪裡。現在問最核心的問題：資料分析與 AI 應用的全景是什麼？」

S05：AI 產品的解剖圖

核心訊息：從資料到洞察再到行動，Python 是貫穿全局的語言。初學者先紮穩前兩層。

講師要點： - 提出核心公式：資料 → 清洗 → 分析 → 視覺化 → 模型 → 洞察 → 行動 - 資料收集與清洗層：原始資料的讀取、清洗、轉換 — pandas / NumPy 的主場 - 分析與視覺化層：探索性分析（EDA）、統計摘要、圖表輸出 — Matplotlib / Seaborn / Plotly - 建模與預測層：特徵工程、模型訓練、評估、解釋 — scikit-learn 的核心場景 - 部署與應用層：模型服務化、API 端點、自動化報告 — FastAPI、Streamlit、MLflow（進階路徑） - 重要洞察：大多數資料分析工作 80% 的時間花在前兩層（資料清洗與 EDA），這也是本課程的核心覆蓋範圍 - 對學員的意義：本課程從 M1 到 M7 系統地帶你通過這條路，終點是具備獨立完成端到端分析的能力 - 課程邊界說明：部署與大型系統工程屬於「系統設計與架構思維」課程的範疇

視覺建議： - 線性流程圖（左到右）：資料 → 清洗 → 分析 → 視覺化 → 建模 → 洞察 → 行動 - 每個節點下標注代表工具 - 用括號標記「本課程覆蓋範圍：清洗 → 建模」，其餘部分用淡色表示

轉場：「你現在知道地圖了。讓我告訴你這 8 個模組如何帶你走過這張地圖。」

S06：課程 8 模組路線圖

核心訊息：每個模組都有明確的任務，你知道每一步要去哪裡。

講師要點：

- M0（3h）：開場 — 為什麼是 Python，為什麼是現在
- 建立全局視野，完成環境建置，建立學習心態
- M1（3h）：資料思維與 Notebook 工作流
- 學會用 Jupyter 思考、資料類型與問題框架、第一次 EDA 體驗
- M2（3h）：Python 基礎 — 資料分析必用語法
- 精要語法（不是全部）：list / dict / 函數 / 條件 / 迴圈，以「資料操作」為導向
- M3（3h）：NumPy — 數值計算的核心
- 陣列操作、廣播規則、向量化計算思維，為 pandas 和 ML 打基礎
- M4（3h）：pandas — 表格資料的瑞士刀
- DataFrame 操作、資料清洗、groupby、merge，核心使用模式
- M5（3h）：資料視覺化 — 讓數字說話
- Matplotlib 基礎 + Seaborn 統計圖表 + 好圖的設計原則
- M6（3h）：統計思維與探索性分析
- 描述統計、分佈直覺、相關性、假設驗證的資料導向方法
- M7（3h）：機器學習前導 — scikit-learn 入門
- ML 工作流（train/test split → 訓練 → 評估）、常見監督學習算法直覺、Pipeline 概念

課程設計說明： - 這是一條單一主線，專注於「資料分析 → AI 入門」 - 每個模組 3 小時，總計 24 小時 - 想深入軟體工程、系統架構的學員，可接續「系統設計與架構思維」課程

視覺建議： - 橫向時間軸，8 個模組等距排列，每格標注模組名稱與時長 - 用顏色區分三階段：基礎期 M0–M2（綠）、核心工具期 M3–M5（藍）、分析與建模期 M6–M7（紫） - 在 M2 和 M5 標記「能力里程碑」檢查點，說明達到後能做到什麼

轉場：「路線圖清楚了。最後，我要談談怎麼學，才能真的學到。」

S07：學習方法論

核心訊息：主動學習者比被動聽課者的知識留存率高 6 倍。

講師要點： - 費曼技巧應用：每個模組結束後，用自己的話解釋一個概念給假想的同事聽。如果解釋不清楚，就是還沒真正學 - 最小可行程式（MVP Code）原則：先讓程式能跑，再讓它跑得好，再讓它跑得快 — 這個順序不能倒 - 錯誤是資料，不是失敗：debug 的過程就是學習的過程，保留你的錯誤紀錄 - 工作坊 > 練習題：每個模組的動手時間佔比超過 30%，不要跳過 - 搜尋策略：遇到問題先看官方文檔，再看 Stack Overflow，最後才問 AI — 這個順序建立真正的理解，而不只是得到答案 - 建立學習日誌：建立一個 `learning_notes/` 目錄，每個模組存一個 `.ipynb` 記錄你的問題和發現。24 小時後，這個目錄本身就是你能力成長的證明

視覺建議： - 「主動 vs. 被動學習」對比圖，右側列出本課程採用的主動學習手段（工作坊、反思紀錄、問題解決） - 學習效率金字塔（Learning Pyramid），標記各種學習方式的知識留存率 - 搜尋策略流程圖：官方文檔 → Stack Overflow → AI 助手，每個節點說明適合什麼情況使用

轉場：「方法論在手，最後一張投影片 — 也是最重要的一張。」

S08：你的票，通向未來

核心訊息：你不是在學一個工具，你在取得一張進入資料與 AI 世界的門票。

講師要點： - 回到開場引言：「Data is the raw material; Python is the operating language.」 - 點題：2026 年，能夠用 Python 操作資料、探索規律、建立模型的人，擁有在資料驅動的職場中創造真實價值的能力 - 這不是誇大其詞 — 這是當前就業市場和產品開發現實的直接反映 - 課程承諾：24 小時之後，你將擁有獨立完成端到端資料分析的能力地基，以及知道在哪裡繼續深挖的地圖 - 對想走得更深的學員：「資料分析是你的起點，不是終點。系統設計、工程化部署、架構思維是下一條路，我們 - 結語留一個問題給學員思考：「你想用 Python 解決的第一個真實資料問題是什麼？」

視覺建議： - 全螢幕極簡設計：深色背景，中央一行大字「你不是在學工具，你在取得通往未來的入場券。」 - 底部小字：課程 GitHub repo 連結 + 社群討論頻道 + M1 預習資源

工作坊 / 練習段落

時長：60 分鐘

工作坊標題：環境建置 + 生態系初探 + 個人學習契約

W01：建立你的學習環境（15 分鐘）

目標：確保所有學員的開發環境統一可用，消除後續模組的環境障礙

```
# [redacted] Python [redacted]
python --version  # [redacted] 3.11 [redacted]

# [redacted]
python -m venv .venv
source .venv/bin/activate  # macOS/Linux
[redacted] .venv\Scripts\activate  # Windows

# [redacted]
pip install jupyter pandas numpy matplotlib seaborn scikit-learn plotly

# [redacted]
python -c "import pandas as pd; print(f'pandas {pd.__version__}')"
python -c "import numpy as np; print(f'numpy {np.__version__}')"
python -c "import sklearn; print(f'scikit-learn {sklearn.__version__}')"

# [redacted] JupyterLab
jupyter lab
```

驗收標準：能成功印出三個套件版本號，JupyterLab 在瀏覽器中正常開啟

常見問題處理： - Windows 路徑問題：建議使用 WSL2 或 Anaconda Prompt - pip 版本過舊：`python -m pip install --upgrade pip` - Port 已佔用：`jupyter lab --port 8889`

W02：生態系地圖的第一次探索（20 分鐘）

目標：用程式碼驗證你理解工具的位置，建立對生態系的第一手感受

在 JupyterLab 中建立一個新 notebook，命名為 `M0_ecosystem_map.ipynb`。

任務一：匯入並確認版本（5 分鐘）

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn

print("=== [redacted] ===")
print(f"pandas      : {pd.__version__}")
print(f"numpy       : {np.__version__}")
print(f"matplotlib  : {plt.matplotlib.__version__}")
print(f"seaborn     : {sns.__version__}")
print(f"scikit-learn: {sklearn.__version__}")
```

任務二：生態系定位練習（10 分鐘）

建立一個 Python `dict`，key 是工具名稱，value 是「用一句話說明這個工具做什麼、在哪一層」。不要複製範例，用你自己的理解寫。

```
# [redacted]
ecosystem = {
    "numpy": "...",          # [redacted]
    "pandas": "...",         # [redacted]
    "matplotlib": "...",     # [redacted]
    "seaborn": "...",        # [redacted]
    "scikit-learn": "...",   # [redacted]
}

print("=== [redacted] ===")
for tool, description in ecosystem.items():
    print(f"{tool:15s}: {description}")
```

任務三：第一張真實圖表（5 分鐘）

用幾行程式碼畫出你的第一張圖，感受「資料 → 視覺」的過程：

```
import numpy as np
import matplotlib.pyplot as plt

# [redacted]
np.random.seed(42)
scores = np.random.normal(loc=72, scale=15, size=100)

plt.figure(figsize=(8, 4))
plt.hist(scores, bins=20, color='steelblue', edgecolor='white', alpha=0.8)
plt.axvline(scores.mean(), color='red', linestyle='--', label=f'[redacted]{scores.mean():.1f}')
plt.title('[redacted]')
plt.xlabel('[redacted]')
plt.ylabel('[redacted]')
plt.legend()
```

```
plt.tight_layout()
plt.show()
[redacted]
print(f"[redacted]{scores.max():.1f}")
print(f"[redacted]{scores.min():.1f}")
print(f"[redacted]{scores.std():.1f}")
```

討論問題（全班）： - 你在圖表中觀察到了什麼？這個分佈「正常」嗎？ - 如果你是老師，你會根據這張圖做什麼決定？

W03：你的第一次資料探索（15 分鐘）

目標：用真實資料集體驗完整的「讀取 → 觀察 → 提問」流程

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
[redacted]
# [redacted] seaborn [redacted] tips [redacted]
tips = sns.load_dataset("tips")
[redacted]
# [redacted]
print("=== [redacted] ===")
print(f"[redacted]{len(tips)}[redacted]")
print(f"[redacted]{list(tips.columns)}[redacted]")
print()
print(tips.head())
print()
print(tips.describe())
```

引導提問（讓學員先思考，再執行）：

- 「你認為小費金額和帳單總額之間有關係嗎？」 `python # [redacted] plt.figure(figsize=(7, 5)) sns.scatterplot(data=tips, x="total_bill", y="tip", hue="sex", alpha=0.7) plt.title("[redacted] vs. [redacted]") plt.show()`
- 「午餐和晚餐的小費習慣有差異嗎？」 `python # [redacted] boxplot [redacted] plt.figure(figsize=(6, 4)) sns.boxplot(data=tips, x="time", y="tip", palette="pastel") plt.title("[redacted] vs. [redacted]") plt.show()`

反思問題：你從這兩張圖中發現了什麼？你還想再問資料哪些問題？

W04：學習契約 — 你的起點紀錄（10 分鐘）

目標：把這一刻的狀態記錄下來。24 小時後，這份記錄會讓你看見自己走了多遠。

在 `M0_ecosystem_map.ipynb` 的最後幾個 cell 中，寫下：

```
# --- [redacted] ---

learning_contract = {
    "today_date": "2026-xx-xx",  # [redacted]

    "real_problem": """
    # [redacted] Python [redacted]
    # [redacted]
    """,

    "most_anticipated_module": """
    # [redacted]
    """,

    "biggest_learning_concern": """
    # [redacted]
    """,

    "current_python_level": """
    # [redacted]1-5 [redacted] Python [redacted]
    # 1 = [redacted]  2 = [redacted]  3 = [redacted]
    # 4 = [redacted]  5 = [redacted]
    """,
}

print("--- [redacted] ---")
for key, value in learning_contract.items():
    print(f"\n[{key}]")
    print(value.strip())
```

重要：儲存這個 notebook。在 M7 的最後一個工作坊，我們會回來看這份契約，對照 24 小時前後的自己。

里程碑 / 引用插入點

以下資源應在對應投影片中引用，確保技術陳述有據可查：

資源	對應投影片	引用重點
JetBrains/PSF Python Developer Survey 2024	S02	Python 使用率、ML/AI 場景佔比
Stack Overflow Developer Survey 2025	S02	最受歡迎語言排名、AI 工具使用統計
Anaconda State of Data Science Report 2024	S02	企業 Python 採用率、Jupyter 使用率
pandas 2.0 What's New	S04	Copy-on-Write、Arrow 後端
NumPy 2.0 Release Notes	S04	API 清理、dtype 行為變更
scikit-learn 1.4 What's New	S04	Pipeline 改進、metadata routing
JupyterLab 4.x Release Notes	S04	效能改善、協作功能
PEP 703 — Making GIL Optional	S04	Free-threaded Python 的意義
Python 3.13 Release Notes	S04	JIT 實驗旗標、PEP 703 實驗性支援

講師備注

常見學員問題預備

Q：我需要先學 R 還是 Python？ A：2026 年的資料科學職缺，80% 以上要求 Python。R 在學術統計領域仍有地位，但如果你的目標是資料分析職位或 AI 相關工作，Python

優先沒有疑問。

Q：Python 這麼慢，為什麼不用 Rust 或 Go？ A：Python 本身慢，但 NumPy、pandas、PyTorch 的核心都是 C/C++/CUDA 寫的。你用 Python 呼叫它們，速度不輸原生語言。Python 是「膠水語言」，黏合高效能元件。

Q：我沒有數學背景，能學 ML 嗎？ A：本課程 M0–M6 完全不需要線性代數或微積分。M7 的機器學習入門部分會用直覺先建立理解，數學推導是可選深入路徑，不是必要條件。

Q：這門課學完之後，下一步是什麼？
A：有兩條路。一是繼續深化資料科學（統計學習、深度學習、大資料工程），二是轉向軟體工程與系統設計（我
AI 入門，但它是一個清晰的起點。

時間控制提示

- S01–S02 最容易超時，學員有很多問題。預留 5 分鐘 buffer。
- S03（生態系地圖）建議做延伸討論，讓學員說出自己的工具認知，這有助於後續模組的情境連結。
- S05（AI 產品解剖圖）是全場情緒最高點，不要急著跳過討論。
- W01（環境建置）常遇到 Windows 路徑問題，建議助教提前準備 troubleshooting checklist。
- W03（tips 資料探索）是本工作坊最有互動潛力的段落，可以根據時間彈性延伸討論。

參考文獻與引用來源

產業調查與報告

1. JetBrains/PSF Python Developers Survey 2024 — Python 使用場景、社群規模與技術趨勢的年度調查。lp.jetbrains.com/python-developers-survey-2024
2. Stack Overflow Developer Survey 2025 — 全球最大開發者調查，涵蓋語言流行度、AI 工具採用率。survey.stackoverflow.co/2025
3. Anaconda State of Data Science Report 2024 — 企業 AI 採用率、資料工作者工具偏好與挑戰。anaconda.com/resources/report/state-of-data-science-report-2024

分析框架

1. Gartner Analytic Ascendancy Model — 分析成熟度四階梯（描述性 / 診斷性 / 預測性 / 規範性）。Gartner, Inc. 原始框架，廣泛引用於企業數據策略文獻。參見 digital.ai/catalyst-blog/gartners-analytics-maturity-model

效能對比數據

1. Excel vs Python 效能對比 — 本課程引用的「100 萬筆資料篩選：Excel 3-5 分鐘 vs pandas 約 3 秒」為教學用估算值，基於典型桌機環境（16GB RAM, SSD）的實測經驗。精確數值依硬體與資料結構而異。

工具版本里程碑

1. pandas 2.0 Release Notes (April 2023) — Arrow backend、Copy-on-Write 語意。pandas.pydata.org/docs/whatsnew/v2.0.0.html
2. NumPy 2.0 Release Notes (June 2024) — 自 2006 年 NumPy 1.0 以來首個 major release，約 100 個已棄用 API 被移除。numpy.org/doc/stable/release/2.0.0-notes.html
3. scikit-learn 1.0 Release (September 2021) — 始於 2007 年 Google Summer of Code，1.0 標誌 API 穩定化里程碑。scikit-learn.org/stable/whats_new/v1.0.html
4. PyTorch 2.0 Release (March 2023) — `torch.compile()` 在 A100 GPU 上平均加速 43%。pytorch.org/get-started/pytorch-2-x
5. Python 3.13 What's New (October 2024) — 實驗性 free-threaded 模式（PEP 703）。docs.python.org/3/whatsnew/3.13.html

工作坊節奏建議

工作坊段落	建議時長	節奏提示
W01 環境建置	15 min	以助教協助為主，講師巡場
W02 生態系探索	20 min	任務一、二獨立完成；任務三可帶著做

W03 資料探索	15 min	先讓學員自行觀察，再引導討論
W04 學習契約	10 min	靜默寫作，不討論，個人儀式感

模組總結

本模組完成了四件事：

1. 建立視野：你知道 Python 在資料與 AI 世界的地位不是偶然，有數據支撐
2. 建立地圖：你知道生態系的工具分層和它們的職責，以及這門課覆蓋哪些範圍
3. 建立環境：你的開發環境已就緒，第一個 notebook 已經能跑起來
4. 建立期望：你知道這 24 小時要帶你去哪裡、用什麼方法帶你去，以及之後的路在哪裡

下一模組（M1：資料思維與 Notebook 工作流）從「如何用資料回答問題」開始 — 不是「從頭教語法」，而是「建立用資料思考的習慣，並學會把 Jupyter 當成你的思考介面」。

本模組屬於「Python 資料分析與 AI 入門」課程體系。 課程版本：v2.0 | 最後更新：2026 年

M1：資料思維與 Notebook 工作流

課程定位： 第二幕 — 建立資料思維底盤，掌握分析師的基礎工作環境 時數： 3 小時 前置模組： M0 開場全景 後續模組： M2 Python 分析基礎

模組學習目標

完成本模組後，學生能夠：

1. 說出「資料分析工作」的真實樣貌，區分清理、探索、推論三個核心動作
2. 識別結構化資料的五個基本元素：列、欄、綱要、型態、鍵
3. 判斷一份資料是否「乾淨」，說出四類常見污染並提出初步處理策略
4. 解釋 Excel 思維與 Python 思維的根本差異，以及選擇時機
5. 說明 Jupyter Notebook 的四合一設計哲學，並在實際環境中操作
6. 辨識常見資料格式（CSV、JSON、SQL 表格、API 回應）的結構特徵
7. 套用資料品質評估框架，對一份新資料進行系統性的健康檢查
8. 使用 Notebook 把分析過程組織成有推論脈絡的敘述，而不只是程式碼輸出

關鍵概念清單

完成模組後，以下每個概念都應能一句話解釋：

- [ ] 資料分析的三個核心動作：清理 / 探索 / 推論
- [ ] 結構化資料：列（觀測值）與欄（變數）
- [ ] 綱要（schema）：欄名 + 型態的完整描述
- [ ] 鍵（key）：唯一識別一筆記錄的欄位

- [ ] 缺失值（missing value）：`NaN`、空字串、假值的差異
- [ ] 重複值（duplicate）：完全重複 vs. 邏輯重複
- [ ] 離群值（outlier）：異常的數值，不一定是錯誤
- [ ] 型態錯誤（type error）：數字被存成字串的情況
- [ ] 資料品質評估框架：完整性 / 一致性 / 準確性 / 時效性
- [ ] CSV：純文字的表格格式，欄位以逗號分隔
- [ ] JSON：鍵值結構，巢狀資料的標準格式
- [ ] SQL 表格：關聯式資料庫的儲存單位，有嚴格型態定義
- [ ] API 回應：動態取得的資料，通常為 JSON 結構
- [ ] Excel 思維：手動操作，難以重現
- [ ] Python 思維：可重現工作流，腳本即文件
- [ ] Jupyter Notebook：文字 + 程式 + 圖表 + 推論四合一
- [ ] 資料分析生命週期：提問 -> 取得 -> 清理 -> 探索 -> 推論 -> 呈現

投影片大綱

#	投影片標題	核心訊息	預估時間
S01	你以為的資料分析 vs. 實際在做什麼	資料分析不是「畫圖表」，是清理 + 探索 + 推論的完整循環	12 min
S02	資料分析生命週期：六個階段	從提問到呈現，每個階段都有明確任務和輸出	12 min
S03	結構化資料的五個元素	列、欄、綱要、型態、鍵，看懂這五個才能開始工作	15 min
S04	什麼叫做「乾淨的資料」	四種污染類型：缺失、重複、離群、型態錯誤，各有處理邏輯	15 min

S05	資料品質評估框架	完整性、一致性、準確性、時效性，系統性而不是憑感覺	12 min
S06	認識常見資料格式	CSV、JSON、SQL 表格、API 回應，各自的結構特徵與使用場景	15 min
S07	Excel 思維 vs. Python 思維	Excel 是手動操作；Python 是可重現的流程腳本	12 min
S08	為什麼 Jupyter Notebook 成為標準	文字 + 程式 + 圖表 + 推論四合一，分析過程可被閱讀與重現	12 min
S09	Notebook 工作流實戰技巧	cell 管理、Markdown 撰寫、執行順序、輸出管理的最佳實踐	15 min
S10	工作坊 A：初探一份真實資料	用 Notebook 的標準工作流，對一份新資料進行系統性探索	30 min
S11	工作坊 B：資料品質健康檢查	套用品質評估框架，找出資料問題並記錄在 Notebook 中	30 min
S12	模組總結：分析師的思維工具箱	資料思維 + Notebook 工作流是所有後續分析技術的地基	10 min

總計：約 190 分鐘（含工作坊與 Q&A）

詳細投影片內容

S01 — 你以為的資料分析 vs. 實際在做什麼

核心訊息： 資料分析不是「畫圖表」，是清理 + 探索 + 推論的完整循環。圖表只是最後的呈現，不是工作的主體。

講師講解要點： - 多數初學者以為資料分析 = 做 dashboard / 做折線圖。這是誤解。圖表是最末一步，前面有大量清理和判斷工作。 - 資料分析師的實際時間分配：業界調查顯示約 60-80% 時間花在資料清理，而非建模或視覺化。 - 三個核心動作定義： - 清理：讓資料可信。沒有這一步，後續所有結果都可能是錯的。 - 探索：找出值得問的問題。資料裡有什麼樣的分布？有沒有異常？有沒有意外的相關性？ - 推論：用資料支撐決策。這裡的「推論」不必然是統計推論，也包括業務判斷：「這個指標下降，是因為活動結 - 強調「推論」需要業務背景，純技術人員容易在這一步出問題：數字算出來了，但解讀錯了。 - 這門課從 M1 到 M6 都在訓練這條鏈，從資料思維和 Python 基礎出發，不是從圖表出發。

視覺建議： 兩欄對比圖。左欄「你以為的流程」：資料 → 圖表 → 結論（三步，箭頭乾淨）；右欄「實際流程」：原始資料 → 清理（缺失 / 重複 / 型態）→ 探索（分布 / 相關 / 異常）→ 推論（假設 / 驗證 / 決策）→ 呈現。用多個循環箭頭暗示這是迭代而非線性。右欄中特別標注「60-80% 時間在這裡」指向清理階段。

課堂互動： 問學員：「你們有沒有用 Excel 做過資料分析？最花時間的步驟是什麼？」讓 2-3 位學員回答，帶出清理工作的普遍性。

過渡語：
「要能做好這三件事，我們需要一個系統性的框架，不能每次拿到資料都從零開始想。這個框架叫做資料分析生

S02 — 資料分析生命週期：六個階段

核心訊息：
一個完整的資料分析任務包含六個階段：提問、取得、清理、探索、推論、呈現。每個階段都有明確的輸入、任

講師講解要點：

第一階段：提問（Define the Question） - 資料分析最重要的一步，也最常被跳過。 - 好問題的特徵：可以用資料回答、有業務意義、結果能影響決策。 - 壞問題的例子：「分析銷售資料」（太廣泛，無法知道什麼時候算「分析完了」）。 - 好問題的例子：「上個月北部地區的退貨率是否高於全國平均，以及退貨最集中的商品類別是哪三個。」

第二階段：取得（Acquire Data） - 來源多樣：CSV 文件、資料庫查詢、API 呼叫、網頁爬蟲、人工收集。 - 取得時就要記錄：資料來源、時間範圍、已知限制。 -

常見陷阱：取得的資料不是你真正需要的（欄位不對、時間範圍不對、抽樣偏差）。

第三階段：清理（Clean） - 處理缺失、重複、離群、型態錯誤（S04 詳細說明）。 - 清理決策要記錄：為什麼刪掉這些行？為什麼用均值填補？這些決策會影響最終結論。

第四階段：探索（Explore） - 又稱 EDA（Exploratory Data Analysis）。 - 看分布（每個欄位的數值範圍是什麼）、看相關（哪些欄位之間有關係）、看異常（有沒有不符預期的模式）。 - 探索的目的是找出值得深入的方向，不是給出結論。

第五階段：推論（Analyze / Infer） - 用統計方法或業務邏輯，把探索發現轉化為可支撐決策的結論。 - 注意：「相關性不等於因果性」是這個階段最重要的警惕。

第六階段：呈現（Communicate） - 面向不同受眾，呈現方式不同：給技術人員看 Notebook，給管理層看簡報，給系統用 API。 - 好的呈現不是「圖表漂亮」，而是「讓決策者能正確理解並行動」。

視覺建議：
六個色塊排成一排的生命週期流程圖，每個色塊包含：階段名稱（中英文）、輸入、關鍵任務（一行）、輸出。↔ 探索之間特別加粗，代表經常來回）。

過渡語： 「生命週期的第一件實際工作，是讀懂資料本身的結構。一份資料，不管是 CSV 還是資料庫，它背後都有五個基本元素。」

S03 — 結構化資料的五個元素

核心訊息：
每一份結構化資料都可以用列（row）、欄（column）、綱要（schema）、型態（type）、鍵（key）來完整描

講師講解要點： - 以一張真實感的訂單資料表為例（訂單 ID / 客戶名稱 / 金額 / 日期 / 狀態），帶出五個元素。

列（row）= 一筆觀測值 - 代表「一件事發生了」。一列訂單 = 一筆交易發生。 - 列的數量 = 這份資料記錄了多少件事。 - 在 pandas 中，`df.shape[0]` 就是列數。

欄（column）= 一個變數 - 描述這件事的一個面向。金額是一個欄，日期是另一個欄。 - 欄的數量 = 我們對每件事記錄了多少個面向。 - 在 pandas 中，`df.columns` 列出所有欄名。

綱要（schema）= 欄名 + 型態的完整定義 - 是資料的「藍圖」。換一份新資料，第一件事是看 schema。 - 例子：`order_id: int, customer_name: str, amount: float, order_date: datetime, status: str`。 - 在 pandas 中，`df.dtypes` 顯示每欄的型態（近似 schema）。

型態（type）= 每個欄的資料類型 - 整數（int）、浮點數（float）、字串（str）、日期（datetime）、布林（bool）。 - 型態決定能做哪些運算：數字型可以加減乘除，字串型可以搜尋替換，日期型可以計算時間差。 - 型態不對是資料問題的重大來源（S04 詳細說明）。

鍵（key）= 唯一識別一筆記錄的欄位 - 通常是 ID 類欄位，保證每一列都是獨立的、不重複的記錄。 - 沒有鍵或鍵不唯一：合併兩份表格時會出現笛卡爾積爆炸（一列變成多列）。 - 自然鍵 vs. 代理鍵：`■■ ID` 是代理鍵（系統生成），`■■■■■` 是自然鍵（現實世界存在）。

實際操作示範：

```
import pandas as pd

df = pd.read_csv("orders.csv")

# ■■■■■■tuple: ■■, ■■■
print(df.shape)

# ■ schema■■■ + ■■■
print(df.dtypes)

# ■■■■
df.head()

# ■■■■■■■■
print(df["order_id"].nunique() == len(df))  # True ■■ order_id ■■■■■■
```

視覺建議：
一張標記過的資料表格截圖，用不同顏色箭頭標出：橫向掃一列（標「一筆觀測值」）、縱向掃一欄（標「一個 ID 欄標「key」、欄標題下方標型態。旁邊對應 pandas 指令。

過渡語：
「看得懂結構，下一步是判斷資料有沒有問題。大部分初學者拿到資料就開始分析，但乾淨的資料才是分析的前

S04 — 什麼叫做「乾淨的資料」

核心訊息：
污染資料有四種形式：缺失值、重複值、離群值、型態錯誤。識別這四類是清理工作的第一步，每一類都有不同

講師講解要點：

缺失值（missing value） - 欄位沒有值。在 Python/pandas 中通常是 `NaN`（Not a Number）。 - 重要區分：空字串 `""` 和假值 `0` 在有些系統中被用來表示「沒有資料」，但它們不是 `NaN`，處理方式不同。 - 三種處理策略： - 刪除：缺失比例極高（> 50%），或缺失本身就代表「這筆記錄無效」。 - 填補：用均值、中位數、眾數，或用業務規則填補（缺失的退貨率可能代表 0）。 - 保留：缺失本身有意義，代表「沒有發生」，不應該刪掉。

```
# 看哪欄有多少缺失
df.isna().sum()

# 缺失比例
df.isna().mean() * 100
```

重複值（duplicate） - 完全重複：整列所有欄位都相同，通常是匯入錯誤（同一批資料被匯入兩次）。 - 邏輯重複：同一個客戶用不同拼法出現兩次（`Alice Wang` vs. `alice wang`），這更難自動抓，需要人工或模糊比對。

```
# 看有幾筆重複
print(df.duplicated().sum())

# 移除重複
df_clean = df.drop_duplicates()
```

離群值（outlier） - 極端的數值。注意：離群值不一定是錯誤。 - 判斷邏輯：一位真實的頂級客戶可能消費是平均值的 100 倍，這是真實的業務現象，不是錯誤。 - 需要先判斷「是資料輸入問題」還是「是真實的業務現象」，再決定是否處理。 - 常見判斷方式：統計描述（`.describe()`）+ 分布圖（M4 會教）。

型態錯誤（type error） - 金額欄位被存成字串 `"$1,200"`、日期被存成 `"2024/01/15"` 字串而非 datetime。 - Python 計算時會報錯，或更糟，默默產生錯誤結果（字串相加 `"1" + "2" = "12"` 不是 3）。 - 清理時要明確轉換型態，並處理轉換失敗的情況。

```
# 金額欄位清理
df['amount'] = df
```

```
    df["amount"]
    .astype(str)
    .str.replace(r"[$,]", "", regex=True)
    .astype(float)
```

清理的目的：

不是讓資料「好看」，是讓分析「可信」。沒有清理就直接分析，結論可能是完全錯誤的。

視覺建議：

四格圖，每格對應一種污染類型，用同一張訂單表格呈現，高亮出問題儲存格：缺失值用灰底、重複列用黃底標

過渡語：

「面對一份新資料，需要有系統性的檢查方法，不能只憑直覺。下一張投影片介紹一個可以反覆套用的評估框架

S05 — 資料品質評估框架

核心訊息：

系統性評估資料品質，用完整性、一致性、準確性、時效性四個維度，取代憑感覺的「看一看」，讓每次資料檢

講師講解要點：

完整性（Completeness） - 問題：資料應該有的記錄，都在嗎？ - 檢查方向： - 欄位層面：每欄的缺失比例是多少？哪些欄位的缺失是不可接受的？ - 記錄層面：預期應該有 30 天的資料，結果只有 28 天，哪兩天不見了？ - 覆蓋層面：這份資料只有「成功訂單」，「取消訂單」呢？

```
[illegible]
def check_completeness(df):
    missing_pct = df.isna().mean() * 100
    print("=== [illegible] ===")
    for col, pct in missing_pct.items():
        status = "OK" if pct == 0 else f"WARNING: {pct:.1f}% missing"
        print(f"  {col}: {status}")
```

一致性（Consistency） - 問題：同一個事實，在不同地方的記錄一致嗎？ - 常見問題： - 同一個人名在不同資料集有不同拼法（`alice` / `Alice` / `ALICE`）。 - 同一個金額在不同欄位記錄不一致（`amount = 1200` 但 `items × price = 1300`）。 - 日期格式不統一（`2024-01-15` vs. `15/01/2024`）。

準確性（Accuracy） - 問題：記錄的值，符合現實嗎？ - 這是最難自動化的一個維度，通常需要業務知識輔助。 - 常見問題：年齡欄位出現 200 歲、出生日期晚於購買日期、退貨率 > 100%。

時效性（Timeliness） - 問題：資料是否足夠新，能用來回答當前問題？ - 舉例：用三年前的顧客行為資料來預測現在的購買意願，可能已過時。 - 檢查方向：資料的最新更新時間、是否有資料延遲的已知問題。

套用框架的標準輸出：
每次拿到新資料，應該產出一份「資料品質摘要」，包含：四個維度的評估結果、已知問題列表、建議的清理步 Notebook 的開頭。

視覺建議： 四個維度的評分卡格式，每個維度包含：維度名稱 + 核心問題 + 檢查方法 + 常見問題例子。底部加上「輸出：資料品質摘要」方框。

過渡語： 「框架告訴你要看什麼，但資料本身有不同的長相。一份 CSV 和一份 JSON 和一個資料庫表格，結構上有什麼異同？」

S06 — 認識常見資料格式

核心訊息： 資料分析師會遇到四種主要資料格式：CSV、JSON、SQL 表格、API 回應。每種格式有其設計用途和結構特徵，讀懂格式是取得資料之後的第一步。

講師講解要點：

CSV（Comma-Separated Values） - 最簡單的表格格式。純文字，每行是一列記錄，欄位用逗號（或 Tab）分隔。 - 優點：任何工具都能讀，人可以直接閱讀。 - 缺點：沒有型態資訊（所有欄位都是字串），沒有架構定義，大檔案效能差。 - 使用場景：資料匯出 / 匯入、小規模資料交換、Excel 的通用格式。

```
import pandas as pd

df = pd.read_csv("data.csv")
# 指定型態，避免 pandas 猜錯
df = pd.read_csv("data.csv", dtype={"amount": float, "order_id": str})
```

JSON（JavaScript Object Notation） - 鍵值對結構，支援巢狀（nested）資料。 - 資料範例：

```
{
  "order_id": "ORD-001",
  "customer": {
    "name": "Alice Wang",
    "tier": "premium"
  },
  "items": [
    {"product": "A", "qty": 2, "price": 150},
    {"product": "B", "qty": 1, "price": 300}
  ]
}
```

- 優點：支援複雜的巢狀結構，是 API 回應的標準格式。
- 挑戰：巢狀結構需要「展平（flatten）」才能放進 pandas 的二維表格。

```
import json
import pandas as pd

with open("orders.json") as f:
    data = json.load(f)

# 展平巢狀結構
df = pd.json_normalize(data)
```

SQL 表格 - 存在關聯式資料庫中（MySQL、PostgreSQL、SQLite 等）。 - 特點：有嚴格的型態定義（建立時就指定），有索引和外鍵關係，支援大規模資料。 - 分析師通常用 SQL 查詢先篩選資料，再用 pandas 做細部分析。

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("sales.db")
# 用 SQL 查詢，結果直接變成 DataFrame
df = pd.read_sql_query("SELECT * FROM orders WHERE status = 'completed'", conn)
conn.close()
```

API 回應 - 透過 HTTP 請求動態取得的資料，通常為 JSON 格式。 - 與靜態檔案的差異：資料是即時的，但有速率限制（rate limit），可能需要分頁（pagination）。 - 取得 API 資料後，通常先轉成 pandas DataFrame 再分析。

```
import requests
import pandas as pd

response = requests.get("https://api.example.com/sales?date=2024-01-01")
```

```
data = response.json()  # [{...}, ...] list of dicts
df = pd.DataFrame(data)
```

格式選擇指南（簡表）：

場景	建議格式
小資料，需要人可讀	CSV
複雜巢狀結構、API 交換	JSON
大規模資料、需要查詢	SQL
即時資料、第三方服務	API（JSON）

視覺建議：
四個並排方框，每個框代表一種格式，包含：格式名稱、簡短範例（程式碼片段）、優點、限制、讀入 pandas 的方式。

過渡語：「看懂了資料的格式和結構，接下來要談工具的選擇。同樣的任務，Excel 和 Python 的做法有什麼根本差異？」

S07 — Excel 思維 vs. Python 思維

核心訊息：Excel 是「手動操作」，Python 是「可重現的工作流」。前者快速直觀，後者需要多寫一點，但腳本本身就是文件，也是自動化的起點。選擇哪

講師講解要點：

Excel 的核心問題不是「不好用」 - Excel 的問題是「操作不留痕跡」。 - 你按了幾十個步驟刪除了某些行、改了某些格式、套用了某個公式，這些動作沒有被記錄下來。 - 下週來一批新資料，你必須重頭做一遍，而且你可能記不清自己上次做了什麼。 - 更嚴重的問題：別人無法驗證你做了什麼，你的結論的可信度無法被獨立審計。

Python 腳本的每一行都是一個可讀的決策 - 「把金額欄轉成 float」、「刪掉缺失值超過 50% 的列」、「日期格式統一成 YYYY-MM-DD」。 -

這些決策是透明的、可讀的、可以被審計的，也可以被修改和改進。 - 可重現性（reproducibility）：相同的輸入 + 相同的腳本 = 相同的輸出，永遠如此。

什麼情況用哪個：

情境	建議工具
快速看一眼資料，一次性	Excel
需要重複執行的分析流程	Python
分析結果需要被別人驗證	Python
資料超過 100 萬行	Python（Excel 有行數限制）
複雜的自訂計算邏輯	Python
和他人協作、版本控制	Python（Git 可追蹤）

不是說「Excel 沒用」 - 工具選對了，才是好的工程判斷。 - 一份 100 列的一次性對照表，Excel 比 Python 快。 - 一個每月執行的業績報表，Python 比 Excel 安全。

視覺建議： 左右對照時間軸圖。左側「Excel 工作流」：下載 CSV → 打開 Excel → 手動篩選 → 手動刪行 → 用公式轉換 → 複製到 PowerPoint，標注「下次要全部重來」。右側「Python 工作流」：下載 CSV → 執行 `clean.py` → 執行 `analyze.py` → 輸出報告，標注「下次只要換一行檔名」。

課堂互動： 請學員分享一個他們曾經用 Excel 做過、後來很希望是用程式碼做的分析任務。

過渡語： 「Python 思維的最佳實踐環境，是 Jupyter Notebook。它不只是一個 IDE，它改變了資料分析工作的呈現方式。」

S08 — 為什麼 Jupyter Notebook 成為標準

核心訊息：Jupyter Notebook 把文字說明、程式碼、執行結果、圖表全部放在同一個文件裡，讓分析過程本身可以被閱讀、被重現、被分享。

講師講解要點：

傳統分析流程的問題 - 程式在 `.py` 檔、說明在 Word、圖表在 PowerPoint，三個地方分開，難以維護。 - 更新一個地方其他兩個不會自動更新，容易出現不一致。 - 三個月後回頭看，很難重建「當時為什麼這樣做」的推論脈絡。

Notebook 的設計哲學：可執行的論文 - 每個 cell 可以是 Markdown 說明，也可以是可執行的 Python 程式。 - 執行結果（數字輸出、圖表）直接顯示在程式碼下方，不需要截圖或複製貼上。 - 一個 `.ipynb` 文件包含： - 文字（Markdown cell）：解釋為什麼這樣做，業務背景，分析假設 - 程式（Code cell）：實際的操作步驟 - 數字輸出：中間計算結果，方便隨時驗證 - 圖表：視覺呈現，直接嵌入文件

「資料故事」vs. 「資料報告」 - 報告：程式 + 圖表，讀者需要自己理解你在做什麼。 - 故事：推論脈絡，讀者能從頭到尾跟著你的思考走，理解每一步的為什麼。 - 一個好的 Notebook 是「可被任何人閱讀的分析過程」，不只是程式的執行記錄。

Notebook 在實際工作中的定位 - 探索分析（EDA）和溝通：Notebook 是最佳工具。 - 生產環境的自動化執行：通常用 `.py` 腳本，更容易測試和部署。 - 這兩個工具不互斥，Notebook 通常是 `.py` 腳本的前身（先探索，再固化為腳本）。

視覺建議： 一個真實 Jupyter Notebook 截圖，用標記框分別圈出：Markdown 說明 cell（標「文字說明：為什麼這樣做」）、程式碼 cell（標「可執行程式：實際操作」）、數字輸出（標「中間結果：隨時驗證」）、matplotlib 圖表輸出（標「視覺呈現：直接嵌入」）。

過渡語： 「知道了 Notebook 為什麼好，接下來要學怎麼用得好。有一些工作流技巧，讓你的 Notebook 從『能跑』變成『能讀』。」

S09 — Notebook 工作流實戰技巧

核心訊息：Notebook 容易用，但容易用得亂。掌握 cell 管理、Markdown 撰寫、執行順序控制、輸出管理這四個工作流習慣，讓你的 Notebook 從「能跑」變成「能讀」。

講師講解要點：

技巧一：Cell 管理原則 - 每個 code cell 做一件事，不要把十步操作塞進一個 cell。 - 長函式定義放在一個 cell，呼叫函式放在另一個 cell，方便分開重跑。 - 常用快捷鍵（Jupyter Lab / Classic Notebook 通用）： - `Shift + Enter`：執行當前 cell，移到下一個 - `B`（命令模式）：在下方新增 cell - `D, D`（命令模式）：刪除當前 cell - `M`（命令模式）：切換為 Markdown cell - `Y`（命令模式）：切換為 Code cell - `Esc`：進入命令模式

技巧二：Markdown 的標準段落結構 每個分析段落應該有： 1. 目的 cell（Markdown）：這一段要回答什麼問題？ 2. 程式 cell（Code）：執行分析操作。 3. 觀察 cell（Markdown）：從輸出看到了什麼？

```
# [illegible]

[illegible]

[illegible] 0-15% [illegible] 3% [illegible] 50%
[illegible]
```

技巧三：執行順序的陷阱 - Notebook 的 cell 可以任意順序執行，這是彈性，也是風險。 - 常見問題：後面的 cell 修改了一個變數，前面的 cell 再執行時行為不同。 - 好習慣：完成分析後，「Restart & Run All」一次，確保從頭到尾順序執行的結果是正確的。 - 左邊的 `[■■]` 顯示執行順序，`[*]` 代表正在執行。

技巧四：輸出管理 - 不是所有中間輸出都需要顯示。用 `;` 或 `_ =` 可以抑制不需要的輸出。 - 對於重要的中間結果，加 Markdown cell 解釋它的意義。 - 提交前清除所有輸出（Kernel → Restart & Clear Output），避免提交到 Git 的 `.ipynb` 包含大量輸出。

```
[illegible]
df.head();  # [illegible]

[illegible]
print(f"[illegible] {len(df_clean)} [illegible] {len(df) - len(df_clean)} [illegible]")
```

技巧五：Notebook 的標準開頭結構

```
# [illegible]
# [illegible] Notebook [illegible]
# [illegible] / API [illegible] / [illegible]
# [illegible]
# [illegible]
```

視覺建議：左側一個「好的 Notebook 結構」示範圖，右側標注五個技巧的對應位置。底部加一行：「評判一個 Notebook 的標準：去掉所有輸出後，你還能讀懂它嗎？」

過渡語：「技巧說完了，現在要動手做。工作坊 A 讓你從頭到尾對一份資料做系統性的探索。」

S10 — 工作坊 A：初探一份真實資料

類型：動手工作坊 時間：30 分鐘 工具：Jupyter Notebook（已預先安裝） 資料：`sales_sample.csv`（課程頁面提供）

目標：使用 Notebook 的標準工作流，套用資料分析生命週期的前三個階段（提問、取得、清理），對一份新資料進行系統性探索，並 Notebook 中記錄每個步驟的觀察。

工作坊 A 任務說明

資料背景：一份假設的電商月銷售資料，包含：訂單 ID、日期、商品類別、銷售金額、訂單數量、退貨率、客戶地區。

Step 0：建立 Notebook 標準開頭（3 分鐘）

新建一個 Notebook，第一個 cell 用 Markdown 寫：

```
# [redacted] A[redacted]
**[redacted]**: [redacted]
**[redacted]**: sales_sample.csv
**[redacted]**: [redacted]
```

Step 1：取得資料，看清結構（8 分鐘）

```
import pandas as pd

df = pd.read_csv("sales_sample.csv")

# [redacted]
print("=== [redacted] ===")
```

```
print(f"Shape: {df.shape}")
print(f"\n[redacted]")
print(df.dtypes)
```

在下方新增 Markdown cell，用你自己的話描述：「這份資料有幾筆記錄、幾個欄位，每個欄位代表什麼。」

Step 2：資料品質快速評估（10 分鐘）

```
# [redacted]
print("=== [redacted] ===")
print(df.isna().sum())
print()
print(df.isna().mean().map(lambda x: f"{x:.1%}").to_frame())

# [redacted]
print("\n=== [redacted] ===")
print(df.dtypes)

# [redacted]
print("\n=== [redacted] ===")
print(df.describe())
```

新增 Markdown cell，套用四維度框架，回答： - 完整性：哪些欄位有缺失？比例如何？ - 一致性：型態是否如預期？ - 準確性：有沒有不合理的數值（負金額、退貨率 > 100%）？ - 時效性：資料的日期範圍是什麼？

Step 3：找一個值得問的問題（9 分鐘）

```
# [redacted]
df.groupby("category")["amount"].sum().sort_values(ascending=False)
```

```
# [redacted]
df.groupby("region")["amount"].agg(["sum", "mean", "count"])
```

新增 Markdown cell，根據上面的輸出，寫下你的觀察： - 哪個發現讓你意外？ - 有什麼問題需要進一步確認？ - 如果要做一個完整分析，你的第一個分析假設是什麼？

Step 4：撰寫資料品質摘要（5 分鐘）

在 Notebook 最末新增一個 Markdown cell：

```
# [redacted]
[redacted]
[redacted]
[redacted]
[redacted]
[redacted]
```

展示與討論（5 分鐘）請 2-3 位學員分享他們的摘要，講師點評：「問題識別是否系統性？觀察是否和資料對應？」

工作坊 A 核心體驗：拿到一份新資料，不是立刻開始分析，而是先做系統性的健康檢查，把觀察記錄在 Notebook 裡。這個習慣，是資料分析師與「剛學會 pandas」的初學者之間的分水嶺。

S11 — 工作坊 B：資料品質健康檢查

類型： 動手工作坊 時間： 30 分鐘 工具： Jupyter Notebook 資料： `dirty_sales.csv`（課程頁面提供，故意包含各類資料問題）

目標： 套用 S04 和 S05 學到的四種污染類型與四維度評估框架，對一份含有已知問題的資料進行完整的品質稽核，並在 Notebook 中記錄每個問題和處理決策。

工作坊 B 任務說明

資料背景： `dirty_sales.csv` 是故意製造的「髒資料」，包含至少一個每種類型的問題。你的任務是找出所有問題，並說明你的處理決策。

Step 1：缺失值稽核（8 分鐘）

```
import pandas as pd
import numpy as np

df = pd.read_csv("dirty_sales.csv")

# [redacted]
missing_report = pd.DataFrame({
    "count": df.isna().sum(),
```

```
    "pct": df.isna().mean() * 100
}).sort_values("pct", ascending=False)

print(missing_report)
```

新增 Markdown cell，對每個有缺失的欄位說明： - 缺失比例是多少？ - 這個欄位的缺失是否可以接受？ - 你的處理策略是什麼（刪除 / 填補 / 保留），理由是什麼？

Step 2：重複值稽核（7 分鐘）

```
# ────
total_dups = df.duplicated().sum()
print(f"────────{total_dups}")

# ────────
if total_dups > 0:
    display(df[df.duplicated(keep=False)].sort_values(df.columns.tolist()))

# ──────
key_col = "order_id"
dup_keys = df[df.duplicated(subset=[key_col])][key_col].tolist()
print(f"\n── {key_col}─{dup_keys}")
```

Step 3：型態與離群值稽核（8 分鐘）

```
# ────
print("──────")
print(df.dtypes)

# ────────────────
try:
    pd.to_numeric(df["amount"], errors="raise")
    print("\namount ──────────")
except:
    # ────────
    bad_rows = df[pd.to_numeric(df["amount"], errors="coerce").isna()]
    print(f"\namount ──── {len(bad_rows)} ────────")
    display(bad_rows.head())
```

```
# ────────
clean_amount = pd.to_numeric(df["amount"].astype(str).str.replace(r"[$,]", "", regex=True), errors
print(clean_amount.describe())

# ────
q99 = clean_amount.quantile(0.99)
outliers = df[clean_amount > q99 * 3]
print(f"\n──────> 3──99──────{len(outliers)} ─")
```

Step 4：撰寫完整稽核報告（7 分鐘）

```
# [redacted]dirty_sales.csv
|
### [redacted]
- [redacted]
|
### [redacted]
- [redacted]
|
### [redacted] + [redacted]
- [redacted]
|
### [redacted]
- [redacted]
|
### [redacted]
- [redacted]|N| [redacted]
- [redacted]|N| [redacted]
- [redacted]|X1| [redacted]
- [redacted]|[redacted]
```

工作坊 B 核心體驗：一份「稽核報告」不是程式碼的輸出，而是你的判斷和決策的記錄。別人看到你的 Notebook，應該能理解你為什麼做了這些清理，而不只是看到清理後的結果。

S12 — 模組總結：分析師的思維工具箱

核心訊息： M1 建立了資料分析師的兩件基本配備：資料思維框架（生命週期 + 品質評估 + 格式識別）和 Notebook 工作流（四合一設計 + 實戰技巧）。這兩件配備是後續所有分析技術的地基。

講師講解要點：

快速回顧 M1 學到的核心概念： - 資料分析生命週期的六個階段，以及每個階段的輸入輸出。 - 結構化資料的五個元素：列、欄、綱要、型態、鍵。 - 四種污染類型：缺失、重複、離群、型態錯誤，各有不同處理策略。 - 資料品質評估的四個維度：完整性、一致性、準確性、時效性。 - 常見資料格式：CSV、JSON、SQL、API，各自的結構特徵。 - Notebook 工作流的最佳實踐：cell 管理、Markdown 結構、執行順序控制。

現在能做的事： - 拿到一份新資料，能系統性地做初步健康檢查。 - 用 Notebook 把分析過程組織成有推論脈絡的敘述。 - 識別資料問題的四種類型，並提出初步處理策略。 - 讀懂 CSV / JSON / SQL / API 的基本結構。

還不能做的事（後續模組會教）： - 用 Python 實際執行清理和分析操作（M2 Python 分析基礎）。 - 用 NumPy 和 pandas 處理大規模表格資料（M3）。 - 用視覺化工具呈現分析結果（M4）。

M2 的問題預告：「你現在知道資料長什麼樣、知道哪裡有問題，但要用 Python 實際動手處理，還需要掌握基本的 Python 語法工具。M2 只教分析師真正需要的那些，不多教也不少教。」

資料思維是這門課最重要的能力之一 - 不是語法，而是「看到資料就知道該問什麼問題」的直覺。 - 這個直覺需要練習，工作坊 A 和 B 是起點，後續每個模組都有機會繼續訓練。

視覺建議： 課程地圖圖，高亮 M0 和 M1 為「已完成」（深色），M2 為「下一站」（邊框高亮），M3-M6 為「待解鎖」（淺灰）。在 M1 方塊內列出本模組涵蓋的核心概念關鍵字。M1 到 M2 的箭頭上標注：「資料思維 → Python 表達力」。

過渡語（模組結束語）： 「帶著你在工作坊 B 寫的稽核報告去 M2，我們要學的是：如何用 Python 把你報告裡寫的那些清理步驟，真正執行出來。」

工作坊總結

工作坊	重點技能	預期輸出	核心體驗
工作坊 A	資料初探、品質快速評估、結構化記錄	一份完整的 Notebook，包含資料品質摘要	拿到新資料的系統性工作流，不是從分段落結構的探索
工作坊 B	資料品質稽核、四種污染類型識別、清理決策	一份資料稽核報告，稽核報告是判斷處理策略的記錄，不只是	

講師注意事項

1. S01 到 S03
是概念建立期，學員可能覺得抽象。建議多用真實資料截圖和具體例子，避免停留在純定義。

2. S02 生命週期 要強調「提問」是最重要的第一步，也是最常被跳過的。可以用一個壞問題 vs.
好問題的對比來讓概念具體化。

3. S04 四種污染
的「離群值不一定是錯誤」這個概念通常讓學員驚訝，花時間讓這個概念清楚，避免學員以後習慣性刪除所

4. S05 品質框架
要強調這個框架是可以重複套用的工具，讓學員把它記住或記在筆記裡，後面每次遇到新資料都拿出來用。

5. S06 資料格式 可以讓學員說說他們在工作或生活中遇過哪些格式，讓課程內容和現實連結。

6. S07 Excel vs. Python 通常引起討論，給 2-3 分鐘讓學員說說自己的 Excel
痛點，但注意不要讓討論偏向「說 Excel 壞話」，而是引導到「選對工具」的判斷力。

7. 工作坊 A 的重點是 Markdown
說明的品質，不是程式碼。如果學員只寫程式不寫觀察，主動引導。

8. 工作坊 B
的重點是決策記錄，不是找到所有問題。沒找到所有問題沒關係，但要能說清楚為什麼這樣處理找到的問題

課前準備清單（學員）

- [ ] 安裝 Anaconda 或 Miniconda（或已有 Python 3.10+ 環境）
- [ ] 確認可以執行 `jupyter notebook` 或 `jupyter lab`
- [ ] 下載範例資料 `sales_sample.csv` 和 `dirty_sales.csv`（課程頁面提供）
- [ ] 閱讀 M0 模組筆記（課程地圖與學習動機）

延伸閱讀

- McCallum, Q.E. (2012). Bad Data Handbook. O'Reilly Media. — 各種真實的資料品質問題案例
- Jupyter Notebook 官方文件：Markdown 語法與 cell 操作快捷鍵。jupyter.org

- Wickham, H. (2014). "Tidy Data." Journal of Statistical Software, 59(10). — 整潔資料的理論基礎。doi.org/10.18637/jss.v059.i10
- JSON 官方規格：json.org — 格式語法的完整定義

參考文獻與引用來源

產業數據

1. 「60-80% 時間花在資料清理」 — 原始來源：CrowdFlower (now Figure Eight). (2016). Data Science Report. 調查顯示資料科學家約 60% 時間用於清理與組織資料，19% 用於收集資料。此數據後被 Forbes、Harvard Business Review 等廣泛引用。另見 Kaggle. (2018). State of Data Science and Machine Learning Survey，報告清理時間約佔 15-25%。實際比例因產業與專案而異，本課程引用 60-80% 為常見範圍。
2. Jupyter Notebook 作為資料分析標準工具 — Project Jupyter. jupyter.org. JetBrains/PSF Python Developers Survey 2024 顯示 Jupyter 仍是資料科學工作者最常用的互動式環境之一。
3. Wickham, H. (2014). "Tidy Data." Journal of Statistical Software, 59(10), 1-23. — 「每一列是一筆觀測，每一欄是一個變數」的整潔資料原則。

M2：Python 分析基礎

課程定位： 第三幕 — 用 Python 表達資料操作，建立分析師的最小語法工具集 時數： 3 小時
前置模組： M1 資料思維與 Notebook 工作流 後續模組： M3 NumPy 與陣列思維

模組學習目標

完成本模組後，學生能夠：

1. 使用 Python 基本型態（int / float / str / bool）正確表達資料中的不同欄位類型
2. 選擇適合的容器（list / dict / tuple）來組織一筆或多筆資料記錄
3. 用 `if / elif / else` 寫出單一記錄的分類邏輯，對應資料標記的業務需求
4. 用 `for` 迴圈和 list comprehension 對資料集合進行批量轉換與篩選
5. 把重複的分析步驟定義成函式，讓清理邏輯可以被重複呼叫
6. 使用 `import` 引入外部套件，理解 Python 資料科學生態的基本架構
7. 閱讀簡單的錯誤訊息（TypeError / ValueError / NameError），找出並修正問題

關鍵概念清單

完成模組後，以下每個概念都應能一句話解釋：

- [ ] `int`（整數）：適合計數、ID、數量
- [ ] `float`（浮點數）：適合金額、比率、統計指標
- [ ] `str`（字串）：適合名稱、類別標籤、未轉換的日期
- [ ] `bool`（布林）：True / False，條件判斷的結果
- [ ] 型態轉換：`int()` / `float()` / `str()` 的使用與轉換失敗的處理

- [ ] `list`（列表）：有序、可重複、可修改，適合一欄資料的集合
- [ ] `dict`（字典）：鍵值對，一筆記錄的自然表達，也是 JSON 的直接對映
- [ ] `tuple`（元組）：不可修改的列表，適合固定設定和多值回傳
- [ ] `if / elif / else`：條件分支，決定在什麼條件下執行什麼邏輯
- [ ] `for` 迴圈：逐元素處理，適合需要對集合中每個元素做相同操作的情況
- [ ] list comprehension：一行的轉換與篩選，是 Python 最具代表性的語法模式
- [ ] 函式（function）：把重複工作封裝成可呼叫的流程，輸入 → 處理 → 輸出
- [ ] 預設參數：函式的參數可以有預設值，讓大多數情況不需要傳入所有參數
- [ ] 回傳多值：函式可以同時回傳多個值（以 tuple 形式）
- [ ] `import`：引入外部套件，擴展語言能力
- [ ] 套件別名：`import numpy as np`，是約定俗成的縮寫，不是任意命名
- [ ] TypeError / ValueError / NameError：三種最常見的錯誤類型及其含義

投影片大綱

#	投影片標題	核心訊息	預估時間
S01	只教分析師用得到的 Python	這堂課不是完整 Python 教學，是「資料分析最小工具集」	8 min
S02	數值型態：int 與 float	計數用 int，測量用 float，型態決定能做哪些計算	12 min
S03	字串型態：資料清理的主戰場	str 是最常見的「髒資料來源」，清理操作是核心技能	15 min
S04	容器：list 與 dict	list 是一欄資料，dict 是一筆記錄，兩個容器對應兩種資料視角	15 min

S05	tuple 與型態轉換	tuple 是不可變的 list，型態轉換是清理的基本動作	12 min
S06	條件分支：if / elif / else	資料標記和分類邏輯的基本句型，一個條件對應一個業務規則	15 min
S07	迴圈：for 與 list comprehension	批量處理資料的語言，從 for 到 comprehension 的思維躍升	18 min
S08	函式：把清理步驟變成可重用的工具	函式不是語法練習，是讓分析腳本從「一次性」變成「可重複使用	18 min
S09	錯誤閱讀：把 error 當成線索	TypeError / ValueError / NameError 的模式識別，快速定位問題	12 min
S10	import 與套件生態	Python 的強大 = 語言 + 生態，import 打開套件的大門	10 min
S11	工作坊 A：用 Python 清理一欄髒資料	把 S02-S08 學到的工具組合起來，完成一個真實的清理任務	30 min
S12	工作坊 B：把五個清理步驟封裝成函式庫	把分析流程模組化，體驗腳本可重現的意義	30 min

總計：約 195 分鐘（含工作坊與 Q&A）

詳細投影片內容

S01 — 只教分析師用得到的 Python

核心訊息： 這堂課不是完整的 Python 程式設計課，而是「資料分析師的最小語法工具集」。學的每一個概念，都對應到分析工作的具體場景，沒有場

講師講解要點：

這堂課的邊界： - 會教的： 型態（int / float / str / bool）、容器（list / dict / tuple）、條件（if / elif / else）、迴圈（for）、函式（def）、套件引入（import）。 - 不教的： 類別（class）、繼承、裝飾器、生成器、非同步、型別系統。 - 這些「不教的」不是因為不重要，而是因為資料分析初學階段用不到。等需要用到的時候，你有了足夠的基礎，

學習原則：每個語法概念都對應一個分析場景 - `int` / `float`：訂單數量 vs. 銷售金額的差異 - `str`：清理商品名稱、統一格式 - `list`：一整欄資料的集合 - `dict`：一筆訂單記錄 - `if`：把金額分成「高、中、低」三個標籤 - `for`：對每個商品名稱進行清理 - `def`：把「清理金額欄」這個步驟封裝起來 - `import pandas`：打開資料分析的大門

從 M1 到 M2 的連接： - M1 你學會了「識別問題」：這份資料有缺失值、金額欄是字串、有重複記錄。 - M2 你要學會「執行操作」：用 Python 實際把這些問題修掉。 - 這兩個能力缺一不可：識別了問題但不會修，沒有用；會修但識別不出問題，更危險。

視覺建議：
一個「分析師工具箱」的圖示，工具箱裡有七個工具格，每格一個概念名稱，旁邊是一行對應的分析場景描述。M3 NumPy/pandas 的前置基礎。」

過渡語： 「從最基本的開始。Python 的型態系統告訴我們，資料裡的不同欄位，本質上是不同的東西。」

S02 — 數值型態：int 與 float

核心訊息： 整數用 `int`，帶小數的數字用 `float`。型態決定能做哪些計算，更重要的是，型態決定計算結果的意義。資料清理中，把字串正確轉換為數值型

講師講解要點：

int（整數）的使用場景： - 計數類：訂單數量（3 件商品，不會是 3.5 件）、用戶 ID、頁數。 - 索引類：資料表的列號、排名。 - 注意：Python 的整數沒有大小限制，可以是任意大的整數。

```
order_count = 7
user_id = 10024
rank = 1

# 整數運算
print(10 // 3)  # 3
print(10 % 3)   # 1
print(10 / 3)   # 3.3333...，Python 3 會回傳 float
```

float（浮點數）的使用場景：- 金額：$1,200.50 - 比率：退貨率 0.035（3.5%） - 統計指標：平均值、標準差

```
amount = 1200.50
return_rate = 0.035
average_order = 856.7

# 浮點數的精度問題
print(0.1 + 0.2)          # 0.30000000000000004，不是 0.3
print(round(0.1 + 0.2, 2) # 0.3，用 round() 修正
```

型態對計算的影響：

```
# 整數運算會變成 float
a = 7
b = 2

print(a / b)   # 3.5，float
print(a // b)  # 3，整數
print(a % b)   # 1

# 字串 vs. 數字
print("100" + "200")  # "100200"，字串連接
print(100 + 200)      # 300，數值相加
```

資料分析常見的數值清理問題：

```
# 從資料庫讀進來的金額
raw_amount = "1,200.50"

# 清洗
cleaned = float(raw_amount.replace(",", ""))
print(cleaned)  # 1200.5
print(type(cleaned))  # <class 'float'>
```

視覺建議：三欄對照表：欄一「型態」、欄二「資料場景例子」、欄三「Python 語法與注意事項」。底部加一個「浮點精度陷阱」的小提示框：`0.1 + 0.2 ≠ 0.3`，並說明 `round()` 的解法。

過渡語：

「數字型態還算直觀。資料清理中更麻煩的，是字串型態。資料裡最多髒東西藏在字串裡。」

S03 — 字串型態：資料清理的主戰場

核心訊息：

字串（str）是最常見的「髒資料來源」，空格、大小寫不一致、混入的特殊字元都在字串欄位裡。掌握字串的清

講師講解要點：

字串的建立與基本特性：

```
name = "Alice Wang"
category = "Electronics"
raw_date = "2024-01-15"  # 日期也是字串

# 字串是 immutable
# 一旦建立不可修改
name_clean = name.strip().lower()
print(name)          # "Alice Wang"
print(name_clean)    # "alice wang"
```

最常用的清理操作（對應真實的髒資料問題）：

```
# 從 1 號檔案讀出的 CSV 欄位
raw = "  Electronics  "
print(raw.strip())   # "Electronics"
print(raw.lstrip())  # "Electronics  "
print(raw.rstrip())  # "  Electronics"
```

```
# 從 2 號檔案讀出的客戶名稱
names = ["Alice Wang", "ALICE WANG", "Alice Wang", "alice  wang"]
unified = [n.strip().lower() for n in names]
# 結果：["alice wang", "alice wang", "alice wang", "alice  wang"]
import re
unified = [re.sub(r"\s+", " ", n.strip().lower()) for n in names]
```

```
# 從 3 號檔案讀出的金額欄位
raw_amount = "$1,200.50"
clean_amount = raw_amount.replace("$", "").replace(",", "")
value = float(clean_amount)  # 1200.5
```

```
# [redacted] 4[redacted]
full_name = "Wang, Alice"
last_name, first_name = full_name.split(", ")
print(first_name)  # "Alice"
print(last_name)  # "Wang"
```

字串格式化（輸出分析結果）：

```
# f-string [redacted]Python 3.6+[redacted]
category = "Electronics"
total = 45600.5
count = 230
|
# f-string[redacted]
print(f"{category}: [redacted] ${total:,.2f}[redacted] {count} [redacted]")
# [redacted]Electronics: [redacted] $45,600.50[redacted] 230 [redacted]
|
# [redacted]
:,.2f  → [redacted] + [redacted]
:>10   → [redacted] 10
:.1%   → [redacted]
rate = 0.0354
print(f"[redacted]{rate:.1%}")  # [redacted]3.5%
```

字串的查詢與判斷：

```
email = "alice@company.com"
product_name = "Wireless Headphone 2024"
|
# [redacted]
print("@" in email)                    # True
print(product_name.startswith("Wireless"))  # True
print(product_name.endswith("2024"))        # True
|
# [redacted]
print(product_name.find("Headphone"))       # 9[redacted]
print(product_name.find("Keyboard"))        # -1[redacted] -1[redacted]
```

視覺建議：一個「字串清理操作速查表」，左欄是「問題類型」，右欄是「解決方法 + 程式碼片段」，涵蓋六種最常見的問題。表格下方一行說明：「90% 的字串清理工作，都可以用這六種操作解決。」

過渡語：「單個值（int / float / str）處理好了。真實的資料是一群值的集合。Python 用三種容器來組織集合：list、dict、tuple。」

S04 — 容器：list 與 dict

核心訊息：`list` 是一欄資料（有序的值的集合），`dict` 是一筆記錄（鍵值對，每個鍵對應一個值）。這兩個容器分別對應資料表格的「欄視角」和「列視角」。

講師講解要點：

list（列表）— 欄視角 - 用途：儲存一個欄位的所有值，或一系列需要按順序處理的元素。 - 特性：有順序（index 從 0 開始）、可以有重複值、可以修改。

```
# [illegible]
amounts = [1200.0, 850.5, 3400.0, 125.0, 2100.5]

# [illegible] 0 [illegible]
print(amounts[0])   # 1200.0[illegible]
print(amounts[-1])  # 2100.5[illegible]
print(amounts[1:3])  # [850.5, 3400.0][illegible] 1 [illegible] 2[illegible]

# [illegible]
print(len(amounts))   # 5[illegible]
print(min(amounts))   # 125.0
print(max(amounts))   # 3400.0
print(sum(amounts))   # 7675.0

# [illegible] list
amounts.append(500.0)  # [illegible]
amounts[0] = 1300.0    # [illegible]
print(amounts)
```

```
# [illegible]
categories = ["Electronics", "Clothing", "Electronics", "Food", "Clothing"]

# [illegible] set [illegible] list
unique_cats = list(set(categories))
print(unique_cats)  # [illegible] set [illegible]

# [illegible]
from collections import Counter
print(Counter(categories))  # {'Electronics': 2, 'Clothing': 2, 'Food': 1}
```

dict（字典）— 列視角 - 用途：儲存一筆記錄的所有欄位，鍵是欄名，值是欄位的值。 - 特性：鍵必須唯一、可以修改值、鍵值對的存取比 list 更語意清晰。

```
# [illegible]
order = {
    "order_id": "ORD-0042",
    "customer": "Alice Wang",
```

```
    "amount": 1200.50,
    "category": "Electronics",
    "status": "completed"
}

# 取值
print(order["order_id"])        # "ORD-0042"
print(order.get("amount"))      # 1200.5
print(order.get("discount", 0))  # 0（沒有 KeyError）

# 檢查鍵是否存在
print("status" in order)        # True
print("discount" in order)      # False

# 修改
order["status"] = "shipped"
order["discount"] = 0.1

# 遍歷
for key, value in order.items():
    print(f"{key}: {value}")
```

list of dicts — 多筆記錄的自然表達 - 真實資料通常是多筆記錄，用 list of dicts 來表達。 - 這也是 JSON 陣列格式的直接對映，也是 pandas DataFrame 的底層結構之一。

```
orders = [
    {"order_id": "ORD-001", "amount": 1200.0, "category": "Electronics"},
    {"order_id": "ORD-002", "amount": 350.5, "category": "Clothing"},
    {"order_id": "ORD-003", "amount": 2800.0, "category": "Electronics"},
]

# 取第一筆
print(orders[0])

# ═══ list comprehension S07 ═══
all_amounts = [o["amount"] for o in orders]
print(all_amounts)  # [1200.0, 350.5, 2800.0]
```

視覺建議： 左側一個 list 的視覺化（垂直方塊，每個格子是一個值，旁邊標 index），右側一個 dict 的視覺化（左右兩欄，左欄是鍵，右欄是值，用箭頭連接）。底部加一個「list of dicts」的示意圖，對應一張二維表格（list 是列，dict 是欄）。

過渡語： 「list 和 dict 是最常用的兩個容器。Python 還有第三個容器 tuple，以及在容器之間切換的型態轉換，資料清理會用到。」

S05 — tuple 與型態轉換

核心訊息： `tuple` 是不可修改的 `list`，適合固定設定和函式的多值回傳。型態轉換（`int()` / `float()` / `str()`）是資料清理的基本動作，轉換失敗時要有防護機制。

講師講解要點：

tuple（元組） - 和 list 的差異：tuple 用小括號 `()` 建立，建立後不能修改（immutable）。 - 使用時機一：函式多值回傳（最常見）。 - 使用時機二：不應被修改的設定值，例如顏色的 RGB 值、座標。 - 使用時機三：pandas 的 `df.shape` 回傳 tuple，代表「這個資料結構的形狀是固定的描述，不是可以修改的值」。

```
# tuple ■■■
dimensions = (1000, 5)  # ■■ 1000 ■ 5 ■
colors = ("red", "green", "blue")

# ■■■■ list ■■■■■
print(dimensions[0])  # 1000
print(dimensions[1])  # 5

# ■■■■■■■■■■ tuple ■■■
rows, cols = dimensions
print(f"■■■■■ {rows} ■■■■{cols} ■■■■")

# ■■■■■■■■■■■
# dimensions[0] = 2000  # TypeError: 'tuple' object does not support item assignment
```

```
# ■■■■■■■■■■ tuple■
def get_amount_stats(amounts: list) -> tuple:
    """Return (min, max, mean) of a list of amounts."""
    return min(amounts), max(amounts), sum(amounts) / len(amounts)

amounts = [1200.0, 850.5, 3400.0, 125.0, 2100.5]
lo, hi, avg = get_amount_stats(amounts)
print(f"■■■{lo}■■■■{hi}■■■■{avg:.2f}")
```

型態轉換（Type Conversion）

資料清理最基本的動作：把字串轉成數值型態，或把數值轉成字串。

```
# str → int
count_str = "230"
count = int(count_str)
print(count + 10)  # 240

# str → float
price_str = "1200.50"
price = float(price_str)
print(price * 0.9)  # 1080.45■■■■
```

```
# int / float → str
amount = 1200.5
print("金額" + str(amount))  # "金額1200.5"
# 推薦：f-string
print(f"金額{amount}")
```

型態轉換的失敗處理（非常重要）

```
# 會產生 ValueError
bad_value = "N/A"
float(bad_value)  # ValueError: could not convert string to float: 'N/A'

# 用 try/except 安全轉換
def safe_to_float(value, default=None):
    """
    Try to convert value to float.
    Returns default if conversion fails.
    """
    try:
        return float(str(value).replace(",", "").replace("$", ""))
    except (ValueError, TypeError):
        return default

print(safe_to_float("1,200.50"))  # 1200.5
print(safe_to_float("N/A"))       # None
print(safe_to_float(""))          # None
print(safe_to_float(None))        # None

# 在 pandas 中 errors='coerce' 會這樣
pd.to_numeric(series, errors="coerce")  # 失敗變成 NaN
```

視覺建議：上半部是 tuple vs. list 的對比表格（特性、使用場景、能否修改）。下半部是型態轉換的流程圖：左邊是原始型態（str），中間是轉換（`float()`），右邊是目標型態，下面有一個分叉：「成功 → 回傳結果」和「失敗（ValueError）→ try/except 處理」。

過渡語：「型態和容器都有了。接下來是條件判斷，這是讓 Python 能根據資料值做不同事情的能力。」

S06 — 條件分支：if / elif / else

核心訊息：`if` / `elif` / `else` 是根據資料值做不同處理的基本句型。在資料分析中，最常用於：新增標記欄位（把數值分類成標籤）、資料驗

講師講解要點：

基本語法結構：

```
[redacted]
if [redacted]
    # [redacted]
elif [redacted]
    # [redacted]
else:
    # [redacted]
```

資料分析最常見的用途一：數值標記（Labeling）

```
def label_order_value(amount: float) -> str:
    [redacted]
    Classify an order into value tiers based on amount.
    [redacted]
    if amount >= 5000:
        return "premium"
    elif amount >= 1000:
        return "standard"
    elif amount >= 100:
        return "economy"
    else:
        return "micro"

[redacted]
print(label_order_value(6500))  # "premium"
print(label_order_value(1500))  # "standard"
print(label_order_value(50))    # "micro"
```

資料分析最常見的用途二：資料驗證

```
def is_valid_order(order: dict) -> bool:
    [redacted]
    Check if an order record has valid required fields.
    Returns True if valid, False otherwise.
    [redacted]
    # [redacted]
    if "order_id" not in order:
        return False

    # [redacted]
    if order.get("amount", 0) <= 0:
        return False

    # [redacted]
    valid_statuses = {"pending", "completed", "cancelled", "refunded"}
```

```
    if order.get("status") not in valid_statuses:
        return False

    return True

good_order = {"order_id": "ORD-001", "amount": 1200.0, "status": "completed"}
bad_order  = {"order_id": "ORD-002", "amount": -50.0, "status": "unknown"}
print(is_valid_order(good_order))  # True
print(is_valid_order(bad_order))   # False
```

資料分析最常見的用途三：缺失值的條件處理

```
def get_display_amount(order: dict) -> str:
    """
    Return formatted amount string, or a placeholder if missing.
    """
    amount = order.get("amount")

    if amount is None:
        return "[illegible]"
    elif amount < 0:
        return f"[illegible] ${abs(amount):,2f}"
    else:
        return f"${amount:,,2f}"

print(get_display_amount({"amount": 1200.5})   # "$1,200.50"
print(get_display_amount({"amount": None}))    # "[illegible]"
print(get_display_amount({"amount": -350.0}))  # "[illegible] $350.00"
```

比較運算子快速整理：

```
x = 1200

# [illegible]
print(x > 1000)   # True
print(x >= 1200)  # True[illegible]
print(x == 1200)  # True[illegible]
print(x != 0)     # True[illegible]

# [illegible]
print(x > 100 and x < 5000)  # True[illegible]
print(x < 100 or x > 1000)   # True[illegible]
print(not (x == 0))          # True[illegible]

# [illegible]
value = None
print(value is None)         # True[illegible] None[illegible] is[illegible] ==[illegible]
print(value is not None)     # False
```

視覺建議：左側一個流程圖（菱形 → 兩條路徑，對應 if/else），右側對應三個使用場景的程式碼框（標記 / 驗證 / 缺失值處理），每個場景下方一行「這個函式會在 M3 pandas 中用 `.apply()` 套用到整欄」。

過渡語：「if 處理的是一筆記錄的判斷。當你需要對一群記錄（一整欄、一整個 list）做同樣的事，就需要迴圈。」

S07 — 迴圈：for 與 list comprehension

核心訊息：`for` 迴圈是對集合中每個元素做相同操作的基本句型。list comprehension 是「可讀的一行版本」，是 Python 最具代表性的語法模式，也是分析師最值得掌握的進階寫法。

講師講解要點：

for 迴圈：基本結構

```
for [redacted] in [redacted]:
    # [redacted]
```

```
[redacted]
product_names = [" Wireless Headphone", "USB Cable ", " Laptop Bag "]

cleaned_names = []
for name in product_names:
    cleaned = name.strip().lower()
    cleaned_names.append(cleaned)

print(cleaned_names)
# ['wireless headphone', 'usb cable', 'laptop bag']
```

for 迴圈的常見資料分析用途：

```
[redacted]
import pandas as pd
df = pd.read_csv("sales.csv")

print("=== [redacted] ===")
for col in df.columns:
    missing_count = df[col].isna().sum()
    missing_pct = df[col].isna().mean() * 100
    if missing_count > 0:
        print(f"{col}: {missing_count} [redacted]{missing_pct:.1f}%[redacted]")
```

```
# [redacted]
numeric_cols = ["amount", "quantity", "return_rate"]

for col in numeric_cols:
    lo = df[col].min()
    hi = df[col].max()
    avg = df[col].mean()
    print(f"{col}: [redacted] {lo:.2f}, [redacted] {hi:.2f}, [redacted] {avg:.2f}")
```

enumerate：同時取索引和值

```
categories = ["Electronics", "Clothing", "Food"]

for i, cat in enumerate(categories):
    print(f"{i}: {cat}")
# 0: Electronics
# 1: Clothing
# 2: Food
```

zip：同時迭代兩個 list

```
col_names = ["amount", "quantity", "return_rate"]
thresholds = [0, 1, 0]

for col, threshold in zip(col_names, thresholds):
    invalid = (df[col] < threshold).sum()
    if invalid > 0:
        print(f"{col} [redacted] {invalid} [redacted] {threshold}[redacted]")
```

list comprehension：一行的轉換與篩選

list comprehension 是 Python 最具代表性的語法，把 for 迴圈 + 條件 + append 壓縮成一行，可讀且高效。

```
# [redacted]
[[redacted] for [redacted] in [redacted]]
[[redacted] for [redacted] in [redacted] if [redacted]]
```

```
# [redacted]
product_names = ["  Wireless Headphone", "USB Cable  ", "  Laptop Bag  "]

# [redacted]
# for [redacted]4 [redacted]
cleaned = []
for name in product_names:
    cleaned.append(name.strip().lower())
```

```
# list comprehension 一行
cleaned = [name.strip().lower() for name in product_names]

print(cleaned)
# ['wireless headphone', 'usb cable', 'laptop bag']
```

```
# 轉換 + 篩選
amounts = [1200.0, 350.5, 2800.0, 125.0, 5600.0, 88.0]

# for 版本
high_value = []
for a in amounts:
    if a >= 1000:
        high_value.append(a)

# list comprehension 版本
high_value = [a for a in amounts if a >= 1000]

print(high_value)  # [1200.0, 2800.0, 5600.0]
```

```
# 一組 list of dicts 模擬訂單
orders = [
    {"order_id": "ORD-001", "amount": 1200.0, "category": "Electronics"},
    {"order_id": "ORD-002", "amount": 350.5, "category": "Clothing"},
    {"order_id": "ORD-003", "amount": 2800.0, "category": "Electronics"},
]

# 篩選：只要 Electronics 的金額
electronics_amounts = [o["amount"] for o in orders if o["category"] == "Electronics"]
print(electronics_amounts)  # [1200.0, 2800.0]
```

什麼時候不用 list comprehension： - 邏輯太複雜（超過一個 if 條件，或需要多行處理）→ 用 for 迴圈或函式 - 資料量很大（超過幾萬筆）→ 用 pandas 的向量化操作（M3 會教），效能更好

視覺建議： 中央一個對比框，左側「for 迴圈版本（5 行）」，右側「list comprehension 版本（1 行）」，用雙箭頭標注「等價」。底部加三個語法模式的小卡片：「轉換」、「篩選」、「轉換 + 篩選」，每個附一行程式碼例子。

過渡語：
「你現在可以對一群資料做批量操作了。但如果這些操作需要反覆用在不同的資料集上，每次都要重寫一遍，這

S08 — 函式：把清理步驟變成可重用的工具

核心訊息：

定義函式不是語法練習，而是把「我這次做了什麼」抽象成「任何時候任何人都能呼叫的清理工具」。函式是讓

講師講解要點：

函式的三要素：名稱 / 參數 / 回傳值

```
def [redacted](a1, a2) -> [redacted]:
    """
    Docstring[redacted]
    """
    # [redacted]
    return [redacted]
```

```
[redacted]
def clean_currency_string(raw: str) -> float:
    """
    Convert a currency string like '$1,200.50' or '1200' to float.
    Returns None if the input cannot be converted.

    Args:
        raw: Raw string value from data source

    Returns:
        Float value, or None if conversion fails
    """
    if raw is None:
        return None
    cleaned = str(raw).strip().replace("$", "").replace(",", "")
    try:
        return float(cleaned)
    except ValueError:
        return None

[redacted]
print(clean_currency_string("$1,200.50"))  # 1200.5
print(clean_currency_string("1200"))       # 1200.0
print(clean_currency_string("N/A"))        # None
print(clean_currency_string(None))         # None
print(clean_currency_string(""))           # None
```

預設參數：讓函式更靈活

```
def summarize_column(values: list, top_n: int = 5, show_pct: bool = True) -> None:
    """
    Print a frequency summary of a categorical column.
    """
    from collections import Counter
    counts = Counter(values).most_common(top_n)
```

```
    total = len(values)

    print(f"■ {top_n} ■■■■■ {total} ■■■■")
    for category, count in counts:
        if show_pct:
            print(f"  {category}: {count} ({count/total: 1%})")
        else:
            print(f"  {category}: {count}")

# ■■■■
categories = ["Electronics", "Clothing", "Electronics", "Food", "Clothing", "Electronics"]
summarize_column(categories)              # ■■■■■top 5■■■■■■
summarize_column(categories, top_n=2)     # ■■■ 2 ■
summarize_column(categories, show_pct=False)  # ■■■■■■
```

回傳多個值：用 tuple

```
def assess_column_quality(values: list) -> tuple:
    """
    Assess the quality of a column's values.
    Returns (total_count, missing_count, missing_pct, unique_count)
    """
    import math
    total = len(values)
    missing = sum(1 for v in values if v is None or (isinstance(v, float) and math.isnan(v)))
    unique = len(set(v for v in values if v is not None))

    return total, missing, missing / total * 100, unique

# ■■■■■■■■■■■
total, missing, missing_pct, unique = assess_column_quality([1, 2, None, 3, 2, None])
print(f"■■ {total} ■■■■ {missing} ■■{missing_pct:.1f}%■■■■■ {unique} ■")
```

函式設計原則（對分析師最實用的部分）：- 一個函式做一件事：`clean_and_summarize()` 這種命名通常是設計問題的信號，應該拆成兩個函式。- 函式名稱用動詞開頭：`clean_amount()`、`label_order()`、`validate_record()`，讓呼叫者一眼看懂這個函式做什麼
- 永遠寫 docstring：至少一行說明「這個函式接受什麼，回傳什麼」。- 處理邊界情況：None、空字串、負數、型態錯誤，要想清楚函式應該怎麼處理，不要讓它默默回傳錯誤結果。

視覺建議：
一個「函式三要素」圖：大方框代表函式，左邊是輸入箭頭（標「參數：這次要處理什麼」），右邊是輸出箭頭（標「回傳值」），中間寫函式名稱，下方加一個「好函式 vs. 壞函式」對比：好函式有 docstring、有預設值、有邊界處理；壞函式沒有說明、依賴外部變數、默默吞掉錯誤。

過渡語：「寫函式時很常遇到錯誤。下一張投影片要教你怎麼閱讀錯誤訊息，把 error 當成定位問題的線索，而不是讓你放棄的障礙。」

S09 — 錯誤閱讀：把 error 當成線索

核心訊息：Python 的錯誤訊息不是「程式壞掉了」，而是「程式告訴你哪裡不對」。學會閱讀三種最常見的錯誤（TypeError / ValueError / NameError），能讓你在 30 秒內定位大多數初學者常見的問題。

講師講解要點：

錯誤訊息的結構

```
Traceback (most recent call last):
  File "analysis.py", line 12, in clean_data        ← 哪個檔案
    return float(raw_amount)                        ← 哪一行程式
ValueError: could not convert string to float: 'N/A'  ← 錯誤類型 + 原因
```

閱讀順序：從最底部開始讀，最後一行是錯誤類型和原因，往上是呼叫鏈。

TypeError：對錯誤的型態做了操作

```
# 字串 + 數字
name = "Alice"
count = 3
print(name + count)   # TypeError: can only concatenate str (not "int") to str
print(name + str(count))   # 正確寫法

# 對 None 呼叫方法
value = None
print value.strip()   # AttributeError: 'NoneType' object has no attribute 'strip'
if value is not None:
    value.strip()   # 確保不是 None

# 對不可索引的物件
number = 42
print(number[0])   # TypeError: 'int' object is not subscriptable
```

ValueError：型態對，但值不合法

```
# 字串無法轉成數字
float("N/A")   # ValueError: could not convert string to float: 'N/A'
```

```
int("12.5")      # ValueError: invalid literal for int() with base 10: '12.5'

def safe_float(s):
    try:
        return float(s)
    except ValueError:
        return None  # 轉換失敗回傳 None

# 搭配 pandas 時通常會先檢查
```

NameError：變數不存在

```
# 定義訂單金額
order_amount = 1200.0
print(order_amout)  # NameError: name 'order_amout' is not defined（拼錯了！）

# 嘗試使用計算結果
print(result)  # NameError: name 'result' is not defined
result = compute_something()
print(result)

# Notebook 常常發生：cell 執行順序
# 先跑 Cell 3，再跑 Cell 5，結果變成 Cell 3 報 NameError
```

KeyError：dict 中不存在的鍵

```
order = {"order_id": "ORD-001", "amount": 1200.0}

print(order["status"])  # KeyError: 'status'

# 安全的做法：.get()
status = order.get("status", "unknown")  # 沒有 "status" 鍵，回傳 "unknown"

# 或先檢查
if "status" in order:
    print(order["status"])
```

IndexError：索引超出範圍

```
# 假設 list 只有 3 個
data = [10, 20, 30]
```

```
print(data[5])  # IndexError: list index out of range

# 空 list 的
results = []
print(results[0])  # IndexError: list index out of range

# 安全的做法
if len(results) > 0:
    print(results[0])
```

五種常見錯誤速查表

錯誤類型	觸發原因	典型場景	修正方向
TypeError	對錯誤型態做操作	`str + int`、對 None 呼叫方法	檢查型態，先轉型
ValueError	型態對但值不合法	`float("N/A")`、`int("12.5")`	try/except 包住
NameError	變數不存在	拼錯名、Notebook cell 順序錯	檢查拼寫、重跑 cell
KeyError	dict 鍵不存在	`order["status"]` 但沒有 status	用 `.get()` 加預設值
IndexError	索引超出範圍	空 list 取 `[0]`、超出長度	取值前檢查 `len()`

除錯的系統性方法（給初學者）：1. 讀最後一行：錯誤類型 + 原因 2. 看錯誤指向的那一行程式碼 3. 加 `print()` 把可疑變數的值印出來，確認型態和內容 4. 問：「我以為這個變數是什麼型態？它實際上是什麼型態？」 5. 查速查表：對照上面五種常見錯誤，找到修正方向

```
# 加上 print 檢查
def clean_row(row: dict) -> dict:
    # 印出收到的資料
    print(f"DEBUG: row = {row}")
```

```
    print(f"DEBUG: amount type = {type(row.get('amount'))}")

    amount = row.get("amount")
    # [redacted]...
```

視覺建議：
三個並排的「錯誤卡片」，每張卡片：錯誤類型名稱（大字）、觸發條件（一行）、典型錯誤訊息（程式碼字體

過渡語：「有了型態、容器、條件、迴圈、函式、除錯，最後一個工具是 import。Python 最強大的地方不是語言本身，而是它的套件生態。」

S10 — import 與套件生態

核心訊息：`import` 打開了 Python 套件生態的大門。資料分析 / 機器學習 / AI 的能力都來自套件，不是來自語言本身。理解套件的引入方式，和知道哪些套件解決哪些問題，是分析師的基礎

講師講解要點：

import 的三種寫法與使用時機：

```
import [redacted]
import math
print(math.sqrt(16))    # 4.0
print(math.ceil(2.3))   # [redacted]

[redacted]
# [redacted] math / os / datetime / json [redacted]
```

```
import [redacted] alias [redacted]
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# np / pd / plt [redacted]
# [redacted] Stack Overflow [redacted]
arr = np.array([1, 2, 3, 4, 5])
df = pd.DataFrame({"amount": [1200, 350, 2800]})

[redacted]
# [redacted]
```

```
# [redacted]
from datetime import datetime, timedelta
from pathlib import Path
from collections import Counter, defaultdict

# [redacted]
today = datetime.now()
data_dir = Path("data/")
counts = Counter(["a", "b", "a", "c", "a"])

# [redacted]
# [redacted]
```

資料分析的核心套件地圖（預告 M3-M6）：

套件	用途	何時學
`pandas`	表格資料處理（DataFrame）	M3
`numpy`	數值計算 / 陣列操作	M3
`matplotlib`	基礎視覺化	M4
`seaborn`	統計視覺化（基於 matplotlib）	M4
`scikit-learn`	機器學習	M5/M6
`requests`	HTTP 請求 / API 呼叫	M5

為什麼說「Python 的強大 = 語言 + 生態」：- 純 Python 語法，和其他腳本語言差異不大。- NumPy 讓 Python 能高效處理大規模數值計算（背後是 C 語言）。- pandas 讓 Python 能處理複雜的表格操作，一行頂過去的十行 for 迴圈。- PyTorch / TensorFlow 讓 Python 成為 AI 訓練的主戰場。- 學 Python，不只是學語法，是加入這個生態系。

套件安裝概念（簡短提及）：

```
# [redacted] Notebook [redacted]
pip install pandas numpy matplotlib
```

```
[redacted]
[redacted] Notebook [redacted] ! [redacted]
!pip install pandas
```

視覺建議： 圓心輻射圖。中心是「Python 語言核心」，往外第一圈是資料分析套件（pandas / numpy / matplotlib / seaborn），第二圈是 ML/AI（scikit-learn / PyTorch），第三圈是工程工具（requests / FastAPI）。每個套件名稱下方標一行用途。在 M3-M6 的模組標示對應的套件。

過渡語： 「你現在有了資料分析師的 Python 工具集。是時候把這些工具組合起來，做一個真實的練習。」

S11 — 工作坊 A：用 Python 清理一欄髒資料

類型： 動手工作坊 時間： 30 分鐘 工具： Jupyter Notebook 資料： `dirty_amounts.py`（清單直接內建在練習中，不需要額外檔案）

目標： 把 S02-S09 學到的型態、字串清理、條件判斷、迴圈、函式組合起來，完成一個從髒資料到乾淨資料的完整清理任務，並把

工作坊 A 任務說明

任務背景：
你收到一份從舊系統匯出的金額欄位，包含各種格式問題。你的任務是寫出一個強健的清理函式，並測試各種邊

Step 0：看清問題所在（5 分鐘）

```
[redacted]
raw_amounts = [
    "$1,200.50",      # [redacted] $ [redacted] + [redacted]
    "856.5",          # [redacted]
    "  3400 ",        # [redacted]
    "N/A",            # [redacted]
    "$125.00",        # [redacted]
    "",               # [redacted]
    "2,100.50",       # [redacted] $ [redacted]
    None,             # [redacted] None
    "-350.00",        # [redacted]
```

```
    "1200abc"        # [redacted]
[redacted]
[redacted]
print(f"[redacted] {len(raw_amounts)} [redacted]")
print("[redacted]")
for i, v in enumerate(raw_amounts):
    print(f"  [{i}] {repr(v)}  (type: {type(v).__name__})")
```

Step 1：寫基本清理函式（8 分鐘）

先寫一個能處理大多數情況的版本：

```
def clean_amount_v1(raw) -> float | None:
    """
    Basic version: clean a single amount value.
    Returns float, or None if cannot be cleaned.
    """
    # [redacted] None
    if raw is None:
        return None

    # [redacted]
    s = str(raw).strip()

    # [redacted]
    if s == "":
        return None

    # [redacted] $ [redacted],
    s = s.replace("$", "").replace(",", "")

    # [redacted]
    try:
        return float(s)
    except ValueError:
        return None

# [redacted] Markdown cell [redacted]
```

Step 2：測試並找出漏洞（5 分鐘）

```
[redacted]
print("=== v1 [redacted] ===")
for raw in raw_amounts:
    result = clean_amount_v1(raw)
    status = "OK" if result is not None else "FAIL"
    print(f"  {repr(raw):20} → {repr(result):12}  [{status}]")
```

在 Markdown cell 說明：哪些測試案例通過了？哪些回傳了 None 但你認為應該有值？

Step 3：改進函式（7 分鐘）

根據測試結果，改進函式：

```
def clean_amount_v2(raw, allow_negative: bool = True) -> float | None:
    """
    Improved version: handles more edge cases.

    Args:
        raw: Raw value to clean (str, int, float, or None)
        allow_negative: If False, negative values are treated as None.

    Returns:
        Float value, or None if cannot be cleaned.
    """
    if raw is None:
        return None

    s = str(raw).strip()
    if not s:
        return None

    # 移除常見符號
    s = s.replace("$", "").replace(",", "")

    try:
        result = float(s)
        if not allow_negative and result < 0:
            return None
        return result
    except ValueError:
        return None

# 測試
print("=== v2 測試 ===")
for raw in raw_amounts:
    result = clean_amount_v2(raw)
    print(f"  {repr(raw):20} → {repr(result)}")
```

Step 4：套用到整個 list（5 分鐘）

```
# 用 list comprehension 處理全部
cleaned_amounts = [clean_amount_v2(v) for v in raw_amounts]

# 統計
valid_amounts = [a for a in cleaned_amounts if a is not None]
failed_count = sum(1 for a in cleaned_amounts if a is None)

print(f"原始：{len(raw_amounts)} 筆")
print(f"成功：{len(valid_amounts)} 筆")
print(f"失敗：{failed_count} 筆")
print(f"最小：{min(valid_amounts):.2f}　最大：{max(valid_amounts):.2f}　總和：{sum(valid_amounts)
```

工作坊 A 核心體驗： 清理函式的開發是「寫 → 測試 → 改進」的迭代過程，不是一次就能寫對的。重要的是：清理函式要能處理所有邊界情況，並且清楚說明「哪些情 None」。

S12 — 工作坊 B：把五個清理步驟封裝成函式庫

類型： 動手工作坊 時間： 30 分鐘 工具： Jupyter Notebook 資料： *`sales_raw.csv`*（課程頁面提供）

目標：
把一份原始資料的清理流程，從「一次性的程式碼」重構成「一組有名字、有說明、可以重用的函式」，體驗模

工作坊 B 任務說明

資料背景： `sales_raw.csv` 包含：訂單 ID（字串）、日期（字串）、商品類別（字串，大小寫不一致）、銷售金額（混合格式字串）、訂單數量（整數 "3.5%"）。

Step 1：寫函式一：資料結構摘要（6 分鐘）

```
import pandas as pd

def describe_dataset(df: pd.DataFrame) -> None:
    """
    Print a quick structural summary of a DataFrame.
    Includes shape, dtypes, and missing value counts.
    """
    print(f"Shape: {df.shape[0]} ■ × {df.shape[1]} ■")
    print(f"\n■■■■■■■:")
    print(df.dtypes.to_string())
    print(f"\n■■■■■:")
    missing = df.isna().sum()
    missing_pct = df.isna().mean() * 100
    for col in df.columns:
        if missing[col] > 0:
            print(f"  {col}: {missing[col]} ■ ({missing_pct[col]:.1f}%)")
        else:
            print(f"  {col}: ■■■")

df = pd.read_csv("sales_raw.csv")
describe_dataset(df)
```

Step 2：寫函式二：金額欄清理（6 分鐘）

```
def clean_amount_column(series: pd.Series) -> pd.Series:
    """
    Clean a currency string column into float.
    Handles formats: '$1,200', '1200.5', '1,000'.
    Values that cannot be converted become NaN.
    """
    return (
        series
        .astype(str)
        .str.strip()
        .str.replace(r"[$,]", "", regex=True)
        .pipe(pd.to_numeric, errors="coerce")
    )

df["amount_clean"] = clean_amount_column(df["amount"])
print("[redacted]")
print(df[["amount", "amount_clean"]].head(10))
print(f"\n[redacted] {df['amount_clean'].isna().sum()} [redacted]")
```

Step 3：寫函式三：類別標準化（5 分鐘）

```
def standardize_categories(series: pd.Series) -> pd.Series:
    """
    Standardize category strings: strip whitespace and lowercase.
    """
    return series.astype(str).str.strip().str.lower()

df["category_clean"] = standardize_categories(df["category"])
print("[redacted]", df["category"].unique())
print("[redacted]", df["category_clean"].unique())
```

Step 4：寫函式四：退貨率解析（5 分鐘）

```
def parse_percentage_column(series: pd.Series) -> pd.Series:
    """
    Parse percentage strings like '3.5%' into float (0.035).
    Returns NaN for values that cannot be parsed.
    """
    return (
        series
        .astype(str)
        .str.strip()
        .str.replace("%", "")
        .pipe(pd.to_numeric, errors="coerce")
        .div(100)
    )

df["return_rate_clean"] = parse_percentage_column(df["return_rate"])
```

```
print(df[["return_rate", "return_rate_clean"]].head())
```

Step 5：寫函式五：完整清理流程（8 分鐘）

把上面四個函式組合成一個「主清理函式」：

```
def clean_sales_dataframe(df: pd.DataFrame) -> pd.DataFrame:
    """
    Apply all cleaning steps to a raw sales DataFrame.
    Returns a new DataFrame with cleaned columns appended.
    Does NOT modify the original DataFrame.
    """
    result = df.copy()

    result["amount_clean"] = clean_amount_column(result["amount"])
    result["category_clean"] = standardize_categories(result["category"])
    result["return_rate_clean"] = parse_percentage_column(result["return_rate"])

    return result

# ■■■■■
df_clean = clean_sales_dataframe(df)

# ■■■■
print("=== ■■■■ ===")
print(f"■■■{len(df)} ■")
print(f"■■■■{len(df_clean)} ■■■■■■■■■■■■■■")
print(f"\n■■■■{[c for c in df_clean.columns if c not in df.columns]}")
```

Step 6：展示與討論（最後 5 分鐘）

講師引導討論： - 如果下週拿到新的銷售資料，這五個函式你還能用嗎？ - 如果業務規則改了（例如把「高價值」的門檻從 1000 改成 2000），需要改幾個地方？ - 函式庫和「把所有程式碼放在同一個 cell」的差異是什麼？

工作坊 B 核心體驗：模組化的清理函式不只是「好看的程式碼」，而是讓你下次遇到同樣資料結構時，只需要呼叫 `clean_sales_dataframe(new_df)` 就能完成清理。這就是 Python 思維的實際體現。

工作坊總結

工作坊	重點技能	預期輸出	核心體驗

工作坊 A	字串清理、型態轉換、邊界處理、函式設計、list comprehension	一個強健的 `clean_amount()` 函式，附帶完整的邊界測試	清理函式的開發是迭代過程，測試邊界
工作坊 B	函式組合、pandas 基本操作、模組化設計	五個可重用的清理函式，加上一個組合這些函式的模組，能讓你換一批新資料，重新跑	

講師注意事項

1. S01 開場 要花足夠時間強調「這堂課的邊界」。學員常常會問「為什麼不教 class？」主動說清楚原因：不是不重要，而是初學階段用不到，等你需要的時候你有足夠的基礎，學起來會
2. S02 浮點精度
通常讓學員意外，這個概念在金融計算中非常重要，值得多花一點時間讓它清楚，但不要深入浮點數的二進
3. S03 字串
是實際工作中最常用的部分，可以讓學員說說自己遇過哪些字串格式問題，用真實場景帶入。
4. S06 條件分支 要多強調「先考慮 None 和邊界情況」的習慣，這是初學者和有經驗者最明顯的差距之一。
5. S07 list comprehension 要反覆強調「等價性」，讓學員不要把它當成魔法，而是「有條件的 for 迴圈的簡寫」。
6. S08 函式 重點是「docstring 和邊界處理」，不是語法本身。花時間在「為什麼要這樣設計」，而不只是「怎麼寫」。
7. S09 錯誤閱讀 可以現場示範：故意寫出一個會報 TypeError 的程式碼，一起閱讀錯誤訊息，找出問題。
8. 工作坊 A 的核心是「測試邊界情況」，不只是「寫出能跑的函式」。確保學員測試了 None、空字串、負數這些邊界。
9. 工作坊 B 中的 pandas 語法（`.str.replace()`、`.pipe()`、`pd.to_numeric()`）學員可能不熟悉。可以說：「這些 pandas 操作的細節 M3 會詳細教，現在先看懂這些函式做什麼就好，不需要記住所有語法。」

課前準備清單（學員）

- [ ] 完成 M1 模組，特別是工作坊 B 的資料稽核報告
- [ ] 確認 Jupyter Notebook 環境可以正常執行
- [ ] 確認已安裝 pandas（`import pandas as pd` 不報錯）
- [ ] 下載範例資料 `sales_raw.csv`（課程頁面提供）

延伸閱讀

參考文獻與引用來源

Python 官方文件

1. Python Built-in Types — 型態系統完整說明。docs.python.org/3/library/stdtypes.html
2. Python Built-in Functions — `int()` / `float()` / `str()` / `len()` / `sum()` 等。docs.python.org/3/library/functions.html
3. PEP 8 — Style Guide for Python Code — 函式命名、docstring 規範。peps.python.org/pep-0008
4. Python 整數無大小限制 — Python 官方文件明確說明：「Integers have unlimited precision.」docs.python.org/3/library/stdtypes.html#numeric-types-int-float-complex

浮點數精度

1. IEEE 754 浮點數標準 — `0.1 + 0.2 != 0.3` 是 IEEE 754 二進制浮點數表示法的固有行為，非 Python 獨有。參見 floating-point-gui.de 與 Python 官方說明 docs.python.org/3/tutorial/floatingpoint.html

延伸閱讀

1. Ramalho, L. (2022). Fluent Python, 2nd Edition. O'Reilly Media. — 第一章資料模型，理解 Python 型態系統的設計哲學

2. Real Python. "List Comprehensions in Python" — comprehension 的完整教學，含效能比較。realpython.com/list-comprehension-python

---

Module 3：NumPy 與陣列思維 — 從數值計算到機器學習的底層語言

所屬課程： Python 數據分析與 AI 工程基礎
模組編號： M3
時長： 3 小時
難度： 中階入門
前置模組： M2 Python 分析基礎
後續模組： M4 pandas 與資料清理

一、模組定位

這個模組是你從「寫 Python」到「做資料運算」的第一道門。

M2 之前，你處理的是字串、串列、字典——通用的資料容器。
M3 之後，你操作的是陣列——為科學計算而生的數值引擎。

NumPy
不只是「讓計算快一點」的工具。它是機器學習框架的共通語言：PyTorch、TensorFlow、scikit-learn 底層全部以多維陣列為核心資料結構。你現在學的 `shape`、`axis`、`broadcasting`，正是日後看懂任何 ML 程式碼的鑰匙。

這個模組教你三件事：

1. 陣列思維：理解為何 AI/ML 用陣列而非串列，以及兩者的根本差異
2. 核心操作：shape 操控、broadcasting、向量化、線性代數基礎
3. 實戰工具：亂數生成、統計計算、切片模式、在 ML 工作流中的定位

二、學習目標

完成本模組後，學生能夠：

1. 說明 ndarray 與 Python list 的根本差異，解釋為何 ML 框架選擇陣列作為基本資料結構
2. 正確使用 `shape`、`axis`、`dtype` 描述與操作一個 NumPy 陣列
3. 理解 broadcasting 規則，寫出不含 for 迴圈的向量化運算
4. 使用 `reshape`、`transpose`、切片等操作靈活改變陣列的視角與形狀
5. 使用 NumPy 完成基本矩陣操作（乘法、轉置、逆矩陣）與統計摘要
6. 使用 NumPy 亂數工具建立模擬資料，支援資料科學實驗
7. 識別可以向量化的程式碼模式，並完成重寫
8. 說明 NumPy 在 ML 工作流中的定位，以及 NumPy 2.0 的重要改動

三、關鍵概念清單

學完本模組應能用自己的話解釋以下所有詞彙：

- [ ] ndarray vs Python list（記憶體結構的根本差異）
- [ ] shape 與 reshape（尺寸描述與重塑）
- [ ] axis（操作方向：行軸 vs 列軸）
- [ ] dtype（數值精度與記憶體的關係）
- [ ] broadcasting（不需要迴圈的條件與規則）
- [ ] vectorized operation（向量化操作，與 for 迴圈的語義差異）
- [ ] np.random（亂數生成與種子固定）
- [ ] 切片模式（基本切片、布林切片、花式索引）
- [ ] 矩陣乘法 `@` 與 `np.dot()`
- [ ] np.linalg（線性代數基本工具）
- [ ] 統計函式（mean、std、var、median、percentile）

- [ ] 特徵矩陣 X 與標籤向量 y 的慣例形狀
- [ ] NumPy 2.0 的重大 API 改動

四、投影片大綱（11 張）

編號	標題	核心主張
S1	你以為在學工具，其實在學模型的底層語言	NumPy 不是可選的，它是 ML 框架的地基
S2	ndarray：為科學計算設計的資料結構	list 是通用容器，ndarray 是計算引擎
S3	shape / axis / dtype：三個你必須看懂的概念	不理解這三個，就看不懂任何模型的 shape mismatch
S4	reshape 與切片：改變陣列的視角	資料本身沒有動，只是你換了看它的方式
S5	Broadcasting：讓資料自己跑起來	理解規則就能消滅 90% 的迴圈操作
S6	向量化思維：不是快，是對的	for 迴圈是思維問題，不只是效能問題
S7	亂數生成：資料科學的實驗工具	可重現的亂數是科學實驗的基礎
S8	統計計算：從陣列到摘要指標	NumPy 的統計函式是資料分析的第一層
S9	基礎線性代數：矩陣乘法與逆矩陣	ML 的核心計算是矩陣乘法，不是數字加減

S10	NumPy 在 ML/DL 的實際角色	從特徵矩陣到模型權重，NumPy 是共通語言
S11	里程碑：NumPy 2.0 — 2006 年以來第一個 major release	工具也有版本意識，演進不停歇

五、逐張投影片詳述

S1 — 你以為在學工具，其實在學模型的底層語言

核心主張：所有 ML/DL 框架的核心計算都建立在多維陣列操作上，NumPy 是理解這一切的最短路徑。

講師講解要點：

- 打開任何 PyTorch 或 TensorFlow 的教學，第一步幾乎都是 `import numpy as np`。為什麼？因為 tensor 的概念與操作幾乎直接繼承自 NumPy。
- 線性迴歸的預測公式 `y = Xw + b`，X 是特徵矩陣、w 是權重向量。沒有矩陣概念，你就只能背公式，無法理解為什麼這樣計算。
- NumPy 不是「讓你算快一點」的工具，而是讓你用正確的語言描述計算問題的工具。快只是副作用。
- 2024 年的 AI 工程師每天用的概念：batch size、feature dimension、embedding dimension，全部是 ndarray 的 shape 概念。
- 學 NumPy 的終點不是 NumPy，而是讓 PyTorch、JAX、TensorFlow 都變得可讀。

視覺建議：一張橫向對照表，左邊是數學符號（向量、矩陣、張量），右邊是對應的 NumPy 程式碼。中間畫箭頭，標示「你已經懂概念，只差語法」。底部附一行：「這堂課學完，你看得懂 ML 框架的原始碼」。

轉場：「但 Python 不是本來就有 list 嗎？為什麼需要 ndarray？下一張說清楚。」

S2 — ndarray：為科學計算設計的資料結構

核心主張：list 是通用容器，可以放任何東西；ndarray 是同質性的數值陣列，這個限制帶來根本性的效能與語義優勢。

講師講解要點：

- Python list 可以放 `[1, "hello", None, [2, 3]]`，這種彈性在記憶體上付出代價：每個元素是一個獨立的 Python 物件，有額外的類型標記、引用計數、記憶體位址。
- ndarray 只放一種型別（例如 float64），所有元素緊密排列在連續記憶體空間中，CPU 的快取命中率大幅提升。
- 效能差距在小資料看不出來，但在 100 萬筆資料時，list 的迭代可能慢 50-100 倍。
- `np.array([1,2,3])` vs `[1,2,3]`：外表相似，底層天壤之別。前者是 C 語言陣列包了一層 Python 介面，後者是 Python 物件指標的集合。
- 關鍵：ndarray 讓你把一個「對整個陣列的操作」送進去，而不是對每個元素分別操作。這就是向量化的基礎。

常見建立方式一覽：

```
import numpy as np

# ■ list ■■
a = np.array([1, 2, 3, 4, 5])

# ■■■■■■■■■■
zeros = np.zeros((3, 4))        # ■■■■
ones = np.ones((2, 3))          # ■■■■
eye = np.eye(4)                 # ■■■■
arange = np.arange(0, 10, 2)    # ■■ range■■■ ndarray
linspace = np.linspace(0, 1, 5) # ■■■ 5 ■■
```

視覺建議：記憶體示意圖。左邊畫 Python list：一排格子，每格指向分散的物件（含型別標記、值、引用計數）。右邊畫 ndarray：一排緊密排列的數值格子，直接是值，沒有額外開銷。底部加一行：「同樣 100 萬個 float，ndarray 用的記憶體約是 list 的 1/8」。

轉場：「有了 ndarray，接下來最重要的三個概念：shape、axis、dtype。理解這三個，你就能看懂任何框架的錯誤訊息。」

S3 — shape / axis / dtype：三個你必須看懂的概念

核心主張： shape 描述尺寸、axis 描述操作方向、dtype 描述元素型別。這三個概念是所有後續操作的語言基礎。

講師講解要點：

- shape：`arr.shape` 回傳一個 tuple，例如 `(100, 28, 28)` 表示 100 張 28×28 的灰階影像。shape 不只是「大小」，它是資料的意義標記。在 ML 裡，`(batch_size, features)` 是你最常看到的 2D shape。
- axis：`np.sum(arr, axis=0)` 是沿著第 0 軸加總，對矩陣來說就是對每一列加總得到一行向量。`axis=1` 則是對每一行加總。關鍵記憶法：axis 是你要「壓縮掉」的那個維度。
- dtype：`float64` 精度高但佔記憶體，`float32` 是深度學習的標準選擇（GPU 的原生精度），`int8` 用於模型量化。型別不只影響記憶體，也影響計算精度。

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr.shape)  # (2, 3) — 2 ■ 3 ■
print(arr.dtype)  # int64■■■■
print(arr.ndim)   # 2 — ■■■■

■ axis ■■
print(arr.sum(axis=0))  # [5, 7, 9]  — ■■■■■■■
print(arr.sum(axis=1))  # [6, 15]    — ■■■■■■■

■ ■■■■
arr_f32 = arr.astype(np.float32)
```

視覺建議： 三個並排的方塊示意圖。第一個：一個 3D 立方體，標示三個軸的方向。第二個：一個 2D 矩陣，展示 `sum(axis=0)` 與 `sum(axis=1)` 的結果方向。第三個：dtype 精度對照表（float64 / float32 / int32 / int8 + 記憶體占用 + 使用場景）。

轉場：「理解 shape 之後，我們看如何重塑陣列的形狀，以及如何精確地取出你要的部分。」

S4 — reshape 與切片：改變陣列的視角

核心主張： reshape 讓你改變陣列的維度結構而不移動資料；切片讓你精確地取出子集。這兩個操作在 ML 資料預處理中每天都在用。

講師講解要點：

- reshape：`arr.reshape(3, 4)` 把陣列重塑為 3 列 4 欄，但資料本身沒有複製，只是換了一個「看的方式」（view）。`-1` 是魔法數字：`arr.reshape(-1, 1)` 表示「欄數為 1，列數自動計算」，在 scikit-learn 餵資料時非常常用。
- flatten 與 ravel：`arr.flatten()` 壓平成一維，會複製資料；`arr.ravel()` 也壓平但盡量回傳 view（不複製）。
- 基本切片：與 Python list 相同語法，但支援多維。`arr[0, :]` 取第 0 列所有欄；`arr[:, 2]` 取所有列的第 2 欄；`arr[1:3, 0:2]` 取子矩陣。
- 布林切片：`arr[arr > 3]` 回傳所有大於 3 的元素（攤平為一維）；`arr[mask]` 支援任意布林陣列。這是資料篩選的核心操作。
- 花式索引（Fancy Indexing）：`arr[[0, 2, 4]]` 取第 0、2、4 列，回傳的是複製（不是 view）。

```
arr = np.arange(12)

# reshape
matrix = arr.reshape(3, 4)
col_vector = arr.reshape(-1, 1)  # shape: (12, 1)

# 切片
print(matrix[0, :])      # 第 0 列
print(matrix[:, -1])     # 最後一欄
print(matrix[0:2, 1:3])  # 前兩列 2×2 子矩陣

# 布林切片
mask = matrix > 6
print(matrix[mask])      # [7, 8, 9, 10, 11]

# 轉置
print(matrix.T.shape)    # (4, 3)
```

視覺建議： 一個 4×4 的矩陣示意圖，用不同顏色區塊標示不同切片模式的結果（第 0 列、第 2 欄、左上子矩陣、對角線）。旁邊附對應的程式碼。底部加一行：「切片回傳的是 view，修改它會影響原始陣列——這是常見的 bug 來源」。

轉場：「理解切片之後，來看 NumPy 最神奇的能力：broadcasting。它讓不同形狀的陣列可以直接運算，不需要迴圈。」

S5 — Broadcasting：讓資料自己跑起來

核心主張：Broadcasting 是 NumPy 的核心魔法，允許不同 shape 的陣列在特定規則下進行操作，消滅大量重複的迴圈程式碼。

講師講解要點：

- 最簡單的案例：`arr + 5`，一個陣列加一個純量。你沒有寫迴圈，NumPy 自動把 5「廣播」到每個元素。這是 broadcasting 的最基本形式。
- 更複雜的案例：一個 `(3, 4)` 的矩陣加一個 `(1, 4)` 的向量，向量會沿著第 0 軸廣播，等效於把向量複製三份再相加。但 NumPy 實際上不複製記憶體，只是在計算時虛擬地廣播。
- Broadcasting 規則（從右往左對齊）：
- 如果兩個維度相同，可以直接操作
- 如果一個維度是 1，可以廣播
- 如果兩個維度不同且都不是 1，報錯
- 實際應用：特徵標準化。一行搞定，不需要任何迴圈：

```
X = np.random.randn(100, 5)  # 100 ■■■■5 ■■■

■■■■■■■■■■■■■■■■■■■■
X_mean = X.mean(axis=0)   # shape: (5,)
X_std = X.std(axis=0)     # shape: (5,)
X_normalized = (X - X_mean) / X_std  # broadcasting ■■■■
# X shape (100,5), X_mean shape (5,) → broadcasting ■ axis=0 ■■
```

- 注意事項：broadcasting 的錯誤訊息是 `operands could not be broadcast together with shapes (3,4) (4,)`，看到這個，先從右到左對齊兩個 shape，找出不相容的維度。

視覺建議：動態圖示（靜態版）：左邊一個 `(3, 4)` 的藍色矩陣，右邊一個 `(1, 4)` 的橙色向量，用虛線箭頭表示橙色向量「複製」三份覆蓋到藍色矩陣上，最後變成一個綠色的 `(3, 4)` 結果矩陣。底部加一個 broadcasting 規則的簡表（合法 vs 非法的 shape 組合各三例）。

轉場：「Broadcasting 讓你不用寫迴圈，但為什麼不用迴圈這件事如此重要？下一張說向量化思維。」

S6 — 向量化思維：不是快，是對的

核心主張：用 for 迴圈處理陣列操作不只是效能問題，而是用了錯誤的抽象層次描述問題。向量化是讓程式碼的語義與數學意圖對

講師講解要點：

- 以計算兩個向量的點積為例，展示三個版本：Python for 迴圈版、NumPy 向量化版、`np.dot()` 版。效能差距可達 100 倍，但更重要的是：哪一個最清楚說明「這是點積運算」？

```
a = np.array([1.0, 2.0, 3.0, 4.0])
b = np.array([5.0, 6.0, 7.0, 8.0])

# ── for 迴圈版 ──
result = 0
for i in range(len(a)):
    result += a[i] * b[i]

# ── 向量化版 ──
result = np.sum(a * b)

# ── dot 版 ──
result = np.dot(a, b)
```

- 向量化迫使你用「整個陣列」來思考，而不是「每個元素」。這和 ML 框架的思維一致：你不操作單一樣本，你操作一個 batch。
- 常見的向量化替換模式：
- `np.where()` 替換條件迴圈（類似三元運算子，但作用在整個陣列）
- `np.cumsum()` 替換累加迴圈
- `np.maximum(0, arr)` 實作 ReLU，不需要任何 if
- 識別「可以向量化」的信號：如果你的 for 迴圈在遍歷 ndarray 的每個元素做相同操作，幾乎都可以向量化。
- 反例：遞歸依賴（後一個值依賴前一個值的計算結果）無法向量化。不要盲目去除所有迴圈。

視覺建議： 左右對比程式碼塊。左邊：8 行的 for 迴圈版。右邊：1 行的 `np.dot(a, b)`。底部加時間比較條形圖（100 萬維向量）：for 迴圈約 2500ms vs NumPy 約 5ms。右邊下方標注：「更短、更清晰、更接近數學原意」。

轉場：「向量化思維建立之後，我們來看一個實際工作中天天用到的工具：亂數生成。」

S7 — 亂數生成：資料科學的實驗工具

核心主張： 可重現的亂數是科學實驗的基礎。NumPy 的亂數工具讓你能建立模擬資料、初始化模型、驗證演算法。

講師講解要點：

- 亂數在資料科學中的使用場景：建立測試資料集、模擬實驗、初始化神經網路權重、交叉驗證的資料分割、抽樣。
- 固定種子（seed）：可重現性是科學實驗的基本要求。`np.random.seed(42)` 固定全域種子，但更推薦使用現代的 Generator API：

```
# [redacted] NumPy 1.17+[redacted]
rng = np.random.default_rng(seed=42)

# [redacted]
normal = rng.normal(loc=0, scale=1, size=(100, 5))  # [redacted]
uniform = rng.uniform(low=0, high=1, size=(100,))  # [redacted]
integers = rng.integers(low=0, high=10, size=(50,))  # [redacted]

# [redacted]
np.random.seed(42)
x = np.random.randn(100, 5)  # [redacted]
y = np.random.randint(0, 2, 100)  # [redacted]

# [redacted]
arr = np.arange(100)
sample = rng.choice(arr, size=10, replace=False)  # [redacted]
shuffled = rng.permutation(arr)  # [redacted]
```

- `np.random.randn` vs `np.random.rand`：前者是標準常態分布（均值 0，標準差 1），後者是 [0,1) 的均勻分布。混用是初學者的常見錯誤。

- 工程意義：在機器學習實驗中，固定 seed 讓結果可重現，報告結果時應說明使用的 seed 值。

視覺建議：兩個並排的直方圖示意：左邊是常態分布 `randn`，右邊是均勻分布 `rand`，各自標示均值與範圍。底部附一個 seed 固定的示意圖：「相同的 seed → 相同的序列 → 結果可重現」。

轉場：「有了資料之後，接下來是從陣列中提取統計摘要。」

S8 — 統計計算：從陣列到摘要指標

核心主張：NumPy 內建完整的統計函式，是資料分析的第一層工具。理解 axis 參數讓你能沿任意維度計算統計量。

講師講解要點：

- NumPy 的統計函式幾乎都支援 `axis` 參數，讓你能對矩陣的行或列分別計算：

```
data = np.array([
    [85, 92, 78, 90],   # 學生 A 四科成績
    [70, 88, 95, 72],   # 學生 B
    [93, 79, 85, 88],   # 學生 C
])

# 全域統計
print(data.mean())          # 整個陣列的平均：83.7
print(data.mean(axis=1))    # 每位學生的平均：[86.25, 81.25, 86.25]
print(data.mean(axis=0))    # 每科的平均：[82.67, 86.33, 86.0, 83.33]

print(data.std(axis=1))     # 每位學生的標準差
print(data.max(axis=0))     # 每科的最高分
print(data.min(axis=1))     # 每位學生的最低分

# 分位數
print(np.percentile(data, 75))          # 整個陣列的 75 分位數
print(np.percentile(data, [25, 50, 75]))  # 四分位數

# 排序
print(np.sort(data, axis=1))            # 每列內部排序
print(np.argsort(data, axis=1))         # 每列排序的索引

# 累積
print(np.cumsum(np.array([1, 2, 3, 4])))  # [1, 3, 6, 10]
```

- `np.percentile` 是分析資料分布的重要工具，也是後續離群值檢測（IQR 方法）的基礎。
- `np.argsort` 回傳排序後的索引，常用於找出「第幾個特徵最大」。

視覺建議： 一個 3×4 的成績矩陣示意圖，用不同顏色箭頭標示 `axis=0`（垂直壓縮，得到 4 個科目均值）和 `axis=1`（水平壓縮，得到 3 個學生均值）的方向。旁邊附對應的數字結果。

轉場：「統計計算之後，我們看 ML 最核心的數學操作：矩陣乘法。」

S9 — 基礎線性代數：矩陣乘法與逆矩陣

核心主張： 機器學習的核心計算是矩陣乘法，不是數字加減。`@` 運算符和 `np.linalg` 是這一層的工具。

講師講解要點：

- 矩陣乘法：ML 中每一層神經網路的前向傳播就是 `output = W @ input + b`，W 是權重矩陣、`@` 是矩陣乘法。

```
# [redacted]
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

C1 = A @ B              # [redacted]Python 3.5+[redacted]
C2 = np.dot(A, B)       # [redacted]
C3 = np.matmul(A, B)    # [redacted]

print(C1)
# [[19 22]
#  [43 50]]

# shape [redacted] (m, k) @ (k, n) → (m, n)
# A.shape = (2,2), B.shape = (2,2) → C.shape = (2,2)
```

- 轉置：`A.T` 或 `A.transpose()`。在線性迴歸的正規方程 `w = (X^T X)^{-1} X^T y` 中，轉置是關鍵步驟。
- 常用 linalg 工具：

```
# [redacted]
det = np.linalg.det(A)

# [redacted]
A_inv = np.linalg.inv(A)
print(A @ A_inv)  # [redacted]

# [redacted]PCA [redacted]
eigenvalues, eigenvectors = np.linalg.eig(A)

# [redacted] Ax = b
b = np.array([1, 2])
x = np.linalg.solve(A, b)

# [redacted]
v = np.array([3.0, 4.0])
print(np.linalg.norm(v))  # 5.0
```

- 對初學者最重要的是理解矩陣乘法的 shape 規則：`(m, k) @ (k, n) → (m, n)`。中間維度必須相同，這是 shape mismatch 最常見的來源。

視覺建議： 一張矩陣乘法的 shape 示意圖：兩個矩陣並排，標示 m、k、k、n 的對應關係，箭頭指向輸出矩陣的 shape (m, n)。旁邊附一個「常見 shape mismatch 錯誤」的解讀示範。

轉場：「所有這些操作在 ML 工作流裡是如何串起來的？」

S10 — NumPy 在 ML/DL 的實際角色

核心主張： NumPy 是 ML 工作流的共通貨幣，從特徵矩陣到模型權重，都以陣列形式流通。

講師講解要點：

- 一個典型的 ML 流程：載入資料 → 預處理 → 建立特徵矩陣 X（shape: `(n_samples, n_features)`）→ 建立標籤向量 y（shape: `(n_samples,)`）→ 送入模型訓練。X 和 y 幾乎永遠是 NumPy ndarray 或與其相容的格式。
- scikit-learn 的所有估計器（Estimator）都接受 NumPy 陣列作為輸入。`clf.fit(X, y)` 中的 X 就是一個 `(n_samples, n_features)` 的 ndarray。
- PyTorch 的 tensor 可以直接從 NumPy 陣列轉換：`torch.from_numpy(arr)`，且在 CPU 上共享記憶體（zero-copy）。PyTorch 的設計刻意與 NumPy 保持介面一致性。

- 深度學習的核心是矩陣乘法：每一層的前向傳播就是 `output = activation(W @ input + b)`。
- 工程意義：學會 NumPy 之後，你看 ML 框架的原始碼不再是天書。`(batch_size, seq_len, hidden_dim)` 這種三維 tensor 的 shape，就是 ndarray 概念的直接延伸。

視覺建議：一張 ML 工作流管道圖，從「原始資料（CSV）」出發，標示每個節點的資料形態與工具：pandas DataFrame → NumPy ndarray（X, y）→ scikit-learn / PyTorch → model output。每個轉換箭頭上標示對應的程式碼（`.to_numpy()`、`torch.from_numpy()` 等）。

轉場：「在進入練習之前，我們先看一個工具演進的里程碑。」

S11 — 里程碑：NumPy 2.0 — 2006 年以來第一個 major release

核心主張： NumPy 2.0（2024 年發布）是 18 年來第一個 major version 升級，代表科學計算 Python 生態在成熟後仍保持演進的能力。

講師講解要點：

- NumPy 1.0 於 2006 年發布。此後 18 年都以 1.x 的形式發展，每個版本都保持嚴格的向後相容性。2024 年的 2.0 是一個明確信號：生態願意接受破壞性變更來提升品質。
- NumPy 2.0 的三個重要改動：
- 清理了大量已棄用的 API，移除長達 10 年的「deprecated 但沒刪」函式
- 提升 Python 類型標註的覆蓋率，讓 IDE 靜態分析更準確
- 字串類型的大幅改進，加入 `StringDType`，讓字串陣列效能大幅提升
- 對學習者的實際意義：如果你搜尋到的教學使用了 `np.bool`、`np.int`、`np.float` 等裸型別名稱，那是 NumPy 1.x 的語法。在 2.0 中這些已被移除，應改用 Python 原生型別 `bool`、`int`、`float` 或 NumPy 特定型別如 `np.float64`。

NumPy 1.x 舊語法	NumPy 2.0 正確語法

`np.bool`	`bool` 或 `np.bool_`
`np.int`	`int` 或 `np.int_`
`np.float`	`float` 或 `np.float64`
`np.complex`	`complex` 或 `np.complex128`

- 工具有版本意識是工程師的基本素養。不要只學「如何用」，也要有能力識別「這是哪個版本的用法」。

視覺建議： 一條時間軸，從 2006 年的 NumPy 1.0 到 2024 年的 NumPy 2.0，中間標示幾個重要的 1.x 里程碑。右側用醒目框格標示 2.0 的三個主要改動。底部加上上方的 API 變更對照表。

六、練習設計

練習 1：ndarray 基礎操作與切片 （S2-S4 結束後，25 分鐘）

情境說明：
你正在準備一組學生成績資料，用於後續的機器學習實驗。需要完成一系列陣列操作，熟悉 NumPy 的基本工具。

任務清單：

1. 建立一個 shape 為 `(6, 5)` 的成績矩陣，使用 `np.random.default_rng(42).integers(60, 100, size=(6, 5))`，代表 6 名學生的 5 科成績
2. 查看陣列的 `shape`、`dtype`、`ndim`，並轉換型別為 `float32`
3. 取出第 0 名學生的所有成績（第 0 列）、所有學生的第 2 科成績（第 2 欄）
4. 取出前 3 名學生的後 3 科成績（子矩陣切片）
5. 找出所有低於 70 分的成績（布林切片），並統計有幾筆
6. 將低於 70 分的成績替換為 70（使用布林索引賦值）

7. 計算每位學生的總分與平均分（使用 `axis=1`），找出總分最高的學生（使用 `np.argmax`）
8. 用 `reshape` 將整個成績矩陣重塑為 `(1, 30)` 的一維格式，再重塑回 `(6, 5)`

延伸思考：

```
arr[mask] = 70
```

這個操作修改了原始陣列。如果你想保留原始資料同時得到修改後的版本，應該怎麼做？（提示：`arr.copy()`）

驗收標準： - 每個操作都能正確印出預期的 shape 與數值 - 能解釋 axis=0 vs axis=1 的方向差異 - reshape 操作前後元素總數一致（6×5 = 30）

練習 2：向量化與 Broadcasting（S5-S6 結束後，25 分鐘）

情境說明：
你是一個機器學習工程師，需要從頭建立一個玩具版本的特徵矩陣，並用向量化操作完成所有預處理，不使用任 for 迴圈。

任務清單：

1. 建立 shape 為 `(8, 4)` 的隨機浮點矩陣 X（使用 `np.random.default_rng(0).normal(size=(8,4))`），代表 8 個樣本、4 個特徵
2. 計算每個特徵（每欄）的均值與標準差，確認 shape 為 `(4,)`
3. 對 X 進行特徵標準化：`(X - X.mean(axis=0)) / X.std(axis=0)`，用向量化一行完成，不使用任何迴圈
4. 確認標準化後每欄的均值接近 0、標準差接近 1（使用 `np.allclose`）
5. 使用 `np.where(X_normalized > 0, X_normalized, 0)` 實作 ReLU 激活函式（負值歸零）
6. 計算 `X_normalized.T @ X_normalized`，說明結果的 shape 與這個計算在統計上的意義（共變異矩陣）
7. 用 `np.linalg.norm(X_normalized, axis=1)` 計算每個樣本的 L2 範數（歐氏長度）
8. Broadcasting 練習：建立一個 `(4,)` 的偏移向量 `bias = np.array([1.0, -1.0, 0.5, -0.5])`，將每個樣本加上這個偏移（不使用迴圈，驗證 shape 廣播正確）

延伸思考：
特徵標準化在機器學習裡為什麼重要？當兩個特徵的數值範圍差距很大（例如年齡 20-60 vs 薪資

30000-100000），不標準化會對梯度下降產生什麼影響？

驗收標準： - 整個練習沒有使用任何 for 迴圈 - 能正確解釋每個操作的 shape 變化 - `np.allclose` 驗證標準化結果通過

練習 3：統計計算與矩陣操作綜合實戰（S7-S9 結束後，35 分鐘）

情境說明：
你需要模擬一份線性迴歸的資料集，並用 NumPy 的矩陣操作手動實作線性迴歸的正規方程（Normal Equation）求解，驗證結果。

任務清單：

Part A：亂數資料生成與統計摘要

1. 固定種子 `rng = np.random.default_rng(seed=2024)`
2. 生成 50 個樣本的特徵矩陣：`X_raw = rng.normal(loc=[5, 10, 3], scale=[2, 5, 1], size=(50, 3))`（3 個特徵，各有不同的均值與標準差）
3. 計算每個特徵的均值、標準差、最小值、最大值、第 25/50/75 百分位數，整理成一份摘要表（可以用 print 輸出）
4. 標準化 `X_raw` 為 `X`（每欄減均值除標準差）
5. 確認標準化後的均值接近 0、標準差接近 1

Part B：手動線性迴歸

1. 生成真實權重 `w_true = np.array([2.0, -1.5, 3.0])`，偏差 `b_true = 5.0`
2. 生成標籤：`y = X @ w_true + b_true + rng.normal(0, 0.5, size=(50,))`（加入少量噪聲）
3. 在 X 的左側加入全 1 欄位（偏差項）：`X_with_bias = np.hstack([np.ones((50,1)), X])`，確認 shape 為 `(50, 4)`
4. 用正規方程求解：`w_hat = np.linalg.inv(X_with_bias.T @ X_with_bias) @ X_with_bias.T @ y`
5. 印出估計的 `w_hat`，對比真實的 `[b_true, *w_true]`，說明為什麼估計值與真實值接近但不完全相同

驗收標準： - 能正確生成具有指定分布的資料 - `np.hstack` 拼接後的 shape 正確 - 正規方程的計算不報錯，估計值合理（與真實值誤差在 0.3 以內） - 能解釋噪聲的存在為何導致估計值不完全等於真實值

七、模組里程碑卡

```
[illegible] — NumPy 2.0 [illegible] 2024 [illegible] 6 [illegible]
│
◆NumPy 1.0 [illegible] 2006 [illegible]
[illegible] 18 [illegible] 1.x [illegible]
2024 [illegible] 2.0 [illegible]
[illegible] API [illegible]
[illegible]
│
[illegible]
• 18 [illegible] 1.0 [illegible] 2.0 [illegible]
• [illegible] 100+ [illegible] API
• StringDType [illegible]
• [illegible]
│
[illegible]
[illegible] np.bool / np.int / np.float
[illegible] NumPy 2.0 [illegible]
[illegible]
```

八、本模組結語

NumPy 不是 Python 資料分析的「前菜」，它是整個 AI/ML 生態的基礎語言。

你現在理解的每一個概念——shape、broadcasting、向量化——在你日後閱讀 PyTorch 原始碼、調試 scikit-learn 模型、理解 batch normalization 的實作時，都會再次出現。

下一個模組 M4 pandas 與資料清理，建立在 NumPy 的基礎上，把結構化表格資料（有欄名、有型別、有索引）帶入你的工具箱。

參考文獻與引用來源

NumPy 官方資源

1. NumPy 2.0.0 Release Notes (June 16, 2024) — 自 2006 年 NumPy 1.0 以來首個 major release，經 11 個月開發、212 位貢獻者、1,078 個 PR。約 100 個主命名空間成員被棄用、移除或搬遷。numpy.org/doc/stable/release/2.0.0-notes.html
2. NumPy 2.0 Migration Guide — 詳細的 API 變更對照表與自動遷移工具（Ruff 規則 NPY201）。numpy.org/devdocs/numpy_2_0_migration_guide.html
3. NumPy Official Documentation — ndarray、broadcasting、dtype 完整說明。numpy.org/doc/stable

效能數據說明

1. ndarray vs list 效能差距 — 本課程引用的「50-100 倍速度差距」與「1/8 記憶體佔用」為教學用估算值，基於連續記憶體佈局（cache locality）與固定 dtype 帶來的優勢。精確數值依操作類型、資料規模與硬體而異。可用 `%timeit` 與 `sys.getsizeof()` 自行驗證。
2. 向量化 vs for 迴圈 — NumPy 向量化操作透過底層 C 實作與 SIMD 指令集加速，避免 Python 直譯器的逐元素開銷。效能差距在大陣列（> 10,000 元素）時最為顯著。

延伸閱讀

1. VanderPlas, J. (2016). Python Data Science Handbook. O'Reilly Media. — Chapter 2: Introduction to NumPy. 免費線上版：jakevdp.github.io/PythonDataScienceHandbook
2. Harris, C.R. et al. (2020). "Array programming with NumPy." Nature, 585, 357-362. — NumPy 的學術論文，發表於 Nature。doi.org/10.1038/s41586-020-2649-2

Module 4：pandas 與資料清理 — 從原始表格到可分析資料

所屬課程： Python 數據分析與 AI 工程基礎
模組編號： M4
時長： 3 小時
難度： 中階入門
前置模組： M3 NumPy 與陣列思維
後續模組： M5 視覺化與 EDA

一、模組定位

NumPy 給了你計算引擎，pandas 給了你工作台。

真實世界的資料不是整齊的數值矩陣：它有欄位名稱、有混合型別、有缺失值、有格式不統一的日期欄、有意義
pandas 完成這段路程。

這個模組教你四件事：

1. 認識 DataFrame：結構、索引、型別推斷的陷阱
2. 分析操作四步驟：篩選、排序、聚合、轉換
3. 資料整合：merge/join、concat、pivot table、時間序列基礎
4. 資料清理：缺失值、離群值、型別修正，把「髒資料」變成「可信資料」

二、學習目標

完成本模組後，學生能夠：

1. 說明 DataFrame 與 Series 的結構，以及 index 在 pandas 中扮演的角色
2. 執行完整的資料初始偵察流程（5 個必備函式）
3. 熟練使用 boolean indexing、sort_values、groupby + agg 處理業務分析問題
4. 使用 merge/join 合併多個 DataFrame，理解不同 join 類型的語意差異
5. 建立 pivot table 完成交叉統計分析
6. 處理時間序列資料的基本操作（解析日期、計算時間差、重採樣）
7. 識別並處理缺失值與離群值，使資料具備可分析狀態
8. 說明 pandas 2.0 的 Copy-on-Write 機制與 Arrow backend 的意義

三、關鍵概念清單

學完本模組應能用自己的話解釋以下所有詞彙：

- [ ] DataFrame vs Series（結構與關係）
- [ ] index（索引不是欄位，是列的身份證）
- [ ] dtype inference（型別推斷陷阱：為什麼 CSV 讀進來型別可能不對）
- [ ] boolean indexing（布林篩選，`&` 而非 `and`）
- [ ] .groupby() 的拆-套-合邏輯（split-apply-combine）
- [ ] merge vs concat（合併的兩種方式）
- [ ] left / right / inner / outer join（四種 join 的語意差異）
- [ ] pivot_table（交叉統計）
- [ ] datetime64 與 pd.to_datetime（時間序列的第一步）
- [ ] 缺失值的三種來源與四種處理策略
- [ ] IQR 離群值識別法
- [ ] Copy-on-Write（pandas 2.0 的鏈式賦值問題）
- [ ] Arrow backend（為什麼記憶體減半）
- [ ] 工具規模感：pandas / Polars / DuckDB / Spark 的邊界

四、投影片大綱（12 張）

編號	標題	核心主張
S1	DataFrame：分析世界的工作台	表格不是 Excel，是帶型別的計算物件
S2	讀進來、看一眼、查型別：每個專案開始之前的第一件事	開始之前，先摸清楚手上的材料
S3	篩選與排序：精確地取出你要的資料	boolean indexing 是 pandas 的核心操作
S4	聚合與轉換：從資料到洞察的橋樑	groupby 的本質是業務切片思維
S5	merge 與 concat：把多張表合成一張	合併資料是現實工作中最常見的操作之一
S6	pivot_table：從長格式到交叉分析	換一個視角看資料，問題就清晰了
S7	時間序列基礎：讓日期欄真的變成日期	時間是資料中最特殊的維度
S8	缺失值：識別、理解、處理	沒處理的缺失值是分析毒藥
S9	離群值：識別、判斷、決策	不是所有極端值都是錯誤，但你必須知道它們在
S10	型別修正與資料格式統一	清理的本質是讓資料「說的語言」統一
S11	里程碑：pandas 2.0 — Copy-on-Write 與 Arrow backend	架構層級的升級解決了長達十年的語意問題

五、逐張投影片詳述

S1 — DataFrame：分析世界的工作台

核心主張： DataFrame 不是試算表軟體的替代品，而是一個帶有型別系統、索引機制與豐富操作 API 的計算物件。

講師講解要點：

- DataFrame 的核心結構：列（rows）與欄（columns）。每個欄是一個 Series（一維帶標籤陣列），每個 Series 有獨立的 dtype。這與 ndarray 不同：ndarray 整個只能有一個 dtype，DataFrame 每欄可以不同。
- Index 的概念：index 不是一個欄位，它是列的「身份證」。預設是 0, 1, 2... 但可以是日期、字串、多層級索引。很多初學者因為不理解 index 而在 merge/join 時遇到意想不到的結果。
- DataFrame 是「帶標籤的矩陣」：分析時用欄名操作，計算時底層是 ndarray（或 Arrow array）。
- 建立 DataFrame 的三種常見方式：

```
import pandas as pd
import numpy as np

# 從字典建立
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie'],
    'age': [25, 30, 35],
    'salary': [50000.0, 75000.0, 60000.0],
    'is_manager': [False, True, False]
})

# 從 CSV 讀取
df = pd.read_csv('data.csv')
```

```
# [redacted] NumPy [redacted]
arr = np.random.randn(4, 3)
df = pd.DataFrame(arr, columns=['A', 'B', 'C'])
[redacted]
# DataFrame [redacted]
print(df.shape)          # ([redacted], [redacted])
print(df.columns)        # [redacted]
print(df.index)          # [redacted]
print(df.dtypes)         # [redacted]
```

視覺建議：一個 DataFrame 的解剖圖：左側標出 index 欄（帶標籤）、頂部標出 columns（欄名）、每欄下方標出 dtype、每個 Series 用不同顏色框起。旁邊附與 Excel 試算表的對比說明，強調「DataFrame 的欄有型別，Excel 的格沒有」。

轉場：「有了 DataFrame，第一件事是什麼？讀進資料，然後看一眼手上的材料。」

S2 — 讀進來、看一眼、查型別：每個專案都從這裡開始

核心主張：
資料分析的第一步不是計算，是偵察。在對資料一無所知時，有一套標準的「初次見面流程」。

講師講解要點：

- 五個必備的初始偵察函式：

```
df.head()        # [redacted] 5 [redacted]
df.info()        # [redacted]
df.describe()    # [redacted]count/mean/std/min/max[redacted]
df.shape         # ([redacted], [redacted])
df.dtypes        # [redacted]
```

- **`df.info()`**
  是最重要的那一個：它同時告訴你欄名、型別、非空數量（從而反推缺失數量）、記憶體使用量。一眼看完
  object）。
- 型別推斷陷阱：pandas 讀 CSV 時會猜型別。常見問題：
- 含有「NT$1,200」的金額欄被讀成 object 而非 float
- 含有「2023-01-15」的日期欄被讀成 object 而非 datetime64
- 含有整數的欄位因為有一個缺失值而變成 float64（因為 NaN 是 float）

- 含有 "True"/"False" 字串的布林欄被讀成 object
- `df.value_counts()` 用於類別型欄位的分布快覽，是 `describe()` 對非數值欄位的補充。
- 培養「資料第一眼的直覺」：看完 `info()` 後，腦子裡要有一張評估表：缺失值多嗎？有沒有型別異常？欄位名稱有沒有空格或特殊字元？樣本量夠嗎

視覺建議：一份模擬的 `df.info()` 輸出截圖，用不同顏色標注關鍵資訊：欄名（藍）、dtype（橙）、非空計數（綠，並標示哪些有缺失）、記憶體 checklist：五個函式依序列出，每個旁邊標注「告訴你什麼」。

轉場：「確認資料長什麼樣子之後，接下來是分析師的日常：篩選和排序。」

S3 — 篩選與排序：精確地取出你要的資料

核心主張：boolean indexing 是 pandas 的核心操作語言。掌握篩選與排序，就能從任意角度切入資料。

講師講解要點：

- 篩選（Boolean Indexing）：

```
# 單條件
young = df[df['age'] < 30]

# 多條件用 & 和 | 不用 and / or
# 括號不可省略
result = df[(df['age'] > 25) & (df['salary'] > 60000)]

# 用 query 語法
result = df.query("age > 25 and salary > 60000")

# 成員
result = df[df['department'].isin(['Engineering', 'Design'])]

# 字串
result = df[df['name'].str.contains('Alice')]
result = df[df['city'].str.startswith('Taipei')]
```

- 排序（Sorting）：

```
# [redacted]
df.sort_values('salary', ascending=False)

# [redacted]
df.sort_values(['department', 'salary'], ascending=[True, False])

# [redacted] index [redacted]
df.sort_index()
```

- `sort_values` 預設回傳新的 DataFrame（不改動原始資料）。這是 pandas 的一般設計原則：大部分操作回傳新物件。
- `loc` vs `iloc`：`loc` 用標籤索引（欄名、index 值），`iloc` 用整數位置索引。

```
df.loc[0, 'salary']        # [redacted] 0 [redacted] salary [redacted]
df.iloc[0, 2]              # [redacted] 0 [redacted] 2 [redacted]
df.loc[0:5, ['name', 'salary']]  # [redacted] 0 [redacted]
```

視覺建議：一個 DataFrame 示意圖，用不同顏色高亮標示篩選結果的列（符合條件的）、排序後的順序箭頭、loc/iloc 取值的目標格子。旁邊附常見的錯誤：`df[df['age'] > 25 and df['salary'] > 60000]` 會報錯，解釋原因並給出正確寫法。

轉場：「篩選之後，我們看更強大的聚合工具：groupby。」

S4 — 聚合與轉換：從資料到洞察的橋樑

核心主張：groupby 的本質是「split-apply-combine」三步驟，它把資料問題轉換成業務問題：按什麼維度切、對每個切片做什麼、

講師講解要點：

- groupby 三步驟：Split（按欄位分組）→ Apply（對每組計算）→ Combine（合併結果）

```
# [redacted] groupby
df.groupby('department')['salary'].mean()
[redacted]
# [redacted]
df.groupby(['region', 'category'])['revenue'].sum()
```

```
# agg 多欄位聚合
df.groupby('department').agg({
    'salary': ['mean', 'std', 'count'],
    'performance': 'median'
})

# agg 搭配 pandas 1.1+ 命名
df.groupby('department').agg(
    avg_salary=('salary', 'mean'),
    num_employees=('salary', 'count'),
    med_performance=('performance', 'median')
)
```

- transform vs agg 的差異：

```
# agg 會縮成一列
dept_avg = df.groupby('department')['salary'].mean()  # 長度 = 部門數

# transform 會保留原 df 長度
# 適合用來加欄位
df['salary_ratio'] = df['salary'] / df.groupby('department')['salary'].transform('mean')
```

- 常用聚合函式：`sum`、`mean`、`median`、`std`、`count`、`nunique`（唯一值數量）、`first`、`last`、`min`、`max`
- 業務問題直接翻譯：「各城市的訂單數量和平均金額」→ `df.groupby('city').agg({'order_id': 'count', 'amount': 'mean'})`

視覺建議：
一張三步驟示意圖：左邊是一個混合的銷售資料表，中間被虛線切成顏色不同的子表（Split），右邊子表各自計 + Combine）。整個圖標記 split-apply-combine 三個步驟。

轉場：「單張表的操作學完了。現實中的資料幾乎都要從多張表合併而來。」

S5 — merge 與 concat：把多張表合成一張

核心主張：資料整合是現實工作中最常見的任務之一。merge 處理有關聯的表，concat 處理結構相同的表。

講師講解要點：

- concat：垂直或水平堆疊

```
# 垂直堆疊（上下合併）
df_q1 = pd.read_csv('sales_q1.csv')
df_q2 = pd.read_csv('sales_q2.csv')
df_all = pd.concat([df_q1, df_q2], ignore_index=True)

# 水平堆疊（左右合併）
df_combined = pd.concat([df_features, df_labels], axis=1)
```

- merge：根據欄位關聯兩張表

```
# orders 有 customer_id，customers 有對應的
orders = pd.DataFrame({'order_id': [1,2,3], 'customer_id': [101, 102, 101], 'amount': [500, 300, 
customers = pd.DataFrame({'customer_id': [101, 102, 103], 'name': ['Alice', 'Bob', 'Charlie'], 'c

# inner join：只保留兩邊都有的
result = pd.merge(orders, customers, on='customer_id', how='inner')

# left join：保留左邊全部，右邊沒對應的會 NaN
result = pd.merge(orders, customers, on='customer_id', how='left')

# 欄位名不同時
result = pd.merge(df1, df2, left_on='user_id', right_on='customer_id')
```

- 四種 join 類型的語意差異：

join 類型	保留哪些列	典型使用場景
`inner`	兩邊都有的	只分析有完整資料的記錄
`left`	左表所有列	保留主表，補充右表資訊
`right`	右表所有列	較少用，通常改用 left
`outer`	兩邊所有列	檢查哪些記錄只存在於一邊

- 常見錯誤：重複欄位名稱導致 merge 後出現 `_x` 和 `_y` 後綴，需要在 merge 前重命名欄位或用 `suffixes` 參數指定。

視覺建議：四個並排的 Venn 圖，分別標示 inner/left/right/outer join 保留的範圍（陰影部分），每個圖下方附上對應的程式碼片段。

轉場：「合併完資料，有時你需要從不同角度看它——這就是 pivot table 的用途。」

S6 — pivot_table：從長格式到交叉分析

核心主張：pivot table 把「長格式」的流水帳資料，轉換為二維的交叉統計表，讓業務問題一目了然。

講師講解要點：

- 長格式 vs 寬格式：原始資料通常是長格式（每列是一筆事件記錄），pivot table 把它轉成寬格式（行是一個維度，欄是另一個維度）。

```
# 長格式
sales = pd.DataFrame(
    {'region': ['North', 'North', 'South', 'South', 'East'],
    'product': ['A', 'B', 'A', 'B', 'A'],
    'quarter': ['Q1', 'Q2', 'Q1', 'Q1', 'Q2'],
    'revenue': [100, 200, 150, 300, 250]
})

# pivot table
pivot = sales.pivot_table(
    values='revenue',
    index='region',
    columns='product',
    aggfunc='sum',
    fill_value=0,        # 缺值補 0
)

# 結果：region 列，product 欄，revenue 值

# 多維度
pivot_multi = sales.pivot_table(
    values='revenue',
    index='region',
    columns='quarter',
    aggfunc=['sum', 'mean']
)

# 回到長格式
long = pivot.reset_index().melt(id_vars='region', var_name='product', value_name='revenue')
```

- `margins=True` 參數加入總計列和欄，等同於 Excel 的「加總」功能。

- pivot_table 與 groupby 的選擇：如果你想要一個二維的交叉表格，用 pivot_table；如果你只需要一維的聚合，用 groupby。

視覺建議： 一張從長格式到寬格式的轉換示意圖：左邊是流水帳格式的原始資料表，右邊是 pivot table 的交叉結果，用箭頭連接，標示 `index`、`columns`、`values` 三個參數對應到圖中的哪個維度。

轉場：「下一個特殊維度是時間。日期欄的處理方式和一般欄位完全不同。」

S7 — 時間序列基礎：讓日期欄真的變成日期

核心主張： 時間是資料中最特殊的維度。把日期欄從字串變成 datetime64，打開了排序、計算時間差、重採樣等一系列分析能力。

講師講解要點：

- 解析日期：CSV 讀進來的日期幾乎一定是 object（字串），第一步是轉換：

```
# [redacted]
df = pd.read_csv('orders.csv', parse_dates=['order_date', 'ship_date'])

# [redacted]
df['order_date'] = pd.to_datetime(df['order_date'])
df['order_date'] = pd.to_datetime(df['order_date'], format='%Y/%m/%d')  # [redacted]
df['order_date'] = pd.to_datetime(df['order_date'], errors='coerce')  # [redacted] NaT
```

- 時間屬性萃取：

```
df['year'] = df['order_date'].dt.year
df['month'] = df['order_date'].dt.month
df['day_of_week'] = df['order_date'].dt.dayofweek  # 0=Monday
df['quarter'] = df['order_date'].dt.quarter
df['is_weekend'] = df['order_date'].dt.dayofweek >= [redacted]
```

- 計算時間差：

```
# [redacted]
df['shipping_days'] = (df['ship_date'] - df['order_date']).dt.days
```

```
# 計算年資
today = pd.Timestamp.today()
df['tenure_years'] = (today - df['hire_date']).dt.days / 365
```

- 時間序列重採樣（Resample）：

```
# 設定 index
df = df.set_index('order_date')

# 月營收
monthly = df['revenue'].resample('ME').sum()

# 季平均
quarterly = df['revenue'].resample('QE').mean()

# 7 日移動平均
df['revenue_7d_avg'] = df['revenue'].rolling(window=7).mean()
```

- 常見陷阱：時區問題（timezone-naive vs timezone-aware）；`NaT`（Not a Time）是時間版的 NaN，`pd.isnull()` 可以檢測。

視覺建議：一條時間軸示意圖，標示 resample 的不同頻率（每日、每週、每月）對應到的聚合結果。旁邊附一個 dt accessor 的常用屬性速查表（year/month/day/dayofweek/quarter/hour 等）。

轉場：「時間處理之後，我們進入清理工作的核心：缺失值。」

S8 — 缺失值：識別、理解、處理

核心主張：
沒有清理過的缺失值是分析毒藥。不同來源的缺失，需要不同的處理策略。亂填、亂刪都是錯的。

講師講解要點：

- 缺失值的三種來源：
- 資料收集時就沒有（例如問卷某題空白、選填欄位）
- 資料合併時的欄位不對齊（LEFT JOIN 後的 NaN）

系統故障或資料管道問題（sensor 未回傳值）

統計學的缺失值分類（MCAR / MAR / MNAR）：

- MCAR（Missing Completely At Random）：缺失和任何變數都無關。例如問卷被風吹走。可以直接刪除，不會產生偏差。
- MAR（Missing At Random）：缺失和「其他已知變數」有關，但和缺失值本身無關。例如年輕人較不填收入欄位。可以用其
- MNAR（Missing Not At Random）：缺失和「缺失值本身」有關。例如高收入者刻意不填收入。最棘手，填補可能產生偏差。

初學者實用判斷：大部分情況你不需要精確分類，但要問自己：「這些缺失是隨機的，還是有系統性原因？

識別缺失值：

```
# [redacted]
df.isnull().sum()
# [redacted]
df.isnull().mean()
# [redacted]
df[df.isnull().any(axis=1)]
# [redacted] seaborn [redacted] missingno [redacted]
```

- 四種處理策略：

```
# 1. [redacted]
df_clean = df.dropna()                    # [redacted]
df_clean = df.dropna(subset=['salary'])   # [redacted]
df_clean = df.dropna(thresh=5)            # [redacted] 5 [redacted]
# 2. [redacted]
df['city'] = df['city'].fillna('Unknown')
df['is_online'] = df['is_online'].fillna(False)
# 3. [redacted]
df['age'] = df['age'].fillna(df['age'].median())
# 4. [redacted]
df['price'] = df['price'].ffill()  # [redacted]
df['price'] = df['price'].bfill()  # [redacted]
```

- 選擇策略的原則：
- 缺失比例 < 5%：刪除通常可接受
- 缺失比例 5-30%：根據來源選擇填補策略
- 缺失比例 > 30%：評估這個欄位是否值得保留
- 缺失不是隨機的（例如高收入者更少填寫年齡）：統計填補可能引入偏差

視覺建議：一張資料框示意圖，標示有缺失的欄位（用灰色格子表示 NaN）。旁邊附一個決策樹：缺失值 → 缺失比例高低？→ 缺失是否隨機？→ 導向不同的處理策略。

轉場：「缺失值之後，我們看另一類問題：數值上的極端值。」

S9 — 離群值：識別、判斷、決策

核心主張：
不是所有極端值都是錯誤，但你必須知道它們在哪裡。離群值的處理需要結合業務理解，不能盲目刪除。

講師講解要點：

- IQR 方法（四分位距法）：

```
Q1 = df['salary'].quantile(0.25)
Q3 = df['salary'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# [illegible]
df['is_salary_outlier'] = (df['salary'] < lower_bound) | (df['salary'] > upper_bound)
outliers = df[df['is_salary_outlier']]
print(f"[illegible] {len(outliers)} [illegible]")
```

- Z-score 方法：

```
from scipy import stats  # [illegible] NumPy [illegible]

```

```
# scores = np.abs((df['salary'] - df['salary'].mean()) / df['salary'].std())
outliers = df[z_scores > 3]  # ■■■ 3 ■■■■
```

- 視覺化識別（配合 M5 視覺化）：箱形圖（boxplot）是最直觀的離群值檢視工具。
- 處理策略：

```
# 1. ■■■■■■■■■■■■
df = df[~df['is_salary_outlier']]

# 2. ■■■Winsorizing■■■■■■■■■■■■■■■■
df['salary_clipped'] = df['salary'].clip(lower=lower_bound, upper=upper_bound)

# 3. ■■■■■■■■■■■■■■■■
```

```
# [redacted]
# True/False, 1/0, 'yes'/'no', 'Y'/'N' [redacted]
df['is_manager'] = df['is_manager'].map({True: True, False: False, 1: True, 0: False, 'Y': True,
]
# [redacted] Categorica[redacted]
df['department'] = df['department'].astype('category')
df['city'] = df['city'].astype('category')
]
# [redacted] NaN [redacted] float64[redacted]pandas 2.0 [redacted]
# pandas 1.0+ [redacted] nullable integer type
df['age'] = df['age'].astype('Int64')  # [redacted] I[redacted] NaN
```

- 欄位名稱清理（現實中的 CSV 欄名往往有空格和特殊字元）：

```
# [redacted]
df.columns = df.columns.str.strip()          # [redacted]
df.columns = df.columns.str.lower()          # [redacted]
df.columns = df.columns.str.replace(' ', '_') # [redacted]
```

- 重複列處理：

```
# [redacted]
print(df.duplicated().sum())
|
# [redacted]
df = df.drop_duplicates()
|
# [redacted]
df = df.drop_duplicates(subset=['order_id'], keep='last')
```

- 清理完成的驗收標準：每欄的 dtype 符合預期；`isnull().sum()` 的結果是已知且有意識的；沒有任何重複的 primary key；所有數值欄的範圍在合理區間內。

視覺建議：一張清理前後對照表：左邊是一個「髒」的 DataFrame（有 NaN、型別錯誤、欄名有空格、疑似離群的數字），右邊是清理後的版本，每個修改位置用顏色標注與說明。

轉場：「資料清理完，我們來看 pandas 這個工具本身的重要演進。」

S11 — 里程碑：pandas 2.0 — Copy-on-Write 與 Arrow backend

核心主張：pandas 2.0 是架構層級的升級，Copy-on-Write 解決了長達十年的語意不一致問題，Arrow backend 帶來大幅度的記憶體效率提升。

講師講解要點：

- Copy-on-Write（CoW）：pandas 1.x 最臭名昭著的問題之一是「鏈式賦值」的不確定行為：

```
# pandas 1.x 的問題
df[df['age'] > 30]['salary'] = 0
# 這會修改 df 嗎？
# 答案：pandas 版本決定
# 會產生 undefined behavior，通常是 SettingWithCopyWarning

# pandas 2.0 Copy-on-Write 的解法
# 明確的寫法
df.loc[df['age'] > 30, 'salary'] = 0

# 或者用 where 保留原資料
df['salary_modified'] = df['salary'].where(df['age'] <= 30, 0)
```

- Arrow backend：pandas 2.0 允許使用 Apache Arrow 作為底層儲存格式（之前預設是 NumPy）。Arrow 格式帶來：
- 支援 nullable integer（原生 NaN，而非用 float 來表示 int 的 NaN）
- 字串效率大幅提升
- 記憶體使用量在典型資料集上可減少 30-50%

```
# 啟用 Arrow backend
df = pd.read_csv('data.csv', dtype_backend='pyarrow')

# 檢查資料型態
print(df.dtypes)  # 顯示 ArrowDtype
```

- 對學習者的實際意義：
- 遇到 `SettingWithCopyWarning` 不要忽略，這是 pandas 在告訴你賦值可能無效
- 在 pandas 2.0 環境下，使用 `.loc` 做有條件的賦值是最安全的寫法
- 如果處理大型資料集，試試 `dtype_backend='pyarrow'` 看是否能減少記憶體用量

視覺建議：兩個並排的程式碼對比框。左框「pandas 1.x 的語意地雷」，展示鏈式賦值，下方顯示警告訊息。右框「pandas 2.0 正確寫法」，展示 `.loc` 賦值，下方顯示「語意清晰，結果可預期」。底部附 Arrow backend 的記憶體節省柱狀比較示意。

轉場：「pandas 很強，但有邊界。最後一張投影片，我們談工具規模感。」

S12 — 工具規模感：小資料到大資料的工具演進

核心主張：pandas 是小到中型資料的最佳選擇，但知道它的邊界以及邊界之後有什麼工具，是工程師成熟度的標誌。

講師講解要點：

- pandas 的適用範圍：記憶體內（in-memory）操作，資料集在幾百 MB 到幾 GB 之間（取決於機器記憶體）。速度對大多數分析任務足夠，生態最成熟，教學資源最豐富。在 Python 資料分析的語境下，pandas 是預設選擇，也是你要最熟練的工具。
- Polars：用 Rust 撰寫的 DataFrame 庫，API 設計上借鑒 pandas 但更嚴格（沒有 index、強制 lazy evaluation 選項）。在單機上比 pandas 快 5-20 倍，Arrow 原生支援，適合需要高效能但資料仍在單機的場景。2023-2024 年快速崛起。
- DuckDB：內嵌式 SQL 分析引擎，可以直接查詢 CSV、Parquet、pandas DataFrame，不需要資料庫伺服器。適合數 GB 到數十 GB 的資料，SQL 介面對有資料庫背景的人非常友善。
- Apache Spark（PySpark）：分散式計算框架，適合 TB 級以上的資料，需要集群環境。PySpark DataFrame API 設計上與 pandas 相似，降低了遷移學習成本。
- 選擇框架的起點直覺：

資料量	推薦工具	理由
< 1 GB	pandas	生態最成熟，語法最易學
1–50 GB（單機）	Polars 或 DuckDB	速度與記憶體效率顯著提升

> 50 GB 或多機	Spark (PySpark)	分散式計算是唯一選項

視覺建議：一張工具規模尺，水平軸是資料量（從 MB 到 PB），上方標示不同工具的適用範圍（用彩色橫條表示）。每個工具下方加 1-2 個關鍵特徵描述。最底部加一行：「本課程專注 pandas，但你應該知道邊界在哪裡。」

六、練習設計

練習 1：DataFrame 操作基礎（S1-S4 結束後，25 分鐘）

情境說明：

以下是一份模擬的電商訂單資料，包含欄位：`order_id`、`customer_id`、`region`、`product_category`、`revenue`、`o`

任務清單：

1. 用下方字典建立 DataFrame，並執行初次偵察五步驟（`head`、`info`、`describe`、`shape`、`dtypes`）

```
import pandas as pd
data = {
    'order_id': range(1, 21),
    'customer_id': [f'C{i:03d}' for i in [101,102,103,101,104,102,105,103,101,102,
                                          104,105,101,103,102,104,105,101,102,103]],
    'region': ['North','South','East','North','West','South','East','North','West','South',
               'North','East','South','West','North','East','South','West','North','South'],
    'product_category': ['Electronics','Clothing','Food','Electronics','Books',
                         'Clothing','Food','Electronics','Books','Clothing',
                         'Electronics','Food','Clothing','Books','Electronics',
                         'Food','Clothing','Books','Electronics','Clothing'],
    'revenue': [1200,350,80,900,45,420,60,1500,30,380,
                1100,70,290,25,800,90,340,40,1300,410],
    'order_date': pd.date_range('2024-01-01', periods=20, freq='3D'),
}
df = pd.DataFrame(data)
```

1. 篩選出 `region == 'North'` 且 `revenue > 500` 的訂單（使用 boolean indexing，用 `&`）
2. 篩選出 `product_category` 為 'Electronics' 或 'Clothing' 的訂單（使用 `.isin()`）

3. 計算每個 `product_category` 的訂單數量、總營收、平均訂單金額，用 `groupby + agg` 一次完成
4. 按總營收降序排列，找出營收最高的前三個產品類別
5. 建立一個新欄位 `revenue_tier`：使用 `pd.cut` 或 `np.select` 把 revenue 分為 'Low'（< 200）、'Medium'（200-800）、'High'（> 800）
6. 計算每個 `region` 的平均訂單金額，並找出最高和最低的地區

驗收標準： - 能正確使用 boolean indexing 的多條件組合（`&` 而非 `and`） - groupby 的 agg 結果包含三個指標（count、sum、mean） - 新欄位的型別合理（Categorical 或 object）

練習 2：資料合併與 pivot table（S5-S6 結束後，25 分鐘）

情境說明：
你有兩張表：訂單表和顧客資料表，需要合併後進行交叉分析。

任務清單：

```
orders = pd.DataFrame(
    'order_id': range(1, 11),
    'customer_id': [101,102,103,101,104,102,105,103,101,106],
    'product': ['A','B','A','C','B','A','C','B','A','B'],
    'amount': [500,300,700,450,250,600,800,350,550,400],
    'month': ['Jan','Jan','Feb','Feb','Mar','Mar','Jan','Feb','Mar','Jan'],
)

customers = pd.DataFrame(
    'customer_id': [101,102,103,104,105],
    'city': ['Taipei','Taipei','Kaohsiung','Taichung','Taipei'],
    'membership': ['Gold','Silver','Bronze','Gold','Silver'],
)
```

1. 用 `inner join` 合併兩張表（`on='customer_id'`），確認結果的 shape
2. 用 `left join` 合併，找出哪些 order 的 customer 不在 customers 表中（觀察 NaN 出現的位置）
3. 用 `pivot_table` 計算每個城市（index）、每個產品（columns）的總銷售金額（values），`fill_value=0`
4. 在 pivot_table 中加入 `margins=True`，觀察總計列和欄

5. 建立另一個 pivot_table：每個月份（index）、每個會員等級（columns）的平均訂單金額
6. 將第 3 步的 pivot table 用 `reset_index().melt(...)` 轉換回長格式，確認欄位名稱正確

驗收標準： - 理解 inner 和 left join 結果列數不同的原因 - pivot_table 的三個核心參數（index/columns/values）使用正確 - melt 轉換回長格式後的結果可以重新建立原始 pivot

---

練習 3：Mini 資料清理工作坊（S7-S10 結束後，45 分鐘）

情境說明：
你拿到一份「髒」的員工資料，目標是完成一份符合分析品質的清理作業，並回答三個業務問題。這是本模組的

資料說明：
欄位包含：`employee_id`、`department`、`hire_date`（有些被讀成字串）、`salary`（有些包含「NT$」前綴與逗號）分，但有幾個值是 99）、`city`、`is_manager`（混用 True/False 和 1/0）、`last_review_date`（有 15% 缺失）

```
import pandas as pd
import numpy as np

np.random.seed(42)
n = 40

data = {
    'employee_id': range(1001, 1001 + n),
    'department': np.random.choice(['Engineering','Marketing','Sales','HR','Finance'], n),
    'hire_date': pd.date_range('2015-01-01', periods=n, freq='45D').astype(str),
    'salary': [f'NT$({s:,})' for s in np.random.randint(40000, 120000, 30)] +
              [str(s) for s in np.random.randint(40000, 120000, 10)],
    'performance_score': [list(np.random.randint(1, 6, 35)) + [99, 99, 99, 99, 99]],
    'city': np.random.choice(['Taipei', 'Kaohsiung', 'Taichung', None], n, p=[0.5, 0.2, 0.2, 0.1]),
    'is_manager': [True, False, 1, 0, True] * 8,
    'last_review_date': list(pd.date_range('2023-01-01', periods=34, freq='10D').astype(str)) + [No
}
df_raw = pd.DataFrame(data)
```

任務清單（依序完成）：

Step 1：資料偵察 - 執行初次偵察五步驟，記錄：有幾個欄位有缺失、哪些欄位型別看起來不對

Step 2：型別修正 - 把 `hire_date` 轉為 datetime64 型別（使用 `pd.to_datetime`） - 把 `salary` 的 'NT$' 前綴和逗號去除，轉為 float（使用 `.str.replace` 和 `.astype`） - 把 `is_manager` 統一為 bool

型別（使用 `.map` 或 `.astype`）

Step 3：缺失值處理 - 確認 `city` 的缺失情況，用 'Unknown' 填補 - 確認 `last_review_date` 的缺失情況（應為約 15%），決定填補策略並說明理由

Step 4：離群值與錯誤值識別 - 用 IQR 方法識別 `salary` 的離群值，列出離群值的員工 ID - 把 `performance_score == 99` 識別為資料錯誤，替換為 `NaN`，再用中位數填補

Step 5：回答業務問題 1. 各部門的平均薪資是多少？用表格呈現，並指出最高與最低薪的部門 2. 管理職（`is_manager == True`）和非管理職的績效分布有何差異（用 `groupby + describe` 比較） 3. 計算每位員工從入職到今日（`pd.Timestamp.today()`）的年資，找出哪個部門的平均年資最長

驗收標準： - 清理後的 DataFrame：每欄的 dtype 正確、無非預期的缺失值、performance_score 無 99 - 缺失值處理有明確的策略說明（為什麼這樣處理） - 三個業務問題的答案都有對應的程式碼支撐 - 整個流程可以從頭到尾不報錯地執行

七、模組里程碑卡

```
[illegible] — pandas 2.0[illegible]2023 [illegible] 4 [illegible]

[illegible] pandas [illegible]
[illegible] df[mask][ 'col' ] = value [illegible]
Copy-on-Write [illegible]
[illegible]

[illegible]
[illegible] Copy-on-Write[illegible]
[illegible] Arrow backend[illegible] 30-50%
[illegible] nullable integer[illegible] float [illegible] int + NaN[illegible]
[illegible]

[illegible]
[illegible] SettingWithCopyWarning [illegible]
[illegible]
[illegible] pd.options.mode.copy_on_write = True [illegible]
```

八、本模組結語

pandas 是資料分析工作的核心工作台，也是你在資料科學旅程中用得最頻繁的工具之一。

你現在掌握的資料清理能力——識別型別錯誤、處理缺失值、找出離群值——是讓任何分析結果「可以被相信」的

接下來的 M5 視覺化與 EDA，會教你如何把清理後的資料變成圖表，讓模式和異常從數字堆裡浮現出來。

參考文獻與引用來源

pandas 官方資源

1. pandas 2.0 Release Notes (April 3, 2023) — Arrow backend、Copy-on-Write 語意變更。pandas.pydata.org/docs/whatsnew/v2.0.0.html
2. pandas Official Documentation — DataFrame API 完整說明。pandas.pydata.org/docs
3. Arrow backend 記憶體改善 — PyArrow 字串欄位相比 NumPy object 欄位可減少最多 70% 記憶體；整體使用 Arrow dtype_backend 約減少 50% 記憶體。來源：pandas 官方文件與社群基準測試。參見 datapythonista.me/blog/pandas-20-and-the-arrow-revolution-part-i

缺失值理論

1. MCAR / MAR / MNAR 分類 — 原始定義來自 Rubin, D.B. (1976). "Inference and Missing Data." Biometrika, 63(3), 581-592. 此三分類是缺失值處理的統計學標準框架。
2. Little, R.J.A. & Rubin, D.B. (2019). Statistical Analysis with Missing Data, 3rd Edition. Wiley. — 缺失值分析的權威教科書。

缺失值處理決策門檻

1. 缺失比例 < 5% / 5-30% / > 30% 門檻 — 此為資料科學實務中常用的經驗法則，非絕對標準。參考來源：Schafer, J.L. (1999). "Multiple imputation: a primer." Statistical Methods in Medical Research, 8(1), 3-15. 以及 Bennett, D.A. (2001). "How can I deal with missing data in my study?" Australian and New Zealand Journal of Public Health, 25(5), 464-469.

工具生態

1. Polars Documentation — 高效能 DataFrame 替代方案。docs.pola.rs
2. DuckDB Documentation — 嵌入式分析資料庫。duckdb.org/docs
3. Apache Arrow — 跨語言欄式記憶體格式。arrow.apache.org

延伸閱讀

1. McKinney, W. (2022). Python for Data Analysis, 3rd Edition. O'Reilly Media. — pandas 作者親撰的權威指南。
2. Wickham, H. (2014). "Tidy Data." Journal of Statistical Software, 59(10). — 整潔資料原則。doi.org/10.18637/jss.v059.i10

Module 5：視覺化與 EDA

課程定位

所屬課程：2026 Python 數據分析與 AI 基礎：從資料思維到機器學習入門（24 小時）
模組編號：Module 5／共 8 個模組 時長：3 小時 前置模組：M4 pandas 與資料清理 後續模組：M6 統計思維與假設檢定

模組核心主張：圖表是思考工具，不是裝飾品。EDA 的終點不是一張漂亮圖，而是一個能被驗證的業務假設。

> "The greatest value of a picture is when it forces us to notice what we never expected to see." — John Tukey

---

學習目標

完成本模組後，學員能夠：

1. 說明 Matplotlib 與 Seaborn 的定位差異，並在正確場景選用適當套件
2. 使用 Matplotlib 基本語法建立常見圖形，包含自訂標題、軸標籤、色彩與圖例
3. 使用 Seaborn 高階 API 快速產出統計圖形，包含 histogram、boxplot、scatterplot、heatmap
4. 說明四種圖表任務類型（分佈、比較、關聯、時序），並為給定問題選擇正確圖形
5. 依照 EDA 標準路徑（單變數 → 雙變數 → 多變數）系統性探索一份陌生資料集
6. 從 EDA 結果中提取洞察，並以「觀察 → 洞察 → 建議」結構輸出業務故事
7. 識別資料中的異常值與分佈特徵，判斷其是資料品質問題還是業務訊號
8. 獨立完成一份包含三個洞察的 EDA 報告

關鍵概念清單

視覺化工具

- [ ] Matplotlib 基礎架構：Figure、Axes、Artist 三層模型
- [ ] plt.subplots() 建立單圖與多子圖
- [ ] 常用圖形 API：plt.plot()、plt.bar()、plt.hist()、plt.scatter()、plt.boxplot()
- [ ] 圖形客製化：title、xlabel、ylabel、legend、color、figsize
- [ ] Seaborn 與 Matplotlib 的關係：Seaborn 建立在 Matplotlib 之上
- [ ] Seaborn 主要圖形：sns.histplot()、sns.boxplot()、sns.scatterplot()、sns.heatmap()、sns.pairplot()
- [ ] 探索圖 vs 報告圖的用途差異

EDA 方法論

- [ ] 圖表任務四分類：分佈圖、比較圖、關聯圖、時序圖
- [ ] EDA 三層路徑：單變數分析、雙變數分析、多變數分析
- [ ] 好的資料問題三要素：可觀察、可比較、可驗證
- [ ] 異常值識別：IQR 法則、3σ 法則、視覺化判讀
- [ ] 相關性矩陣：Pearson 相關係數的直覺意義與局限
- [ ] 資料故事結構：觀察（Observation）→ 洞察（Insight）→ 建議（Recommendation）

投影片大綱

#	投影片標題	核心訊息
1	圖表不是裝飾，是思考的外掛	圖表的價值在揭露，不在呈現

2	Matplotlib：視覺化的基礎層	先懂基礎，才能駕馭高階工具
3	Seaborn：統計視覺化的快車道	一行程式碼做出正確的統計圖
4	四種圖表任務類型	選圖從問題出發，不從套件出發
5	什麼是好的資料問題？	問題的品質決定圖的品質
6	EDA 路徑：單變數分析	先看每個變數的形狀，再問關係
7	EDA 路徑：雙變數與多變數分析	關係藏在變數之間，不在變數之內
8	異常值：資料品質問題還是業務訊號	異常值不等於錯誤，先判斷再處理
9	真實案例：電商用戶行為 EDA 全流程	看一次完整的 EDA 怎麼進行
10	如何用一頁圖表講業務故事	從觀察到洞察到建議的轉化
11	【練習 A】完整 EDA 探索	實作：走完完整的 EDA 路徑
12	【練習 B】三段洞察簡報	實作：一份資料集，三個洞察

詳細投影片內容

Slide 1：圖表不是裝飾，是思考的外掛

核心訊息：圖表不是用來讓報告看起來專業，而是用來幫你發現資料裡肉眼看不到的結構。

講師講解要點：

- 大多數人學視覺化的動機是「讓報告好看」，但這是本末倒置——好圖的標準是「它讓你發現了什麼」
- 試著用純數字表格回答「這兩組的差異大嗎？」，再換成圖，感受資訊密度的差異
- Anscombe's Quartet：四組統計數字完全相同的資料集（均值、變異數、相關係數全部一樣），但一畫出來結構完全不同
- EDA 中的圖是給自己看的草稿，不是給老闆看的成品——探索圖和報告圖是兩件事，要分開對待
- 重要觀念：探索階段的圖目的是「問問題」，報告階段的圖目的是「傳達答案」

程式碼範例：

```
import matplotlib.pyplot as plt
import numpy as np

# Anscombe's Quartet - [redacted]
datasets = {
    'I':   {'x': [10,8,13,9,11,14,6,4,12,7,5],  'y': [8.04,6.95,7.58,8.81,8.33,9.96,7.24,4.26,10.84,
    'II':  {'x': [10,8,13,9,11,14,6,4,12,7,5],  'y': [9.14,8.14,8.74,8.77,9.26,8.10,6.13,3.10,9.13,
    'III': {'x': [10,8,13,9,11,14,6,4,12,7,5],  'y': [7.46,6.77,12.74,7.11,7.81,8.84,6.08,5.39,8.15,
    'IV':  {'x': [8,8,8,8,8,8,8,19,8,8,8],      'y': [6.58,5.76,7.71,8.84,8.47,7.04,5.25,12.50,5.56,
}

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, (name, data) in zip(axes, datasets.items()):
    ax.scatter(data['x'], data['y'], color='steelblue', alpha=0.8, s=60)
    ax.set_title(f'Dataset {name}', fontsize=13)
    ax.set_xlabel('x'); ax.set_ylabel('y')
    ax.set_xlim(0, 20); ax.set_ylim(0, 14)

fig.suptitle('Anscombe's Quartet[redacted]', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```

視覺建議：左右對比圖——左側是一張密密麻麻的數字表格，右側是 Anscombe's Quartet 的四個散佈圖。底部標注：「這四組資料的均值、變異數、相關係數完全相同——你還覺得數字夠用嗎？」

銜接下一張：「既然視覺化這麼重要，我們先從最基礎的工具 Matplotlib 開始，建立正確的操作模型。」

Slide 2：Matplotlib：視覺化的基礎層

核心訊息：Matplotlib 是 Python 視覺化的基礎。理解它的三層架構，你才能精確控制每一個視覺細節。

講師講解要點：

- 三層架構：
- Figure：整張畫布（canvas），可以包含一個或多個子圖
- Axes：一個獨立的座標系（subplot），是實際畫圖的地方
- Artist：所有可見元素（線條、點、文字、刻度）都是 Artist 物件
- 兩種使用風格：
- `plt.xxx()` 直接呼叫（快速但不靈活，適合單圖快速探索）
- `fig, ax = plt.subplots()` 物件導向（明確、可擴展，推薦用於正式圖表）
- 常見圖形 API

  速查：`ax.plot()`（折線）、`ax.bar()`（長條）、`ax.hist()`（直方圖）、`ax.scatter()`（散佈）、`ax.boxplo`
- 圖形客製化核心參數：`title`、`xlabel/ylabel`、`legend`、`color`、`figsize`、`alpha`

程式碼範例：

```
import matplotlib.pyplot as plt
import numpy as np

# ---- [redacted] pyplot [redacted] ----
x = np.linspace(0, 10, 100)
plt.figure(figsize=(8, 4))
plt.plot(x, np.sin(x), label='sin(x)', color='steelblue', linewidth=2)
plt.plot(x, np.cos(x), label='cos(x)', color='coral', linewidth=2, linestyle='--')
plt.title('[redacted]', fontsize=14)
plt.xlabel('x [redacted]'); plt.ylabel('y [redacted]')
plt.legend(); plt.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()

# ---- [redacted] ----
fig, axes = plt.subplots(1, 3, figsize=(14, 4))

# [redacted]
categories = ['A [redacted]', 'B [redacted]', 'C [redacted]', 'D [redacted]']
values     = [23, 45, 31, 52]
axes[0].bar(categories, values, color=['#4C72B0','#DD8452','#55A868','#C44E52'], alpha=0.85)
axes[0].set_title('[redacted]'); axes[0].set_ylabel('[redacted]')

# [redacted]
data = np.random.normal(loc=170, scale=8, size=500)
axes[1].hist(data, bins=25, color='steelblue', alpha=0.75, edgecolor='white')
```

```
axes[1].set_title('[redacted]'); axes[1].set_xlabel('[redacted] (cm)')
[redacted]
# [redacted]
x_s = np.random.normal(0, 1, 200)
y_s = 2 * x_s + np.random.normal(0, 0.5, 200)
axes[2].scatter(x_s, y_s, alpha=0.5, color='coral', s=30)
axes[2].set_title('[redacted]'); axes[2].set_xlabel('X'); axes[2].set_ylabel('Y')
[redacted]
plt.suptitle('Matplotlib [redacted]', fontsize=14, y=1.02)
plt.tight_layout(); plt.show()
```

視覺建議：上方顯示 Figure / Axes / Artist 三層架構示意圖（巢狀方框），下方顯示程式碼範例產生的三種圖形。每個圖形旁邊標注適用情境關鍵字。

銜接下一張：「Matplotlib 靈活但需要寫不少程式碼。當我們想快速產出統計圖時，Seaborn 大幅降低了門檻。」

Slide 3：Seaborn：統計視覺化的快車道

核心訊息：Seaborn 建立在 Matplotlib 之上，一行程式碼就能畫出正確的統計圖形，並自動處理好配色與格式。

講師講解要點：

- Seaborn vs Matplotlib 的定位差異：
- Matplotlib：底層控制，適合客製化複雜圖形
- Seaborn：高階封裝，適合快速統計探索
- 兩者並不互斥——Seaborn 畫完之後，可以用 Matplotlib API 進一步微調
- Seaborn 常用圖形速查：
- `sns.histplot(data, x='col')` — 帶 KDE 的直方圖
- `sns.boxplot(data, x='group', y='value')` — 分組箱形圖
- `sns.scatterplot(data, x='col1', y='col2', hue='category')` — 帶類別色彩的散佈圖
- `sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')` — 相關性熱力圖
- `sns.pairplot(data, hue='category')` — 多變數配對圖（EDA 神器）
- 主題設定：`sns.set_theme(style='whitegrid')` 一行讓所有圖形更好看

程式碼範例：

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

sns.set_theme(style='whitegrid', palette='muted')

# [redacted] seaborn [redacted]
tips = sns.load_dataset('tips')

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# 1. [redacted] + KDE[redacted]
sns.histplot(data=tips, x='total_bill', kde=True, ax=axes[0,0], color='steelblue')
axes[0,0].set_title('[redacted] KDE')

# 2. [redacted]
sns.boxplot(data=tips, x='day', y='total_bill', hue='sex', ax=axes[0,1])
axes[0,1].set_title('[redacted]')

# 3. [redacted] vs [redacted]
sns.scatterplot(data=tips, x='total_bill', y='tip', hue='smoker',
                style='time', alpha=0.8, ax=axes[1,0])
axes[1,0].set_title('[redacted] vs [redacted]')

# 4. [redacted]
corr = tips[['total_bill', 'tip', 'size']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', vmin=-1, vmax=1,
            fmt='.2f', ax=axes[1,1])
axes[1,1].set_title('[redacted]')

plt.suptitle('Seaborn [redacted]', fontsize=15, y=1.01)
plt.tight_layout(); plt.show()
```

視覺建議：2x2 子圖格，每個圖形下方標注對應的 Seaborn API 呼叫（一行程式碼）。旁邊加一個對比表格：同一張箱形圖，左側 Matplotlib 版本需要 12 行，右側 Seaborn 版本只需要 1 行。

銜接下一張：
「工具就位了。接下來的關鍵問題是：面對一份資料，我應該畫什麼圖？答案是：先問問題，問題決定圖形。」

Slide 4：四種圖表任務類型

核心訊息：選圖的起點是問題，不是套件。先問「我想知道什麼」，再選工具。

講師講解要點：

- 分佈圖（Distribution）：「這筆數值的形狀是什麼？集中還是分散？有沒有雙峰？」
- 工具：histogram、KDE plot、boxplot、violin plot

範例問題：「用戶停留時間的分佈是正態的嗎？有沒有重尾？」

比較圖（Comparison）：「A 和 B 哪個比較大？不同群組之間有差異嗎？」

- 工具：bar chart、grouped boxplot、dot plot

範例問題：「週末的平均訂單金額比平日高嗎？」

關聯圖（Relationship）：「X 變動時，Y 怎麼變？有沒有相關性或群集？」

- 工具：scatter plot、heatmap、pairplot、bubble chart

範例問題：「廣告投放金額和轉換率之間有什麼關係？」

時序圖（Time Series）：「這個指標隨時間如何演變？有沒有趨勢或季節性？」

- 工具：line chart、area chart、candlestick

範例問題：「過去 12 個月的月活躍用戶數趨勢如何？」

常見錯誤示範：想問「A 和 B 哪個比較大」卻畫了 pie chart；想看分佈卻只算均值

程式碼範例：

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

sns.set_theme(style='whitegrid')
np.random.seed(42)
n = 300

df = pd.DataFrame(
    'purchase_amount': np.concatenate(
        np.random.normal(200, 50, 200),   # 一般客
        np.random.normal(800, 100, 100)   # 大客戶
    ),
    'channel': np.random.choice(['organic', 'paid', 'social'], n),
    'ad_spend': np.random.uniform(100, 5000, n),
    'conversion_rate': None
```

```
    'date': pd.date_range('2024-01-01', periods=n, freq='D')[:n],
})
df['conversion_rate'] = 0.05 + df['ad_spend'] / 100000 + np.random.normal(0, 0.01, n)

fig, axes = plt.subplots(2, 2, figsize=(13, 10))

# [redacted]
sns.histplot(df['purchase_amount'], kde=True, bins=30, ax=axes[0,0], color='steelblue')
axes[0,0].set_title('[redacted]')
axes[0,0].set_xlabel('[redacted]')

# [redacted]
sns.boxplot(data=df, x='channel', y='purchase_amount', ax=axes[0,1], palette='pastel')
axes[0,1].set_title('[redacted]')

# [redacted]
sns.scatterplot(data=df, x='ad_spend', y='conversion_rate',
                hue='channel', alpha=0.6, ax=axes[1,0])
axes[1,0].set_title('[redacted] vs [redacted]')

# [redacted]
monthly = df.set_index('date')['purchase_amount'].resample('W').mean()
axes[1,1].plot(monthly.index, monthly.values, color='coral', linewidth=2)
axes[1,1].set_title('[redacted]')
axes[1,1].set_xlabel('[redacted]'); axes[1,1].tick_params(axis='x', rotation=30)

plt.suptitle('[redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
```

視覺建議：2x2 卡片格，每格一個任務類型，包含：任務名稱、代表問題範例、縮圖示意、適合的圖形工具列表。使用一致的色

銜接下一張：「知道有哪些圖，但還不夠——更根本的問題是：什麼樣的資料問題值得畫圖？」

Slide 5：什麼是好的資料問題？

核心訊息：圖來自問題，不是來自資料集。沒有問題，圖只是雜訊。

講師講解要點：

- 壞問題範例：「我想看看這份資料長什麼樣。」→ 沒有焦點，會畫出 20 張沒有結論的圖
- 好問題的三個條件：
- 可觀察：能在資料裡找到對應的欄位
- 可比較：有比較基準（群組、時間、目標值）

- 可驗證：結果能幫助做決定或推進分析
- 好問題範例：「新用戶的首週購買金額，是否比老用戶低？」
- 可觀察：有購買金額欄位✓
- 可比較：新用戶 vs 老用戶✓
- 可驗證：結果影響行銷策略✓
- 問題從哪裡來？業務痛點、異常警示、目標對齊——不是「看到什麼欄位就問什麼問題」
- 練習思維轉換：給定一個業務情境，先寫下問題，再決定畫哪種圖

壞問題 vs 好問題對照表：

壞問題	問題出在哪裡	改良後的好問題
「分析一下銷售資料」	沒有方向，無法畫圖	「哪個商品類別的退貨率最高？」
「看看用戶行為」	沒有比較基準	「新用戶和回訪用戶的瀏覽深度有差異嗎？」
「了解廣告效果」	沒有具體指標	「投放 FB 廣告的 ROI 是否優於 Google 廣告？」

視覺建議：
三欄對比表：左欄「壞問題」（模糊、無方向），中欄「好問題三要素」（可觀察、可比較、可驗證），右欄「
→ 問題 → 圖表任務類型 → 選圖工具。

銜接下一張：「有了問題，怎麼系統性地探索一份新資料？從單一變數開始。」

Slide 6：EDA 路徑：單變數分析

核心訊息：EDA
的第一步是看每個變數的形狀——不是關係，不是趨勢，是每個欄位本身的分佈與特性。

講師講解要點：

- 為什麼從單變數開始？

- 在研究兩個變數的關係之前，先確認每個變數「正不正常」
- 單變數分析能發現：異常值、偏態、資料品質問題、意外的群集
- 數值型變數的單變數分析：
- `df.describe()` 快速摘要（但記住：均值和標準差不能代表全部）
- Histogram 看形狀：對稱？偏態？雙峰？
- Boxplot 找異常：四分位距（IQR）法則識別可疑值
- 計算偏度（skewness）：`df['col'].skew()` > 1 代表右偏
- 類別型變數的單變數分析：
- `df['col'].value_counts()` 頻率分佈
- Bar chart 視覺化類別分佈
- 注意：是否有「其他」類別佔大多數？類別數量是否合理？

程式碼範例：

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(style='whitegrid')
np.random.seed(42)

# ========
n = 1000
df = pd.DataFrame(
    'order_amount':   np.concatenate([np.random.lognormal(5, 0.8, 950),
                                      np.random.uniform(5000, 20000, 50)]),  # ====
    'category':       np.random.choice(['==', '==', '==', '==', '=='],
                                       n, p=[0.25, 0.30, 0.20, 0.15, 0.10]),
    'is_new_user':    np.random.choice([True, False], n, p=[0.35, 0.65]),
    'days_since_join': np.random.exponential(180, n).astype(int)
)

fig, axes = plt.subplots(2, 3, figsize=(15, 9))

# ========
sns.histplot(df['order_amount'], kde=True, bins=50, ax=axes[0,0], color='steelblue')
axes[0,0].set_title('==========');  axes[0,0].set_xlabel('==')

# ==========
sns.boxplot(y=df['order_amount'], ax=axes[0,1], color='lightcoral')
axes[0,1].set_title('==========
```

```
[redacted]
[redacted]
sns.histplot(np.log1p(df['order_amount']), kde=True, bins=40, ax=axes[0,2], color='mediumseagreen
axes[0,2].set_title('log([redacted]) [redacted]')
[redacted]
[redacted]
cat_counts = df['category'].value_counts()
axes[1,0].barh(cat_counts.index, cat_counts.values, color=sns.color_palette('pastel'))
axes[1,0].set_title('[redacted]'); axes[1,0].set_ylabel('[redacted]')
[redacted]
[redacted]
sns.histplot(df['days_since_join'], bins=40, kde=True, ax=axes[1,1], color='plum')
axes[1,1].set_title('[redacted]')
[redacted]
[redacted]
print("=== [redacted] ===")
print(df['order_amount'].describe().round(2))
print(f"\n[redacted] (Skewness): {df['order_amount'].skew():.2f}")
print(f"[redacted] (Kurtosis): {df['order_amount'].kurt():.2f}")
[redacted]
axes[1,2].axis('off')
plt.suptitle('[redacted] + [redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
```

視覺建議： 2x3 子圖格，同一個數值欄位用三種不同圖形展示（histogram、boxplot、log 轉換後），讓學員看到「同一份資料，不同圖形揭露不同問題」。

銜接下一張：「看完每個變數後，接下來最關鍵的問題出現了：變數之間有什麼關係？」

Slide 7：EDA 路徑：雙變數與多變數分析

核心訊息：洞察藏在變數的關係裡。雙變數分析看兩個欄位的互動；多變數分析在複雜關係中找結構。

講師講解要點：

- 雙變數分析類型：
- 數值 vs 數值：Scatter plot + 相關係數
- 數值 vs 類別：Grouped boxplot / violin plot
- 類別 vs 類別：Crosstab + heatmap / grouped bar chart
- 相關係數的直覺：
- Pearson r 的範圍：-1（完全負相關）到 +1（完全正相關）
- |r| > 0.7 → 強相關；0.3-0.7 → 中度相關；< 0.3 → 弱相關

- 陷阱：相關性 ≠ 因果關係；非線性關係 r 也可能很低
- 多變數分析：
- `sns.pairplot()` 一次看所有數值欄位的兩兩關係（EDA 神器）
- Correlation heatmap：快速找出哪些欄位強相關
- 用 `hue` 參數加入類別維度，看群組在多維空間的分佈

程式碼範例：

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(style='whitegrid')
np.random.seed(42)

# 模擬資料
n = 400
df = pd.DataFrame({
    'order_amount':    np.random.lognormal(5, 0.7, n),
    'browsing_time':   np.random.exponential(15, n),
    'num_items':       np.random.poisson(3, n) + 1,
    'discount_rate':   np.random.uniform(0, 0.5, n),
    'user_type':       np.random.choice(['新客', '回客', 'VIP'], n, p=[0.3, 0.5, 0.2])
})
# 加入相關性：num_items 越多，order_amount 越高
df['order_amount'] = df['order_amount'] * (1 + df['num_items'] * 0.1)

fig, axes = plt.subplots(2, 3, figsize=(15, 9))

# 散點圖：數量 vs 金額，按顧客類型上色
sns.scatterplot(data=df, x='num_items', y='order_amount', alpha=0.5,
                hue='user_type', ax=axes[0,0])
axes[0,0].set_title('數量 vs 金額')

# 箱型圖：各類顧客的金額分佈（grouped boxplot）
sns.boxplot(data=df, x='user_type', y='order_amount',
            order=['新客', '回客', 'VIP'],
            palette='Set2', ax=axes[0,1])
axes[0,1].set_title('顧客類型 vs 金額分佈')

# 小提琴圖：瀏覽時間 vs 顧客類型（看密度形狀）
sns.violinplot(data=df, x='user_type', y='browsing_time',
               order=['新客', '回客', 'VIP'],
               palette='pastel', ax=axes[0,2])
axes[0,2].set_title('瀏覽時間 vs 顧客類型')

# 相關性熱圖
numeric_cols = ['order_amount', 'browsing_time', 'num_items', 'discount_rate']
```

```
corr = df[numeric_cols].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', vmin=-1, vmax=1,
            fmt='.2f', square=True, ax=axes[1,0])
axes[1,0].set_title('[redacted]')
[redacted]
axes[1,1].axis('off')
axes[1,1].text(0.1, 0.6, "[redacted]", fontsize=14, fontweight='bold', transform=axes[1,1].transAxes)
axes[1,1].text(0.1, 0.45, "[redacted] vs [redacted]r = 0.97", fontsize=10, transform=axes[1,1].transAxe
axes[1,1].text(0.1, 0.35, "[redacted]", fontsize=10, color='coral', transform=axes[1,1].transAxe
axes[1,1].text(0.1, 0.20, "[redacted] ≠ [redacted]\n[redacted]", fontsize=10, transform=axes[1,1]
[redacted]
[redacted]pairplot [redacted]
axes[1,2].axis('off')
axes[1,2].text(0.5, 0.5, "sns.pairplot(df, hue='user_type')\n\n[redacted]",
               ha='center', va='center', fontsize=11, transform=axes[1,2].transAxes,
               bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.8))
[redacted]
plt.suptitle('[redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
```

視覺建議：

重點標注相關性矩陣中顏色深淺代表的含義，以及「冰淇淋與溺水」的反例插圖，強化「相關性不等於因果」的

銜接下一張：

「分析過程中，我們常常遇到讓人困惑的點：那些看起來不正常的值。它們是錯誤還是訊號？」

Slide 8：異常值：資料品質問題還是業務訊號？

核心訊息：異常值不等於錯誤。在刪除之前，先判斷它的來源——有些異常值是最有價值的洞察。

講師講解要點：

- 異常值的兩種來源：
- 資料品質問題：輸入錯誤（年齡 = 999）、系統 bug（金額 = -1）、合併錯誤
- 真實業務訊號：超級 VIP 的高消費訂單、惡意刷單行為、黑天鵝事件
- 識別方法：
- IQR 法則：Q1 - 1.5×IQR 到 Q3 + 1.5×IQR 之外視為異常（Boxplot 就是這個邏輯）
- 3σ 法則：均值 ± 3 個標準差之外（適用常態分佈）
- 視覺化判讀：Boxplot、Scatter plot、直方圖尾部

- 判斷流程：
- 識別（在哪裡？有多少筆？）
- 分類（資料品質問題 or 真實業務事件？）
- 處理（刪除 / 修正 / 保留並標記 / 單獨分析）
- 最重要的原則：永遠不要不問原因就刪除異常值

程式碼範例：

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

np.random.seed(42)
n = 500
amounts = np.concatenate([
    np.random.lognormal(5, 0.6, 480),      # 正常交易
    np.random.uniform(30000, 80000, 10),   # 大額 VIP 交易
    [-99, 0, 999999, 100000, -1]           # 異常值
])

# IQR 方法
Q1, Q3 = np.percentile(amounts, [25, 75])
IQR = Q3 - Q1
lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
outliers = amounts[(amounts < lower) | (amounts > upper)]

print(f"Q1={Q1:.0f}, Q3={Q3:.0f}, IQR={IQR:.0f}")
print(f"正常範圍: [{lower:.0f}, {upper:.0f}]")
print(f"異常值 {len(outliers)} 筆: {sorted(outliers)[:10]}")

# 畫圖
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# 直方圖
axes[0].hist(amounts, bins=50, color='steelblue', alpha=0.7)
axes[0].axvline(lower, color='red', linestyle='--', label=f'下限 {lower:.0f}')
axes[0].axvline(upper, color='red', linestyle='--', label=f'上限 {upper:.0f}')
axes[0].set_title('金額分布 + 異常值'); axes[0].legend()

# Boxplot：IQR 視覺化
axes[1].boxplot(amounts, vert=True, patch_artist=True,
                boxprops=dict(facecolor='lightblue'))
axes[1].set_title('Boxplot：四分位數')
axes[1].set_ylabel('金額')

# 清洗後
clean = amounts[(amounts > 0) & (amounts < 200000)]  # 簡單規則
axes[2].hist(clean, bins=50, color='mediumseagreen', alpha=0.7)
```

```
axes[2].set_title('[redacted]')
[redacted]
plt.suptitle('[redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
[redacted]
[redacted]
strategies = {
    '[redacted] (-99, -1)':    '[redacted] → [redacted]',
    '[redacted] (0)':          '[redacted]',
    '[redacted] VIP [redacted]':  '[redacted] → [redacted]',
    '[redacted] (999999)':  '[redacted] → [redacted]'
[redacted]
print("\n=== [redacted] ===")
for k, v in strategies.items():
    print(f"  {k}: {v}")
```

視覺建議： 左側顯示三步決策流程（識別 → 分類 → 處理），右側顯示一個具體範例：同樣是「金額 > 上界」，VIP 大單（保留）和輸入錯誤（刪除）的不同處理邏輯。

銜接下一張：「現在讓我們把前面學到的東西整合成一個完整的 EDA 實戰流程。」

Slide 9：真實案例：電商用戶行為 EDA 全流程

核心訊息：真實的 EDA 不是線性的——它是不斷從發現到追問再到發現的迭代過程。讓我們完整走一遍。

講師講解要點：

此投影片展示一個完整的 EDA 流程，從資料取得到形成假設，模擬真實工作中的思考過程。

業務情境： 電商平台希望了解：「哪些因素影響用戶的首次購買行為？如何提升新用戶轉換率？」

EDA 全流程程式碼：

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy import stats
[redacted]
sns.set_theme(style='whitegrid', palette='muted')
np.random.seed(42)
```

```
# ========== [redacted] 0 [redacted] ==========
n = 2000
df = pd.DataFrame({
    'user_id':          range(1, n+1),
    'register_channel': np.random.choice(['organic', 'paid_search', 'social', 'referral'],
                                         n, p=[0.35, 0.30, 0.25, 0.10]),
    'device':           np.random.choice(['mobile', 'desktop', 'tablet'], n, p=[0.55, 0.35, 0.10]),
    'city_tier':        np.random.choice([1, 2, 3], n, p=[0.30, 0.45, 0.25]),
    'browsing_pages':   np.random.poisson(8, n) + 1,
    'session_duration': np.random.exponential(12, n),  # [redacted]
    'days_to_purchase': np.random.exponential(5, n)    # [redacted]
})
# [redacted]
purchase_prob = 0.05 + df['browsing_pages'] * 0.03 + (df['device'] == 'desktop').astype(float) * 0
purchase_prob = purchase_prob.clip(0, 0.85)
df['converted'] = np.random.binomial(1, purchase_prob)
df['first_order_amount'] = np.where(
    df['converted'] == 1,
    np.random.lognormal(5.2, 0.6, n),
    np.nan
)

print("=" * 50)
print("[redacted] 1[redacted]")
print("=" * 50)
print(f"[redacted]{df.shape}")
print(f"\n[redacted]\n{df.dtypes}")
print(f"\n[redacted]\n{df.isnull().sum()}")
print(f"\n[redacted]\n{df.describe().round(2)}")
print(f"\n[redacted]{df['converted'].mean():.1%}")

# ---------- [redacted] 2[redacted] ----------
print("\n" + "=" * 50)
print("[redacted] 2[redacted]")
print("=" * 50)

fig, axes = plt.subplots(2, 3, figsize=(15, 9))

# [redacted]
channel_counts = df['register_channel'].value_counts()
axes[0,0].bar(channel_counts.index, channel_counts.values,
              color=sns.color_palette('pastel'))
axes[0,0].set_title('[redacted]'); axes[0,0].set_ylabel('[redacted]')

# [redacted]
sns.histplot(df['browsing_pages'], bins=30, kde=False, ax=axes[0,1], color='steelblue')
axes[0,1].set_title(f'[redacted]={df['browsing_pages'].mean():.1f}[redacted]')

# [redacted]
sns.histplot(df['session_duration'], bins=40, kde=True, ax=axes[0,2], color='coral')
axes[0,2].set_title('[redacted]')

# [redacted]
device_counts = df['device'].value_counts()
axes[1,0].pie(device_counts.values, labels=device_counts.index,
```

```
                    autopct='%1.1f%%', colors=sns.color_palette('Set2'))
axes[1,0].set_title('[redacted]')

# [redacted]
city_counts = df['city_tier'].value_counts()
axes[1,1].bar(city_counts.index, city_counts.values, color=sns.color_palette('muted'))
axes[1,1].set_title('[redacted]')

# [redacted]
converted_df = df[df['converted'] == 1]
sns.histplot(converted_df['first_order_amount'], bins=40, kde=True, ax=axes[1,2], color='mediumseg
axes[1,2].set_title(f'[redacted]n={len(converted_df)}[redacted]')

plt.suptitle('[redacted] 2[redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()

# ========== [redacted] 3[redacted]==========
print("\n" + "=" * 50)
print("[redacted] 3[redacted] — [redacted]")
print("=" * 50)

fig, axes = plt.subplots(2, 2, figsize=(13, 10))

# [redacted] vs [redacted]
channel_conv = df.groupby('register_channel')['converted'].mean().sort_values(ascending=False)
axes[0,0].bar(channel_conv.index, channel_conv.values,
              color=['#2ecc71' if v > df['converted'].mean() else '#e74c3c' for v in channel_conv.v
axes[0,0].axhline(df['converted'].mean(), color='navy', linestyle='--', label=f'[redacted] {df["convert
axes[0,0].set_title('[redacted]'); axes[0,0].set_ylabel('[redacted]')
axes[0,0].legend()
for i, (ch, val) in enumerate(channel_conv.items()):
    axes[0,0].text(i, val + 0.005, f'{val:.1%}', ha='center', fontsize=10)

# [redacted] vs [redacted]
device_conv = df.groupby('device')['converted'].mean().sort_values(ascending=False)
axes[0,1].bar(device_conv.index, device_conv.values, color=sns.color_palette('Set2', 3))
axes[0,1].axhline(df['converted'].mean(), color='navy', linestyle='--')
axes[0,1].set_title('[redacted]'); axes[0,1].set_ylabel('[redacted]')

# [redacted] vs [redacted] bin [redacted]
df['page_bin'] = pd.cut(df['browsing_pages'], bins=[0, 3, 6, 10, 15, 100],
                        labels=['1-3[redacted]', '4-6[redacted]', '7-10[redacted]', '11-15[redacted]', '15[redacted]'])
page_conv = df.groupby('page_bin', observed=True)['converted'].mean()
axes[1,0].bar(range(len(page_conv)), page_conv.values, color='steelblue', alpha=0.8)
axes[1,0].set_xticks(range(len(page_conv)))
axes[1,0].set_xticklabels(page_conv.index, rotation=20)
axes[1,0].set_title('[redacted] vs [redacted]')
axes[1,0].set_ylabel('[redacted]')

# [redacted] vs [redacted]
sns.boxplot(data=converted_df, x='city_tier', y='first_order_amount',
            order=['[redacted]', '[redacted]', '[redacted]'], palette='muted', ax=axes[1,1])
axes[1,1].set_title('[redacted] vs [redacted]')

plt.suptitle('[redacted] 3[redacted]', fontsize=14, y=1.01)
```

```
plt.tight_layout(); plt.show()

# ========== 步驟 4：產生假設 ==========
print("\n" + "=" * 50)
print("步驟 4：產生假設")
print("=" * 50)
hypotheses = [
    "H1：週末訂單量顯著高於平日，建議調整週末人力排班",
    "H2：新客戶在第一週的轉換率最低，建議加強首購引導流程",
    "H3：高客單價訂單集中在特定時段，建議推出時段限定優惠",
]
for h in hypotheses:
    print(f"  {h}")
print("\n[提示] M6: 把這些假設寫成可驗證的實驗")
```

視覺建議：
用四個步驟的標籤框架呈現整個流程，每個步驟顯示關鍵輸出（截圖或示意圖），並標注「在這個步驟發現了什

銜接下一張：「找到了洞察，怎麼讓別人也理解它？這需要把分析結果包裝成業務語言。」

Slide 10：如何用一頁圖表講業務故事

核心訊息：圖表本身不說話，說話的是圖表背後的詮釋結構：觀察 → 洞察 → 建議。

講師講解要點：

- 大多數分析師的圖表只停在「觀察」層：「這個月銷售額下降了 15%。」這是描述，不是洞察
- 觀察（Observation）：資料告訴我們什麼事實？客觀陳述，有數字支撐
- 範例：「桌機用戶的轉換率（32%）比手機用戶（18%）高出 78%」
- 洞察（Insight）：為什麼會這樣？背後的機制或模式是什麼？
- 範例：「桌機購物流程讓用戶能比較更多商品，複雜購物決策在桌機上完成率更高」
- 建議（Recommendation）：基於這個洞察，我們應該做什麼？
- 範例：「優先優化手機端的商品比較功能，目標是將手機轉換率提升至 25%」
- 一頁紙的力量：三段文字 + 一張主圖 + 一個洞察句標題，強迫自己聚焦

業務故事範本：

```
[redacted]
│
[redacted]
│
[redacted]
[redacted]
[redacted]
│
[redacted]
[redacted]
[redacted]
│
[redacted]
[redacted]
[redacted]
```

視覺建議：
示範一頁「業務故事圖」樣板，包含：大標題（洞察句，不是「銷售分析」），主圖（標注關鍵數據點），三段

銜接下一張：「理論說完了。現在輪到你，用真實資料做一次完整的 EDA。」

Slide 11：【練習 A】完整 EDA 探索

核心訊息：EDA 能力只能從實作中建立，不能從聽講中建立。

練習說明：

資料集：零售電商訂單資料（提供清洗後的 CSV，約 5000 筆）

欄位包含：訂單日期、用戶 ID、商品分類、購買金額、是否首購、城市等級、獲客渠道、瀏覽頁數

任務說明：

1. 看全貌（5 分鐘）
2. 執行 `df.info()`、`df.describe()`、缺值統計

記錄：資料維度、有問題的欄位、任何立即可見的異常

單變數分析（10 分鐘）

5. 至少對 3 個數值欄位畫 histogram 或 boxplot
6. 至少對 2 個類別欄位做 value_counts() 並畫 bar chart

記錄：哪些欄位分佈異常？有沒有異常值需要處理？

雙變數分析（10 分鐘）

9. 至少探索 3 組變數關係（包含數值×數值、數值×類別各至少一組）

10. 畫相關性矩陣熱力圖

記錄：哪些關係最強？有什麼出乎意料的發現？

形成假設（5 分鐘）

13. 列出 2-3 個「基於 EDA 發現，值得進一步驗證的業務假設」

評估標準： - 是否完整走過三個層次（單變數 → 雙變數） - 圖表選型是否符合問題類型 - 假設是否具體且可被驗證

時間：30 分鐘實作，5 分鐘小組分享

講師引導要點： - 提醒學員先寫問題再畫圖，不要打開資料就開始畫 - 引導觀察：「你畫這張圖想問什麼問題？這張圖有沒有回答到？」 - 常見卡關點：只有觀察沒有假設 → 追問「如果這個發現是真的，對業務意味著什麼？」

Slide 12：【練習 B】三段洞察簡報

核心訊息：分析的終點不是圖表，是可以讓他人理解並行動的洞察。

練習說明：

基於練習 A 的 EDA 結果，選出你認為最有業務價值的三個發現，製作三頁洞察簡報。

每一頁的格式：

```
## [illegible] #N [illegible]

### [illegible]
[illegible]

### [illegible]
[illegible]

### [illegible]
```

[redacted]

評估標準：

評估面向	優秀	及格	需改善
圖表選型	圖形類型完全匹配問題類型	圖形能回答問題但非最佳選擇	圖形和問題不匹配
觀察層	有具體數字，客觀陳述	有描述但缺乏數字支撐	只說「有差異」沒有程度
洞察層	有解釋機制，超越觀察	有部分解釋	洞察 = 觀察的重複
建議層	具體、可行動、有優先順序	方向正確但不夠具體	太模糊或不現實

時間：25 分鐘製作，10 分鐘互評分享

講師引導要點：
- 重點不在報告精美，在於洞察是否有「解釋力」
- 常見錯誤：把觀察層和洞察層寫成一樣的東西
- 追問技巧：「如果移除這張圖，你的結論還成立嗎？那就是說，這張圖沒有在支撐你的結論。」

補充：What -> So What -> Now What 洞察框架

這是顧問業與資料團隊最常用的洞察組織框架，能幫你把圖表上的發現，轉化為可以行動的商業建議。

層次	問的是	例子
What（觀察）	資料告訴我們什麼客觀事實？	「Q3 的退貨率從 5% 上升到 12%」
So What（洞察）	這代表什麼？為什麼重要？	「退貨集中在新品類，可能是品質控管或商品指

Now What（建議）	應該採取什麼行動？	「暫停新品類上架，先進行品質抽檢與商品頁改

使用時機： - 練習 B 的三段洞察簡報，每一頁都應該走完這三層 - EDA 報告的每個關鍵發現，都用這個框架組織 - 向非技術人員報告時，這三層就是你的故事結構

常見錯誤： - 只有 What 沒有 So What：「退貨率上升了」— 然後呢？ - So What 和 What 重複：「退貨率上升代表退貨變多了」— 這不是洞察 - Now What 太模糊：「應該關注退貨問題」— 要具體到可以行動

補充：視覺化反模式 — 常見錯誤與修正

反模式	問題	正確做法
用圓餅圖做比較	人眼對角度的判斷力遠不如長度	改用水平長條圖
Y 軸不從 0 開始	誇大差異，誤導讀者	折線圖可不從 0，長條圖必須從 0
3D 圖表	透視變形讓數值判讀失真	永遠用 2D，除非資料本身是三維
彩虹色（rainbow colormap）	色彩順序無直覺對應，色盲不友善	用連續漸層（viridis）或語義色（紅=負面、綠=
一張圖塞太多系列	超過 5-7 條線就無法區分	拆成多張子圖（subplot），或用互動式圖表
缺少標題和軸標籤	讀者不知道在看什麼	每張圖必須有標題，軸標籤含單位
裝飾性圖表	加了很多格線、陰影、背景色	「少即是多」：去除非必要的視覺元素（Tufte 原則）

一句話原則：如果移除一個視覺元素後圖表的訊息不變，那就移除它。

模組總結

本模組的核心主線

視覺化工具（Matplotlib + Seaborn）只是手段，EDA 方法論才是核心。工具知道怎麼用，但只有掌握「單變數 -> 雙變數 -> 多變數」的系統路徑，才能從資料中提取有價值的洞察。

四個不能忘的原則

1. 先問問題，再畫圖：每一張圖都應該對應一個具體問題
2. 異常值先判斷再處理：刪除之前，先問「這是錯誤還是訊號？」
3. 洞察必須超越觀察：圖表不說話，詮釋才說話（用 What -> So What -> Now What 框架）
4. 避開反模式：圓餅圖少用、3D 不用、彩虹色不用、標題軸標必寫

下一步：M6 統計思維與假設檢定

EDA 的終點是形成假設。M6 將學習如何用統計工具驗證這些假設——把「看起來有差異」轉化為「在 X% 信心水準下確認有差異」。本模組在 EDA 中發現的那些「可能的關係」，在 M6 會學到怎麼嚴謹地確認它們。

參考文獻與引用來源

經典文獻

1. Tukey, J.W. (1977). Exploratory Data Analysis. Pearson. — EDA 方法論的奠基之作。「The greatest value of a picture is when it forces us to notice what we never expected to see.」出自本書第 vi 頁。
2. Anscombe, F.J. (1973). "Graphs in Statistical Analysis." The American Statistician, 27(1), 17-21. — Anscombe's Quartet

的原始論文，示範統計量相同但分布截然不同的四組資料。doi.org/10.1080/00031305.1973.10478966

視覺化原則

1. Tufte, E.R. (2001). The Visual Display of Quantitative Information, 2nd Edition. Graphics Press. — 資料視覺化設計的聖經，「data-ink ratio」原則的來源。
2. Cairo, A. (2016). The Truthful Art: Data, Charts, and Maps for Communication. New Riders. — 資料新聞與商業視覺化的實務指南。
3. Knaflic, C.N. (2015). Storytelling with Data. Wiley. — 「用資料說故事」的商業簡報方法論。

洞察框架

1. What -> So What -> Now What 框架 — 此框架廣泛使用於管理顧問業（McKinsey、BCG 等）與企業資料團隊。最早的系統化描述可追溯至 Minto, B. (1987). The Pyramid Principle. Pearson. 該書提出的金字塔原理強調「結論先行、以上統下」的思維方式。

視覺化反模式

1. 圓餅圖的判讀問題 — Cleveland, W.S. & McGill, R. (1984). "Graphical Perception: Theory, Experimentation, and Application to the Development of Graphical Methods." Journal of the American Statistical Association, 79(387), 531-554.
此研究證實人眼對長度的判讀精度遠高於角度。
2. 色彩選擇 — viridis 色票的設計考量可參見 matplotlib.org/stable/users/explain/colors/colormaps.html

工具文件

1. Matplotlib Official Documentation — matplotlib.org
2. Seaborn Official Documentation — seaborn.pydata.org

延伸閱讀

- Tukey, J.W. (1977). Exploratory Data Analysis — EDA 的奠基之作，確立了「探索」而非「確認」的分析哲學
- Cairo, A. (2016). The Truthful Art — 視覺化的倫理與設計原則，每位資料分析師必讀
- Hunter, J.D. (2007). Matplotlib: A 2D Graphics Environment — Matplotlib 作者的原始論文，理解設計哲學
- Waskom, M. (2021). Seaborn: Statistical Data Visualization — Seaborn 官方文件，包含所有圖形的詳細參數說明
- Wickham, H. (2010). A Layered Grammar of Graphics — 圖形語法的理論基礎，理解「為什麼選這個圖」

Module 6：統計思維與假設檢定

課程定位

所屬課程：2026 Python 數據分析與 AI 基礎：從資料思維到機器學習入門（24 小時）
模組編號：Module 6／共 8 個模組 時長：3 小時 前置模組：M5 視覺化與 EDA 後續模組：M7 ML 前導與學習路線

模組核心主張：沒有統計，你只是在描述現象，不是在判斷現象。描述說「有差異」，統計說「這個差異可靠嗎」。

> "Statistics turns observations into confidence."

學習目標

完成本模組後，學員能夠：

1. 區分母體與樣本、描述統計與推論統計，解釋為什麼光靠均值會誤導決策
2. 說明常態分佈、二項分佈、泊松分佈的直覺意義，並識別哪種業務場景適用哪種分佈
3. 理解信賴區間的含義，正確解讀「95% 信賴區間」的意思（及常見誤解）
4. 解釋假設檢定的邏輯：H0、H1、p-value、顯著水準，並識別常見的誤用陷阱
5. 區分統計顯著性與實際顯著性（effect size），避免「$p < 0.05$ 就代表重要」的誤解
6. 使用 `scipy.stats` 執行單樣本 t 檢定、雙樣本 t 檢定、卡方檢定
7. 設計並解讀一個完整的 A/B Test：從假設設定到樣本數計算到結果解讀
8. 意識到多重比較問題，理解為什麼同時測試 20 個假設會有一個「顯著」的機率

關鍵概念清單

統計基礎

- [ ] 母體（population）vs 樣本（sample）的本質差異
- [ ] 描述統計 vs 推論統計的目的差異
- [ ] 均值、變異數、標準差的直覺意義
- [ ] 常態分佈：為什麼它無所不在？中央極限定理直覺
- [ ] 二項分佈：適用場景（成功/失敗事件）
- [ ] 泊松分佈：適用場景（單位時間內的隨機事件計數）
- [ ] 偏態分佈：收入、房價、用戶停留時間的真實形狀
- [ ] 抽樣分佈與標準誤（Standard Error）

信賴區間與假設檢定

- [ ] 信賴區間：「95% CI」的正確解讀方式
- [ ] 假設檢定邏輯：H0、H1、p-value、顯著水準 α
- [ ] Type I Error（假陽性）vs Type II Error（假陰性）
- [ ] 效果量（Effect Size）：Cohen's d 的直覺
- [ ] 統計顯著（$p < 0.05$）vs 實際顯著（有業務意義）
- [ ] 相關性 vs 因果關係：為什麼觀察研究不能下因果結論

常用統計檢定

- [ ] 單樣本 t 檢定：樣本均值 vs 目標值
- [ ] 雙樣本 t 檢定（獨立樣本）：兩組均值差異
- [ ] 卡方檢定：類別變數的獨立性
- [ ] `scipy.stats` 基本使用：`ttest_1samp`、`ttest_ind`、`chi2_contingency`

A/B Test

- [ ] A/B Test 的實驗設計要素：隨機分組、樣本數計算、測試時間
- [ ] 統計功效（Statistical Power）與樣本數的關係
- [ ] A/B Test 結果解讀：四象限決策矩陣
- [ ] 多重比較問題（Multiple Testing Problem）的直覺理解

投影片大綱

#	投影片標題	核心訊息
1	為什麼分析需要統計？	沒有統計，你只是在描述，不是在判斷
2	母體、樣本與描述 vs 推論統計	統計是不確定性的語言，不是計算器
3	分佈直覺：三種你必須認識的分佈	不同的隨機現象，有不同的自然形狀
4	均值不夠用：為什麼要看分佈？	相同均值，完全不同的故事
5	信賴區間：「我有多確定？」	點估計只給你答案，CI 給你答案的可靠範圍
6	假設檢定：差異可靠嗎？	它問的是：這個差異是不是可靠到足以讓我行動
7	p-value 的正確解讀與常見陷阱	$p < 0.05 \neq$ 差異很大；$\neq$ H0 是假的
8	效果量：統計顯著 vs 實際顯著	百萬樣本讓任何微小差異都「顯著」
9	scipy.stats 實戰：三種常用檢定	用程式碼做出有意義的統計判斷

10	A/B Test 完整案例：從設計到決策	統計是產品決策的底層工具
11	多重比較與相關性陷阱	同時測 20 個，必然有一個「顯著」
12	【練習】業務問題翻譯機 + A/B Test 設計	實作：把業務問題翻成統計語言

詳細投影片內容

Slide 1：為什麼分析需要統計？

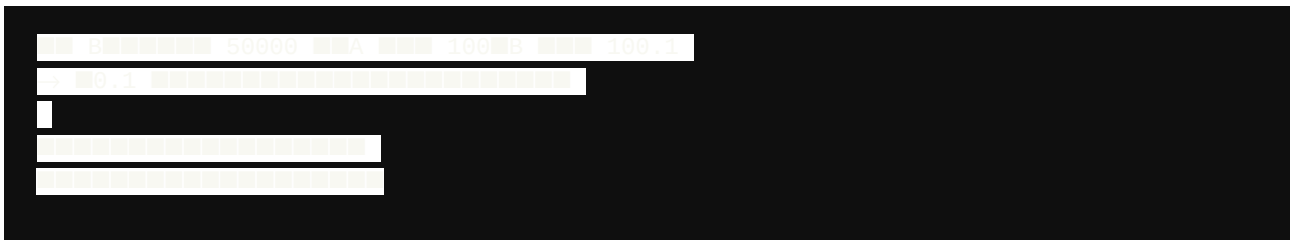
核心訊息：沒有統計，你只是在描述現象，不是在判斷現象。

講師講解要點：

- 情境題：你做了一個新的 App 引導流程，A 版轉換率 12%，B 版轉換率 14%。B 版比較好嗎？
- 純描述的回答：「B 版高了 2 個百分點。」——這是觀察，不是判斷
- 統計的回答：「在樣本數 n=500 的情況下，這 2% 的差異在 95% 信賴水準下無法排除是隨機誤差造成的。」——這是判斷
- 統計的本質：在資料永遠不完整的情況下，用嚴謹的方式量化我們對判斷的信心
- 不用統計的代價：做了很多「看起來有效」但其實只是噪聲的決策，浪費資源，甚至反效果
- 統計不是數學課：不是為了算出答案，而是為了知道答案有多可靠

直覺範例：

```
■■ A■■■■■■■ 50 ■■A ■■■ 100■B ■■■ 105
→ ■5 ■■■■■■■
```



視覺建議： 左右對比：左側「純描述的世界」——一張柱狀圖，A=12%, B=14%，下方標注「看起來 B 贏了」。右側「有統計的世界」——同一張圖加上誤差棒（error bar），誤差棒重疊，下方標注「差異在信心範圍內無法確認」。兩個問題框對比：「哪個高？」vs「差異是真實

銜接下一張：
「要理解統計的判斷力，必須先建立幾個核心概念的直覺：母體、樣本，以及兩種統計的目的。」

Slide 2：母體、樣本與描述 vs 推論統計

核心訊息：統計不是計算器，是不確定性的語言。每個數字背後都有一個關於「我們能知道多少」的假設。

講師講解要點：

- 母體（Population）：你真正想了解的全集——所有用戶、所有訂單、所有未來可能的實驗結果。我們幾
- 樣本（Sample）：我們實際拿到的那一部分。分析的對象是樣本，但判斷的目標是母體
- 母體參數 vs 樣本統計量：
- 母體均值 μ（mu）：未知，是我們想估計的目標
- 樣本均值 $\bar{x}$（x-bar）：已知，是我們手上的估計值
- 標準誤（Standard Error）= $\sigma / \sqrt{n}$：衡量 $\bar{x}$ 有多可能偏離 μ
- 描述統計（Descriptive Statistics）：總結手上的樣本資料——均值、中位數、標準差、分佈形狀。目的：「描述我有什麼」
- 推論統計（Inferential Statistics）：從樣本推斷母體——假設檢定、信賴區間、回歸分析。目的：「推論我不知道的」
- 關鍵直覺：樣本越大，樣本統計量越接近母體參數，不確定性越小

概念對照表：

概念	符號	性質	可得性
母體均值	μ（mu）	固定但未知	幾乎不可得
樣本均值	$\bar{x}$	已知，隨樣本不同而變化	從資料計算
母體標準差	σ（sigma）	固定但未知	幾乎不可得
樣本標準差	s	已知，隨樣本不同而變化	從資料計算
標準誤	$SE = s/\sqrt{n}$	衡量樣本均值的不確定性	從資料計算

程式碼範例：

```
import numpy as np
from scipy import stats

np.random.seed(42)
# ══════════════════════
population = np.random.normal(loc=500, scale=80, size=100_000)
mu_true = population.mean()
print(f"═══════ μ = {mu_true:.2f}")

# ═══════════════
for n in [10, 50, 200, 1000]:
    sample = np.random.choice(population, size=n, replace=False)
    x_bar = sample.mean()
    se = sample.std() / np.sqrt(n)  # ═══
    print(f"  n={n:5d}: x̄={x_bar:.2f}, SE={se:.2f}, ══={abs(x_bar - mu_true):.2f}")

# ═══════════════
sample_means = [np.random.choice(population, size=50).mean() for _ in range(1000)]
print(f"\n1000═══════n=50═══════")
print(f"  ══════ = {np.mean(sample_means):.2f}═══════ μ = {mu_true:.2f}═")
print(f"  ═══════ = {np.std(sample_means):.2f}═══════════")
```

視覺建議：

四個方塊概念圖：母體（大圓虛線）、樣本（小圓實線）、描述統計（總結樣本）、推論統計（從樣本推估母體
越大，SE 越小」的視覺示意：多個小樣本均值形成的分佈，隨 n 增大而收窄。

銜接下一張：

「理解了母體和樣本，我們需要認識幾種常見的資料分佈——因為不同的隨機現象有不同的自然形狀。」

Slide 3：分佈直覺：三種你必須認識的分佈

核心訊息：不同的隨機現象有不同的自然形狀。選對分佈，你的模型才接地氣。

講師講解要點：

- 為什麼要認識分佈？
- 理解資料的「自然形狀」，才能做出正確的統計判斷

假設資料符合常態分佈但實際上是偏態 → 統計結論全錯

常態分佈（Normal Distribution）

- 參數：μ（中心）、σ（寬度）
- 為什麼到處都是？中央極限定理：大量獨立隨機因素疊加的結果，趨向常態
- 現實例子：測量誤差、身高、考試成績

68-95-99.7 法則：μ±1σ 包含 68%，μ±2σ 包含 95%，μ±3σ 包含 99.7%

二項分佈（Binomial Distribution）

- 參數：n（試驗次數）、p（成功機率）
- 適用場景：「n 次獨立試驗中，成功 k 次的機率」
- 現實例子：n 個用戶中有幾個轉換、n 個郵件中有幾個被點開

A/B Test 的轉換率比較就是二項分佈的應用

泊松分佈（Poisson Distribution）

- 參數：λ（單位時間/空間內的平均事件數）
- 適用場景：「固定時間/空間內，罕見事件發生次數」
- 現實例子：每小時客服來電數、每天系統錯誤次數、每週大單數量

程式碼範例：

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
```

```
import seaborn as sns

sns.set_theme(style='whitegrid')
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# ---- 1. [redacted] ----
x = np.linspace(-4, 4, 300)
for mu, sigma, label, color in [(0, 1, 'μ=0, σ=1[redacted]', 'steelblue'),
                                (0, 2, 'μ=0, σ=2[redacted]', 'coral'),
                                (1, 1, 'μ=1, σ=1[redacted]', 'mediumseagreen')]:
    axes[0].plot(x, stats.norm.pdf(x, mu, sigma), label=label, linewidth=2, color=color)
axes[0].axvspan(-1, 1, alpha=0.1, color='steelblue')
axes[0].set_title('[redacted]\n[redacted]')
axes[0].set_xlabel('[redacted]'); axes[0].set_ylabel('[redacted]')
axes[0].legend(fontsize=9)
axes[0].text(0, 0.15, '68%', ha='center', fontsize=10, color='steelblue')

# ---- 2. [redacted] ----
k = np.arange(0, 31)
for n, p, color, label in [(30, 0.3, 'steelblue', 'n=30, p=0.3[redacted]'),
                           (30, 0.5, 'coral',     'n=30, p=0.5[redacted]'),
                           (30, 0.7, 'mediumseagreen', 'n=30, p=0.7[redacted]')]:
    pmf = stats.binom.pmf(k, n, p)
    axes[1].plot(k, pmf, 'o-', label=label, color=color, markersize=4, linewidth=1.5)
axes[1].set_title('[redacted]\n[redacted] k [redacted]')
axes[1].set_xlabel('[redacted] k'); axes[1].set_ylabel('[redacted]')
axes[1].legend(fontsize=9)

# ---- 3. [redacted] ----
k2 = np.arange(0, 20)
for lam, color, label in [(2, 'steelblue', 'λ=2[redacted]'),
                          (5, 'coral',     'λ=5[redacted]'),
                          (10, 'mediumseagreen', 'λ=10[redacted]')]:
    pmf = stats.poisson.pmf(k2, lam)
    axes[2].plot(k2, pmf, 'o-', label=label, color=color, markersize=4, linewidth=1.5)
axes[2].set_title('[redacted]\n[redacted]')
axes[2].set_xlabel('[redacted] k'); axes[2].set_ylabel('[redacted]')
axes[2].legend(fontsize=9)

plt.suptitle('[redacted]', fontsize=14, y=1.02)
plt.tight_layout(); plt.show()

# [redacted]
print("=== [redacted] ===")
print("[redacted]")
pop = np.random.exponential(scale=5, size=100_000)  # [redacted]
fig2, axes2 = plt.subplots(1, 4, figsize=(14, 4))
axes2[0].hist(pop[:500], bins=30, color='coral', alpha=0.7)
axes2[0].set_title('[redacted]')
for i, n in enumerate([5, 30, 100], 1):
    means = [np.random.choice(pop, n).mean() for _ in range(2000)]
    axes2[i].hist(means, bins=30, color='steelblue', alpha=0.7)
    axes2[i].set_title(f'[redacted] n={n}[redacted]')
plt.suptitle('[redacted]n [redacted]', y=1.02)
plt.tight_layout(); plt.show()
```

視覺建議：
三列對照，每列一種分佈：分佈名稱、數學直覺（用文字，不用公式）、現實業務場景例子、圖形示意。底部加
→ 是固定試驗次數的成功計數嗎 → 二項；是單位時間內的罕見事件 → 泊松；是連續數值且中央極限定理適用 → 常態。

銜接下一張：「理解了分佈，我們回到一個核心問題：為什麼光看均值不夠？」

Slide 4：均值不夠用：為什麼要看分佈？

核心訊息：均值是一個點，分佈是完整的故事。兩組相同均值的資料，可以有完全不同的業務含義。

講師講解要點：

- 均值的陷阱：比爾・蓋茲走進一間酒吧，酒吧裡所有人的「平均財富」立刻暴增——但沒有人變富了。均
- 為什麼看分佈更重要：
- 分佈讓你看到「大多數人在哪裡」而不只是「中心在哪裡」
- 雙峰分佈意味著存在兩個完全不同的用戶群體，用均值代表他們是荒唐的
- 右偏分佈（收入、房價）中，均值 >> 中位數，用均值描述「典型用戶」會嚴重誤導
- 偏度（Skewness）的直覺：
- 正偏（右偏）：長尾在右，均值 > 中位數。收入、消費金額、留存天數
- 負偏（左偏）：長尾在左，均值 < 中位數。考試成績（難度低時）
- 多種集中趨勢量數的對比：
- 均值：受極端值影響最大，代表「數學重心」
- 中位數：不受極端值影響，代表「典型值」
- 眾數：出現最多次的值，代表「最常見值」

程式碼範例：

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

sns.set_theme(style='whitegrid')
```

```
np.random.seed(42)
[redacted]
fig, axes = plt.subplots(2, 2, figsize=(13, 10))
[redacted]
# ---- [redacted] Anscombe [redacted] ----
np.random.seed(42)
group_a = np.random.normal(500, 50, 500)                    # [redacted]
group_b = np.concatenate([np.random.normal(300, 30, 250),   # [redacted]
                          np.random.normal(700, 30, 250)])
# [redacted]
group_b = group_b - group_b.mean() + group_a.mean()
[redacted]
axes[0,0].hist(group_a, bins=40, alpha=0.6, label=f'A [redacted]={group_a.mean():.0f}[redacted]', color='steelb
axes[0,0].hist(group_b, bins=40, alpha=0.6, label=f'B [redacted]={group_b.mean():.0f}[redacted]', color='coral'
axes[0,0].axvline(group_a.mean(), color='navy',   linestyle='--', linewidth=2)
axes[0,0].axvline(group_b.mean(), color='darkred', linestyle='--', linewidth=2)
axes[0,0].set_title('[redacted]\n[redacted] ≠ [redacted]')
axes[0,0].legend()
[redacted]
# ---- [redacted]2[redacted] vs [redacted] vs [redacted] ----
income = np.concatenate([np.random.lognormal(10, 0.5, 950),
                         np.random.uniform(5e6, 1e7, 50)])  # [redacted]
mean_inc = income.mean()
median_inc = np.median(income)
mode_approx = stats.mode(income.astype(int))[0]
[redacted]
axes[0,1].hist(income, bins=100, color='steelblue', alpha=0.7)
axes[0,1].axvline(mean_inc,   color='red',   linestyle='--', linewidth=2, label=f'[redacted] {mean_inc/1
axes[0,1].axvline(median_inc, color='green', linestyle='--', linewidth=2, label=f'[redacted] {median_in
axes[0,1].set_xlim(0, income.quantile(0.99) if hasattr(income, 'quantile') else np.percentile(inco
axes[0,1].set_title('[redacted]\n[redacted]')
axes[0,1].legend()
axes[0,1].set_xlabel('[redacted]')
[redacted]
# [redacted] xlim
axes[0,1].set_xlim(0, np.percentile(income, 99))
[redacted]
# ---- [redacted]3[redacted] ----
x = np.linspace(0, 1000, 300)
for sigma, color, label in [(50, 'steelblue', 'σ=50[redacted]'),
                            (150, 'coral', 'σ=150[redacted]'),
                            (300, 'mediumseagreen', 'σ=300[redacted]')]:
    from scipy.stats import norm
    axes[1,0].plot(x, norm.pdf(x, 500, sigma), label=label, linewidth=2, color=color)
axes[1,0].axvline(500, color='black', linestyle=':', linewidth=1.5, label='[redacted]=500')
axes[1,0].set_title('[redacted]\n[redacted]')
axes[1,0].legend(); axes[1,0].set_xlabel('[redacted]')
[redacted]
# ---- [redacted]4[redacted] ----
right_skew = np.random.lognormal(3, 1, 1000)
normal     = np.random.normal(right_skew.mean(), right_skew.std()*0.5, 1000)
axes[1,1].hist(right_skew, bins=50, alpha=0.6, label=f'[redacted]={stats.skew(right_skew):.1f}[redacted]', col
axes[1,1].hist(normal,     bins=50, alpha=0.6, label=f'[redacted]={stats.skew(normal):.1f}[redacted]', color=
axes[1,1].axvline(right_skew.mean(), color='darkred', linestyle='--', linewidth=1.5)
axes[1,1].axvline(np.median(right_skew), color='red', linestyle=':', linewidth=1.5)
```

```
axes[1,1].set_title('[redacted]\n[redacted]')
axes[1,1].legend()

plt.suptitle('[redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
```

視覺建議：重點標注第一個子圖（相同均值的雙峰 vs 單峰），用紅色虛線指出兩條均值線重疊，但分佈形狀截然不同。旁邊加文字框：「如果你只回報均值，你的老

銜接下一張：
「知道了分佈，我們現在需要一個工具，讓估計值帶著不確定性一起傳遞——這就是信賴區間。」

Slide 5：信賴區間：「我有多確定？」

核心訊息：點估計只給你一個答案，信賴區間給你「答案有多可靠」的範圍。

講師講解要點：

- 為什麼需要信賴區間？
- 樣本均值 $\bar{x}$ 只是一個點估計。每次重新抽樣都會得到不同的 $\bar{x}$
- 信賴區間告訴我們：「如果重複做 100 次這個研究，約有 95 次，真實的母體均值 μ 會落在這個區間內」

它量化了我們的不確定性，讓決策更有根據

95% CI 的正確解讀：

- ☒ 正確：「在相同方法重複抽樣 100 次下，約有 95 個區間會包含真實母體均值」
- ☒ 錯誤：「母體均值有 95% 的機率落在這個區間」（母體均值是固定的，不是隨機的！）

☒ 錯誤：「95% 的資料落在這個區間」（這是說分佈範圍，不是說 CI）

CI 越窄越好嗎？

- 更窄的 CI = 更高的確定性，但需要更大的樣本數
- 寬 CI + 不包含零 → 有差異但不精確
- 窄 CI + 不包含零 → 有差異且很精確（最好的情況）

任何 CI 包含零 → 無法排除「沒有差異」

CI 的計算直覺：CI = x̄ ± z × SE，其中 SE = s / √n

程式碼範例：

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import seaborn as sns

sns.set_theme(style='whitegrid')
np.random.seed(42)

# ---- [redacted] ----
population_mean = 500  # [redacted]
population = np.random.normal(population_mean, 80, 100_000)

n = 50
num_experiments = 100
contained = 0

fig, ax = plt.subplots(figsize=(10, 8))
for i in range(num_experiments):
    sample = np.random.choice(population, size=n)
    x_bar = sample.mean()
    se = sample.std() / np.sqrt(n)
    ci_low = x_bar - 1.96 * se
    ci_high = x_bar + 1.96 * se

    color = 'steelblue' if ci_low <= population_mean <= ci_high else 'red'
    if color == 'steelblue':
        contained += 1
    ax.plot([ci_low, ci_high], [i, i], color=color, alpha=0.6, linewidth=1.5)
    ax.plot(x_bar, i, 'o', color=color, markersize=3)

ax.axvline(population_mean, color='black', linestyle='--', linewidth=2, label=f'[redacted] μ={popula
ax.set_xlabel('[redacted]')
ax.set_title(f'100 [redacted] 95% [redacted]\n[redacted]={contained} [redacted] μ[redacted]={100-contained} [redacted]\n[redacted] 95
ax.legend(); plt.tight_layout(); plt.show()

# ---- [redacted] CI [redacted] ----
sample_from_pop = np.random.choice(population, size=1000)  # [redacted]
print('--- [redacted] vs [redacted] ---')
for n_size in [10, 30, 100, 500, 1000]:
    sample_n = sample_from_pop[:n_size]
    ci = stats.t.interval(0.95, df=n_size-1,
                          loc=sample_n.mean(),
                          scale=stats.sem(sample_n))
    width = ci[1] - ci[0]
    print(f'  n={n_size:4d}: CI=[{ci[0]:.1f}, {ci[1]:.1f}], [redacted]={width:.1f}, [redacted]={sample_n.mean()

# ---- [redacted] CI ----
sample = np.random.normal(520, 75, 80)
ci_95 = stats.t.interval(0.95, df=len(sample)-1, loc=sample.mean(), scale=stats.sem(sample))
```

```
ci_99 = stats.t.interval(0.99, df=len(sample)-1, loc=sample.mean(), scale=stats.sem(sample))
print(f"\n[redacted]n={len(sample)}[redacted]={sample.mean():.1f}, SE={stats.sem(sample):.1f}")
print(f"95% CI: [{ci_95[0]:.1f}, {ci_95[1]:.1f}] [redacted]={ci_95[1]-ci_95[0]:.1f}")
print(f"99% CI: [{ci_99[0]:.1f}, {ci_99[1]:.1f}] [redacted]={ci_99[1]-ci_99[0]:.1f}")
```

視覺建議：重點展示 100 個 CI 的圖，大多數藍色（包含真實值），少數紅色（未包含），直覺理解「95% CI 的含義」。旁邊加一個「CI 寬度 vs 樣本數」的反比關係圖。

銜接下一張：
「信賴區間給了我們估計的可靠範圍。假設檢定更進一步——它讓我們用嚴謹的邏輯做出是/否的判斷。」

Slide 6：假設檢定：差異可靠嗎？

核心訊息：假設檢定問的不是「有沒有差異」，而是「這個差異可靠到足以讓我行動嗎」。

講師講解要點：

- 假設檢定的核心邏輯（法庭類比）：
- H0（虛無假設）= 無罪推定：預設「沒有差異，你看到的差異只是抽樣噪聲」
- H1（對立假設）= 起訴理由：「有差異，而且這個差異是真實的」
- p-value = 反對無罪的證據強度：「如果 H0 是真的，看到這麼大差異的機率是多少？」
- 顯著水準 α = 定罪門檻：通常設為 0.05

結論：$p < \alpha \rightarrow$ 拒絕 H0（定罪）；$p \geq \alpha \rightarrow$ 無法拒絕 H0（無罪釋放）

假設檢定的五步驟：

- 設定 H0 和 H1（H0 永遠是「沒有效果」的假設）
- 選擇顯著水準 α（最常用 0.05，醫療或飛安用 0.01）
- 收集資料，計算檢定統計量（t 值、z 值、χ^2 值）
- 計算 p-value

根據 p vs α 做決定

Type I Error vs Type II Error：

- Type I（α 錯誤，假陽性）：H0 是真的，但我們拒絕了它。「冤枉好人」
- Type II（β 錯誤，假陰性）：H0 是假的，但我們沒有拒絕它。「放跑壞人」
- 統計功效（Power）= 1 - β：正確拒絕假 H0 的機率，目標通常 ≥ 0.8

假設設定練習：

```
[illegible]
|
H0: [illegible] ≤ [illegible]
    [illegible] μ_new ≤ μ_old [illegible] ≤ 0
|
H1: [illegible] > [illegible]
    [illegible] μ_new > μ_old [illegible] > 0
|
[illegible] α = 0.05
[illegible] p < 0.05 [illegible]
```

視覺建議： 流程圖：情境描述 → 設定 H0/H1 → 收集資料 → 計算統計量 → 計算 p-value → 與 α 比較 → 做決定。旁邊加「法庭類比」對照表，以及 Type I/II Error 的 2x2 矩陣（真實情況 × 我們的決定）。

銜接下一張： 「p-value 是假設檢定的核心輸出，但它也是最常被誤解的統計概念之一。讓我們把誤解一個個拆除。」

Slide 7：p-value 的正確解讀與常見陷阱

核心訊息：p < 0.05 不代表差異很大，不代表 H0 是假的，也不代表結果可以重現。

講師講解要點：

p-value 的正確定義：「如果 H0 為真（真的沒有差異），那麼我們觀察到這麼大（或更大）差異的機率。」 p = 0.03 意思是：「如果根本沒有差異，我有 3% 的機率看到這麼大的差異。」

常見誤解逐一破解：

誤解	正確理解

$p < 0.05$ 代表差異很大	不是。p 只說差異是否可靠，不說差異有多大
$p = 0.03$ 代表 H0 有 3% 機率是真的	不是。p 是在 H0 為真的條件下計算的，不是 H0 為真的機率
$p > 0.05$ 代表 H0 是真的	不是。只是證據不足以拒絕 H0，而不是證明 H0
$p < 0.05$ 代表結果一定能重現	不是。樣本波動仍然存在
$\alpha = 0.05$ 是自然法則	不是。這是一個慣例，應根據決策代價調整

- 單尾 vs 雙尾檢定：
- 雙尾（two-tailed）：問「有沒有差異」（方向未知）
- 單尾（one-tailed）：問「是不是比較大/小」（方向已知）
- 建議：除非有強力理論依據，預設用雙尾檢定，更保守

視覺建議：左側顯示常態分佈，標注臨界區域（拒絕 H0 的區域）和 p-value 的位置。右側顯示「誤解清單」，每個誤解用紅色 ☒ 標記，正確理解用綠色 ✓ 標記。底部加一個「p-value 單頁速查卡」。

銜接下一張：「p-value 只說差異是否可靠，但不說差異有多大。這個「多大」的問題，需要效果量來回答。」

Slide 8：效果量：統計顯著 vs 實際顯著

核心訊息：百萬樣本能讓任何微小差異都「統計顯著」——但 0.001% 的轉換率提升值得全面改版嗎？

講師講解要點：

- 為什麼需要效果量（Effect Size）？
- 統計顯著性受樣本數影響極大：n 足夠大，任何差異都會 $p < 0.05$
- 效果量衡量差異的「實際大小」，獨立於樣本數

決策應該同時考慮：統計顯著（差異是真實的嗎？）AND 實際顯著（差異夠大嗎？）

Cohen's d（連續變數效果量）：

- d = (μ[redacted] - μ[redacted]) / s_pooled
- 解讀：d = 0.2（小效果），d = 0.5（中等效果），d = 0.8（大效果）

直覺：「兩組均值差了幾個標準差？」

業務決策四象限：

統計顯著（p < 0.05）	統計不顯著（p ≥ 0.05）
實際顯著（有大效果量）：支持大改善	延長測試，或增加樣本數
實際不顯著（微效果）：評估 ROI	維持現狀，沒有證據也沒有意義

程式碼範例：

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import seaborn as sns

sns.set_theme(style='whitegrid')
np.random.seed(42)

# ═══ [redacted] ═══
print("═══ [redacted] p-value [redacted] ═══")
true_diff = 0.5   # [redacted] 0.5 [redacted]
std       = 50.0  # [redacted] 50 [redacted]

for n in [30, 100, 500, 2000, 10000, 100000]:
    # [redacted]
    group_a = np.random.normal(500, std, n)
    group_b = np.random.normal(500 + true_diff, std, n)
    t_stat, p_val = stats.ttest_ind(group_a, group_b)
    cohens_d = true_diff / std  # [redacted]
    print(f"  n={n:7d}: p={p_val:.4f} {'***' [redacted]' if p_val < 0.05 else '    [redacted]'},"
          f"Cohen's d={cohens_d:.3f}[redacted]{'[redacted]' if cohens_d > 0.8 else '[redacted]' if cohens_d > 0.5 else '[redacted]' i

# ═══ [redacted] p-value ═══
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

sample_sizes = [30, 50, 100, 200, 500, 1000, 5000, 10000]
```

```
p_values = []
for n in sample_sizes:
    a = np.random.normal(500, 50, n)
    b = np.random.normal(501, 50, n)  # 微小差異 d ≈ 0.02
    _, p = stats.ttest_ind(a, b)
    p_values.append(p)

axes[0].plot(sample_sizes, p_values, 'o-', color='steelblue', linewidth=2)
axes[0].axhline(0.05, color='red', linestyle='--', linewidth=1.5, label='α = 0.05')
axes[0].set_xscale('log')
axes[0].set_xlabel('樣本數 (log scale)'); axes[0].set_ylabel('p-value')
axes[0].set_title('效果量固定 d=0.02，樣本越大越顯著\n(效果量不變，只是統計力量增加)')
axes[0].legend()

# Cohen's d 直覺
x = np.linspace(-4, 6, 300)
for d, label, color in [(0.2, "小效果 (d=0.2)", 'steelblue'),
                        (0.5, "中效果 (d=0.5)", 'coral'),
                        (0.8, "大效果 (d=0.8)", 'mediumseagreen')]:
    axes[1].plot(x, stats.norm.pdf(x, 0, 1), color=color, linewidth=2, linestyle='--', alpha=0.5)
    axes[1].plot(x, stats.norm.pdf(x, d, 1), color=color, linewidth=2, label=label)
axes[1].axvline(0, color='black', linestyle=':', linewidth=1)
axes[1].set_title("Cohen's d 直覺：兩組分佈重疊程度")
axes[1].set_xlabel('標準差'); axes[1].legend()

plt.suptitle('樣本數 vs 效果量', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
```

視覺建議： 左圖展示「樣本數對 p-value 的影響」（效果量固定但樣本越大越顯著），右圖展示 Cohen's d 的直覺（兩組分佈重疊程度）。底部放決策四象限表格。

銜接下一張：「理論夠了，來看程式碼怎麼做——scipy.stats 讓統計判斷變成三行 Python。」

Slide 9：scipy.stats 實戰：三種常用檢定

核心訊息：用程式碼落地統計判斷，掌握 scipy.stats 的三個核心 API。

講師講解要點：

- 單樣本 t 檢定：我的樣本均值和目標值（或歷史值）有沒有顯著差異？
- 使用：`stats.ttest_1samp(sample, popmean=target)`
- 獨立雙樣本 t 檢定：兩組樣本的均值有沒有顯著差異？
- 使用：`stats.ttest_ind(group_a, group_b, equal_var=False)`
- 注意：`equal_var=False` 使用 Welch's t-test，不假設兩組變異數相等（更保守，更常用）

- 卡方獨立性檢定：兩個類別變數之間有沒有關聯？
- 使用：`stats.chi2_contingency(contingency_table)`
- 適合：點擊率 vs 設備類型、轉換率 vs 來源渠道

完整程式碼範例：

```
import numpy as np
from scipy import stats
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

np.random.seed(42)
print("="*60)
print("[redacted] t [redacted] NPS [redacted] 70 [redacted]")
print("="*60)
nps_scores = np.random.normal(72, 15, 80)  # [redacted] NPS
target_nps = 70
t_stat, p_val = stats.ttest_1samp(nps_scores, popmean=target_nps)
print(f"[redacted]: {nps_scores.mean():.2f}")
print(f"t [redacted]: {t_stat:.3f}")
print(f"p-value: {p_val:.4f}")
if p_val < 0.05:
    direction = "[redacted]" if nps_scores.mean() > target_nps else "[redacted]"
    print(f"[redacted]: [redacted] → [redacted] NPS [redacted]{direction}[redacted] {target_nps} [redacted]95% [redacted]")
else:
    print(f"[redacted]: [redacted] → [redacted] NPS [redacted] {target_nps} [redacted]")

print("\n" + "="*60)
print("[redacted] t [redacted] vs [redacted]")
print("="*60)
desktop_orders = np.random.normal(650, 120, 150)
mobile_orders = np.random.normal(580, 160, 300)  # [redacted]
t_stat, p_val = stats.ttest_ind(desktop_orders, mobile_orders, equal_var=False)
cohens_d = (desktop_orders.mean() - mobile_orders.mean()) / \
           np.sqrt((desktop_orders.std()**2 + mobile_orders.std()**2) / 2)
print(f"[redacted]: {desktop_orders.mean():.2f}[redacted]n={len(desktop_orders)}[redacted]")
print(f"[redacted]: {mobile_orders.mean():.2f}[redacted]n={len(mobile_orders)}[redacted]")
print(f"[redacted]: {desktop_orders.mean() - mobile_orders.mean():.2f}")
print(f"t [redacted]: {t_stat:.3f}  p-value: {p_val:.4f}")
print(f"Cohen's d: {cohens_d:.3f} ({'[redacted]' if abs(cohens_d) > 0.8 else '[redacted]' if abs(cohens_d) > 0.5 el
if p_val < 0.05:
    print("[redacted]: [redacted]")
    print(f"  [redacted] {desktop_orders.mean()-mobile_orders.mean():.0f} [redacted]
          f"[redacted] {abs(cohens_d):.2f}[redacted]{'[redacted]' if abs(cohens_d) > 0.5 else '[redacted]'}[redacted]")
else:
    print("[redacted]: [redacted]")

print("\n" + "="*60)
print("[redacted]")
print("="*60)
```

```
[redacted]vs [redacted]
contingency = pd.DataFrame({
    '[redacted]': [850, 720, 680, 195],
    '[redacted]': [150, 280, 120, 105]
}, index=['organic', 'paid', 'social', 'referral'])
print("[redacted]")
print(contingency)
print(f"\n[redacted]")
print((contingency['[redacted]'] / contingency.sum(axis=1)).round(3))
[redacted]
chi2, p_val, dof, expected = stats.chi2_contingency(contingency)
print(f"\n[redacted]: {chi2:.3f}, df={dof}")
print(f"p-value: {p_val:.6f}")
if p_val < 0.05:
    print("[redacted]: [redacted]")
else:
    print("[redacted]: [redacted]")
```

視覺建議：
三段程式碼用不同色彩框分隔，每段後面標注「適用情境」。右側加一個「選擇哪種檢定」決策樹：比較均值 → 一組 vs 目標值 → 單樣本 t；兩組均值 → 雙樣本 t；類別 vs 類別 → 卡方。

銜接下一張：「現在把所有概念整合在一個完整的 A/B Test 案例裡，從設計到決策走一遍。」

Slide 10：A/B Test 完整案例：從設計到決策

核心訊息：A/B Test 是用統計把產品決策從「感覺上比較好」變成「數據上可以確認更好」。

業務情境：[redacted]電商平台在考慮改版結帳頁面（簡化步驟），預期可提升轉換率。目前轉換率：18%。希望能提升至 21%（效果量 +3 個百分點）。

完整 A/B Test 流程：

```
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns
[redacted]
sns.set_theme(style='whitegrid')
np.random.seed(42)
[redacted]
[redacted]
print("=" * 60)
print("[redacted]")
print("=" * 60)
```

[redacted]

```
p_control = 0.18  # [redacted]
p_treatment = 0.21  # [redacted]
alpha = 0.05  # [redacted]
power = 0.80  # [redacted]80% [redacted]

# [redacted]using [redacted]
def calc_sample_size(p1, p2, alpha=0.05, power=0.80):
    z_alpha2 = stats.norm.ppf(1 - alpha/2)  # [redacted]
    z_beta = stats.norm.ppf(power)
    p_bar = (p1 + p2) / 2
    n = ((z_alpha2 * np.sqrt(2 * p_bar * (1 - p_bar)) +
         z_beta * np.sqrt(p1*(1-p1) + p2*(1-p2))) / (p2 - p1)) ** 2
    return int(np.ceil(n))

n_required = calc_sample_size(p_control, p_treatment, alpha, power)
print(f"[redacted]: {p_control:.0%}")
print(f"[redacted]: {p_treatment:.0%}[redacted] MDE = {p_treatment-p_control:.0%}[redacted]")
print(f"[redacted] α: {alpha}")
print(f"[redacted]: {power:.0%}")
print(f"[redacted]: {n_required}")
print(f"[redacted]: {n_required * 2}[redacted] {n_required}[redacted]")
print(f"[redacted] 5000[redacted] {n_required * 2 / 5000:.1f} [redacted]")

# ============ [redacted] 2[redacted]============
print("\n" + "=" * 60)
print("[redacted] 2[redacted]")
print("=" * 60)
n_actual = n_required  # [redacted]
control = np.random.binomial(1, p_control, n_actual)
treatment = np.random.binomial(1, p_treatment + 0.005, n_actual)  # [redacted]

conv_c = control.mean()
conv_t = treatment.mean()
print(f"[redacted]A[redacted]: n={n_actual}, [redacted]={conv_c:.2%}")
print(f"[redacted]B[redacted]: n={n_actual}, [redacted]={conv_t:.2%}")
print(f"[redacted]: {conv_t - conv_c:+.2%}")

# ========== [redacted] 3[redacted] ==========
print("\n" + "=" * 60)
print("[redacted] 3[redacted]")
print("=" * 60)
print("H0[redacted] ≤ [redacted]")
print("H1[redacted] > [redacted]")

# [redacted] z [redacted]
from statsmodels.stats.proportion import proportions_ztest
count = np.array([treatment.sum(), control.sum()])
nobs = np.array([n_actual, n_actual])
z_stat, p_val = proportions_ztest(count, nobs, alternative='larger')

print(f"\nz [redacted]: {z_stat:.3f}")
print(f"p-value: {p_val:.4f}")

# [redacted]
```

```
effect_size = conv_t - conv_c
relative_lift = (conv_t - conv_c) / conv_c
print(f"[redacted]: {effect_size:+.2%}")
print(f"[redacted]: {relative_lift:+.1%}")
[redacted]
# ========== [redacted] 4[redacted] ==========
print("\n" + "=" * 60)
print("[redacted] 4[redacted]95% [redacted]")
print("=" * 60)
from statsmodels.stats.proportion import confint_proportions_2indep
ci = confint_proportions_2indep(treatment.sum(), n_actual,
                                control.sum(),   n_actual,
                                method='agresti-caffo')
print(f"[redacted] 95% CI: [{ci[0]:.3%}, {ci[1]:.2%}]")
if ci[0] > 0:
    print("CI [redacted] > 0[redacted]")
else:
    print("CI [redacted]")
[redacted]
# ========== [redacted] 5[redacted] ==========
print("\n" + "=" * 60)
print("[redacted] 5[redacted]")
print("=" * 60)
[redacted]
stat_sig = p_val < alpha
practical_sig = abs(effect_size) >= 0.02  # [redacted] 2 [redacted]
[redacted]
if stat_sig and practical_sig:
    decision = "[redacted]"
    reason = f"[redacted]p={p_val:.4f}<{alpha}[redacted]{effect_size:+.2%} > 2%[redacted]"
elif stat_sig and not practical_sig:
    decision = "[redacted] ROI [redacted]"
    reason = f"[redacted]{effect_size:+.2%} < 2%[redacted]"
else:
    decision = "[redacted]"
    reason = f"[redacted]p={p_val:.4f}[redacted]"
[redacted]
print(f"[redacted]{decision}")
print(f"[redacted]{reason}")
[redacted]
# ========== [redacted] ==========
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
[redacted]
# [redacted]
bars = axes[0].bar(["[redacted]", "[redacted]"], [conv_c, conv_t],
                   color=['#3498db', '#2ecc71'], alpha=0.85, width=0.5)
axes[0].axhline(conv_c, color='blue', linestyle='--', alpha=0.5)
for bar, val in zip(bars, [conv_c, conv_t]):
    axes[0].text(bar.get_x() + bar.get_width()/2, val + 0.005, f"{val:.2%}",
                 ha='center', fontsize=12, fontweight='bold')
axes[0].set_title(f"A/B Test [redacted]\np={p_val:.4f}[redacted]={effect_size:+.2%}[redacted]{"[redacted]" if stat_sig else "[redacted]
axes[0].set_ylabel("[redacted]"); axes[0].set_ylim(0, max(conv_c, conv_t) * 1.3)
```

```
[redacted]
[redacted]
matrix_data = [['[redacted]\n[redacted]', '[redacted]\n[redacted]'],
               ['[redacted] ROI\n[redacted]', '[redacted]\n[redacted]']]
colors_matrix = [['#2ecc71', '#f39c12'], ['#3498db', '#95a5a6']]
[redacted]
axes[1].axis('off')
table = axes[1].table(cellText=matrix_data,
                      rowLabels=['[redacted]\n([redacted])', '[redacted]\n([redacted])'],
                      colLabels=['[redacted]\n(p < 0.05)', '[redacted]\n(p ≥ 0.05)'],
                      cellColours=colors_matrix,
                      loc='center', cellLoc='center')
table.scale(1.5, 3)
table.auto_set_font_size(False); table.set_fontsize(10)
axes[1].set_title('A/B Test [redacted]', fontsize=12, pad=20)
[redacted]
[redacted]
current_row = 0 if practical_sig else 1
current_col = 0 if stat_sig else 1
print(f"\n[redacted] {current_row+1} [redacted] {current_col+1} [redacted]")
[redacted]
plt.suptitle('A/B Test [redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
```

視覺建議：上方顯示 A/B Test 全流程（五個步驟的標籤框），下方左側是轉換率柱狀圖（含誤差棒），右側是決策四象限矩陣，用顏色標注當

銜接下一張：「A/B Test 是強大的工具，但有兩個常見陷阱必須認識：多重比較問題和相關性的因果誤解。」

Slide 11：多重比較與相關性陷阱

核心訊息：同時測試 20 個假設，必然有一個「顯著」——這不是統計的成功，是多重比較的失敗。

講師講解要點：

多重比較問題（Multiple Testing Problem）：

- 單次檢定，α = 0.05：假陽性率 = 5%
- 同時做 20 個獨立檢定：至少一個假陽性的機率 = 1 - (0.95)^20 ≈ 64%
- 業務現場的常見陷阱：同時測試「點擊率、轉換率、客均價、停留時間、跳出率...」20 個指標
- 解決方案：Bonferroni 校正（最保守）：α' = α / m，其中 m 是檢定次數

- 做 20 個檢定時，每個的顯著水準降為 0.05 / 20 = 0.0025
- 更實際的做法：事前聲明主要指標（pre-registration），只有主要指標需要嚴格校正

相關性與因果關係的界限：

- 觀察到 X 和 Y 相關，可能的原因：
- X 導致 Y（X → Y）
- Y 導致 X（反向因果）
- 有第三變數 Z 同時導致 X 和 Y（混淆變數）
- 純屬巧合（偽相關）
- 隨機對照實驗（RCT）才能建立因果關係，因為隨機分組消除了混淆變數
- A/B Test 是在網際網路公司最接近 RCT 的方法

程式碼範例：

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import seaborn as sns

sns.set_theme(style='whitegrid')
np.random.seed(42)

# ---- [redacted] ----
print("=== [redacted] 10000 [redacted] ===")
print("[redacted]A/B [redacted]H0 [redacted]")

num_simulations = 10000
num_tests       = 20  # [redacted] 20 [redacted]
false_positive_any = 0

for _ in range(num_simulations):
    found_any_significant = False
    for _ in range(num_tests):
        a = np.random.normal(0, 1, 50)
        b = np.random.normal(0, 1, 50)  # [redacted]
        _, p = stats.ttest_ind(a, b)
        if p < 0.05:
            found_any_significant = True
            break
    if found_any_significant:
        false_positive_any += 1

print(f"\n[redacted] {num_simulations} [redacted]")
```

```
print(f"  [redacted]{false_positive_any/num_simulations:.1%}[redacted] 64%[redacted]")
print(f"  [redacted]1 - (0.95)^20 = {1-(0.95)**20:.1%}")
[redacted]
print("\n=== Bonferroni [redacted] ===")
alpha_adjusted = 0.05 / num_tests
print(f"[redacted] α: 0.05")
print(f"[redacted] α: {alpha_adjusted:.4f}[redacted] = 0.05 / {num_tests}[redacted]")
print(f"[redacted]Type II Error [redacted]")
[redacted]
# ═══ [redacted] ═══
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
[redacted]
num_tests_range = range(1, 51)
fp_probs = [1 - (1 - 0.05)**k for k in num_tests_range]
axes[0].plot(num_tests_range, fp_probs, color='coral', linewidth=2)
axes[0].axhline(0.05, color='steelblue', linestyle='--', linewidth=1.5, label='[redacted] 5%')
axes[0].axhline(0.64, color='red', linestyle=':', linewidth=1.5, label='20 [redacted] 64%')
axes[0].fill_between(num_tests_range, fp_probs, 0.05, alpha=0.2, color='coral')
axes[0].set_xlabel('[redacted]')
axes[0].set_ylabel('[redacted]')
axes[0].set_title('[redacted]\n[redacted]')
axes[0].legend(), axes[0].set_ylim(0, 1)
[redacted]
# [redacted]Spurious Correlation[redacted]
years = np.arange(1999, 2010)
# [redacted]
pool_drownings = [109, 102, 102, 98, 85, 95, 96, 98, 123, 94, 102][:len(years)]
cage_movies    = [2, 2, 2, 3, 1, 1, 2, 3, 4, 1, 4][:len(years)]
[redacted]
ax2 = axes[1]
color_a, color_b = 'steelblue', 'coral'
ln1 = ax2.plot(years, pool_drownings, 'o-', color=color_a, linewidth=2, label='[redacted]')
ax2.set_xlabel('[redacted]'); ax2.set_ylabel('[redacted]', color=color_a)
ax2_twin = ax2.twinx()
ln2 = ax2_twin.plot(years, cage_movies, 's--', color=color_b, linewidth=2, label='Nicolas Cage [redacted]
ax2_twin.set_ylabel('Nicolas Cage [redacted]', color=color_b)
[redacted]
r, p = stats.pearsonr(pool_drownings[:len(years)], cage_movies[:len(years)])
ax2.set_title(f'[redacted]\nr = {r:.2f}[redacted]p = {p:.3f}\n[redacted]')
[redacted]
lns = ln1 + ln2
labels = [l.get_label() for l in lns]
ax2.legend(lns, labels, loc='upper left')
[redacted]
plt.suptitle('[redacted] + [redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
[redacted]
# ═══ [redacted] ═══
print("\n=== [redacted] ===")
causal_criteria = {
    "[redacted]": "X [redacted] Y [redacted]",
    "[redacted]": "X [redacted] Y [redacted]",
    "[redacted]": "[redacted] Z [redacted] X [redacted] Y[redacted]",
    "[redacted]": "[redacted] X [redacted] Y[redacted]",
    "[redacted]": "[redacted]X [redacted]A/B test[redacted]",
```

```
for k, v in causal_criteria.items():
    print(f"  [{k}] {v}")
```

視覺建議：左圖展示「檢定次數 vs 假陽性率」的曲線（隨 m 增大快速上升），右圖展示一個著名的偽相關例子（Nicolas Cage 電影數 vs 泳池溺水人數），用幽默的方式強化「相關 ≠ 因果」的印象。

銜接下一張：「把所有概念整合在一起，輪到你把業務問題翻譯成統計問題了。」

Slide 12：【練習】業務問題翻譯機 + A/B Test 設計

核心訊息：統計的實用價值在於，它能讓你把「我覺得」換成「資料顯示，在 X% 信心水準下」。

練習 A：業務問題翻譯機（20 分鐘）

將以下三個業務問題翻譯成可被統計回答的統計問題：

#	業務問題	翻譯目標
1	「新的 App 推播設計讓用戶更願意打開嗎？」	翻譯成 A/B Test 假設檢定設計
2	「週末的平均訂單金額真的比平日高嗎？」	翻譯成雙樣本 t 檢定設計
3	「我們的用戶留存率有沒有在最近三個月改善？」	翻譯成時序比較 + 信賴區間設計

翻譯格式模板：

```
[0] [redacted]
[1] [redacted]
[redacted]
[redacted]
[redacted] ___ [redacted] / [redacted] / [redacted] z [redacted] / ...
```

[redacted]

練習 B：A/B Test 設計（15 分鐘）

情境：你的電商平台首頁目前有一個「通知欄」，你的 PM 想把它改成「個人化推薦欄」。你需要設計一個 A/B Test 來評估這個改版。

設計要求：

1. 定義主要指標（Primary Metric）和次要指標（Secondary Metrics）
2. 設定 H0 和 H1
3. 計算最小需要的樣本數（假設：目前點擊率 8%，希望偵測到 2% 的提升，α=0.05，Power=0.80）
4. 說明如何避免多重比較問題（若同時監控多個指標）
5. 設計一個「實驗不純染」的防護措施（避免 A 組用戶看到 B 組內容）

評估標準： - H0 和 H1 方向是否正確（H0 是「沒有效果」） - 樣本數計算過程是否合理 - 是否意識到多重比較風險並有對應方案 - 實驗設計的嚴謹性

練習 C：解讀 A/B Test 結果（10 分鐘）

以下是一個已完成的 A/B Test 結果，請回答問題：

[redacted]

問題： 1. 主要指標（點擊率）的結論是什麼？效果量如何評估？ 2. 「跳出率顯著下降（p=0.03）」是否應該列入上線理由？為什麼？ 3. 你的最終建議是什麼？（上線 / 延長測試 / 維持現狀）

講師標準答案要點： - 點擊率：統計顯著（p=0.021<0.05），但效果量 h=0.033 屬於微小效果 - 跳出率的 p=0.03 是 8 個次要指標同時比較的結果，需要 Bonferroni 校正後 α=0.00625，p=0.03 不再顯著 - 建議：需評估「0.9% 的點擊率提升」對業務的實際價值，若業務價值明確則可上線

時間分配： - 練習 A：20 分鐘個人實作 - 練習 B：15 分鐘個人實作 - 練習 C：10 分鐘閱讀與回答 - 講師講解：15 分鐘標準答案與討論

模組總結

本模組的核心主線

統計不是數學課，是決策工具。
從「均值有差異」到「差異在統計上可靠」到「差異在業務上值得行動」——這三步缺一不可。

三個不能忘的原則

1. 均值不夠，看分佈：相同均值可以有完全不同的故事
2. $p < 0.05 \neq$ 差異很大：統計顯著性和實際顯著性是兩件事，需要效果量才完整
3. 多重比較是陷阱：同時監控 20 個指標，必然有一個「顯著」，要事前聲明主要指標

下一步：M7 ML 前導與學習路線

本模組建立的統計直覺（分佈、假設檢定、相關性）是機器學習的理論基礎。M7
將展示統計如何延伸成學習算法——從線性回歸的最小二乘法，到邏輯回歸的最大似然估計，到決策樹的信息增

延伸閱讀

參考文獻與引用來源

統計學經典文獻

1. Cohen, J. (1988). Statistical Power Analysis for the Behavioral Sciences, 2nd Edition. Lawrence Erlbaum Associates. — 效果量（effect size）標準的原始來源。Cohen's d 門檻：0.2（小）、0.5（中）、0.8（大）。Cohen 本人曾警告這些是相對標準，應依研究領域調整。
2. Wasserstein, R.L. & Lazar, N.A. (2016). "The ASA Statement on Statistical Significance and p-Values." The American Statistician, 70(2), 129-133. — 美國統計學會（ASA）的官方聲明，釐清 p-value 的六項原則。doi.org/10.1080/00031305.2016.1154108
3. Nuzzo, R. (2014). "Statistical Errors." Nature, 506, 150-152. — 關於 p-value 誤用的重要短文。doi.org/10.1038/506150a

分布與定理

1. 68-95-99.7 法則 — 常態分布的經驗法則，源自 Gauss 的常態分布理論（1809）。任何統計學教科書均有說明，例如 Devore, J.L. Probability and Statistics for Engineering and the Sciences。
2. 中央極限定理（CLT） — 由 Laplace (1810) 首次嚴格陳述，Lindeberg (1922) 與 Feller (1935) 進一步推廣。現代教科書參考：Rice, J.A. (2006). Mathematical Statistics and Data Analysis, 3rd Edition.

相關性門檻

1. 相關係數強度分類（|r| > 0.7 強、0.3-0.7 中、< 0.3 弱） — 此為 Cohen (1988) 提出的慣例門檻。需注意：不同領域的「強相關」標準不同（社會科學 r=0.3 已算中度，物理學 r=0.9 才算起步）。

多重比較問題

1. 多重檢定問題 — Bonferroni 校正原始來源：Bonferroni, C.E. (1936). "Teoria statistica delle classi e calcolo delle probabilita." 現代說明可參見 Bland, J.M. & Altman, D.G. (1995). "Multiple

significance tests: the Bonferroni method." BMJ, 310, 170.

A/B Test 實務

1. Kohavi, R., Tang, D., & Xu, Y. (2020). Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing. Cambridge University Press. — Amazon/Microsoft 的 A/B Test 工業標準參考書。

冰淇淋與溺水相關性

1. 冰淇淋銷量 vs 溺水事故 — 此為統計學教學中的經典虛假相關案例（confounding variable: 氣溫）。學術引用可參見 Vigen, T. (2015). Spurious Correlations. Hachette Books. 以及 tylervigen.com/spurious-correlations

延伸閱讀

1. Downey, A.B. (2014). Think Stats, 2nd Edition. O'Reilly Media. — 用 Python 程式碼建立統計直覺。免費版本：greenteapress.com/thinkstats2
2. Gelman, A. & Hill, J. (2006). Data Analysis Using Regression and Multilevel/Hierarchical Models. Cambridge University Press. — 進階統計建模的標準參考書
3. scipy.stats 官方文件 — docs.scipy.org/doc/scipy/reference/stats.html

---

Module 7：ML 前導與學習路線

課程總時長： 3 小時（本課程最終模組） 模組定位：
在六個模組建立的資料分析底盤之上，打開前方三條技術縱深的視野，並以五條職涯路線為學員指出離開課程後

> "Machine learning turns patterns into prediction."

一、模組定位

這是一門 24 小時課程的最後一塊拼圖。前六個模組建立了：

- M0 開場：課程全景與學習心法
- M1 資料思維：從問題出發看資料
- M2 Python 基礎：型別、語法、資料結構
- M3 NumPy：數值運算與 tensor 概念
- M4 pandas：表格處理與 EDA
- M5 視覺化：從數字到洞察
- M6 統計：推論思維與假設檢定

本模組不是全新的開始，而是一次收束與展望：讓學員看清楚自己已站在哪裡，以及前方哪些路是真實可走的。

三小時結構：

時段	主題	時長
Part A	ML 前導：函數逼近的本質	60 分鐘
Part B	DL + Big Data 前導：規模與架構的轉折點	60 分鐘

Part C	課程收束 + 五條職涯路線地圖	60 分鐘

二、模組學習目標

完成本模組後，學員能夠：

1. 用「函數逼近」的語言解釋機器學習是什麼，而不是用魔法類比
2. 正確描述監督式學習的四個要素：特徵、目標、訓練集、測試集
3. 說明 scikit-learn 在 Python ML 生態中的定位，以及 1.0 版本的里程碑意義
4. 走完一個完整的建模工作流（分割 → 訓練 → 評估 → 迭代），而不是只知道 `.fit()`
5. 解釋深度學習為何在學完 NumPy / tensor 概念後更容易理解，而不是從零開始
6. 描述 PyTorch 2.0 的定位轉移：從「寫模型」到「編譯優化與效能工程」
7. 知道 pandas 的邊界在哪裡，以及 Arrow / Polars / DuckDB / PySpark 各自解決什麼規模的問題
8. 根據自身背景與興趣，選擇一條具體的後續學習路線並說出理由

三、投影片大綱

投影片總覽

#	標題	核心訊息	時段歸屬
S01	你已經走了多遠	回顧七個模組建立的底盤	Part A 開場
S02	ML 不是魔法，是函數逼近	解除「AI 神秘化」的心理屏障	Part A

S03	監督式學習的一張圖	X / y / train / test / feature / target	Part A
S04	scikit-learn：Python ML 的標準入口	里程碑 1.0 的意義	Part A
S05	建模工作流：不只是一行 `.fit()`	split → train → evaluate → iterate	Part A
S06	動手做：scikit-learn 分類 / 迴歸實作	在熟悉資料集上跑完完整流程	Part A 收束
S07	為什麼深度學習在這裡順理成章	tensor / autograd 已在前面出現過	Part B 開場
S08	PyTorch 2.0：從寫模型到效能工程	`torch.compile()` 的轉折	Part B
S09	當 pandas 不夠用時	Arrow → Polars / DuckDB → PySpark 的規模梯度	Part B
S10	PySpark：Python 進入大數據分散式的橋樑	分散式執行模型直覺	Part B 收束
S11	五條職涯路線地圖	統計 / ML / DL / DE / LLM 工程	Part C
S12	課程終章：你搭了一條底盤	最終心法與鼓勵	Part C 收束

S01：你已經走了多遠

核心訊息：
在進入前導內容之前，先讓學員看見自己已建立的資產，而不是只看到前方有多少還沒學。

講師要點：

- 前七個模組不是在「學 Python」，而是在建立一套資料分析的共同語言：型別、陣列、表格、視覺化、統計直覺
- 這些能力之間彼此相連：pandas 的 DataFrame 背後是 NumPy array；NumPy array 的概念直接對應 PyTorch tensor
- 你現在能做的事包含：從原始資料到清理、從清理到分析、從分析到洞察、從洞察到視覺化呈現——這本
- 本模組的任務不是「把這些也教完」，而是讓你知道「接下來要走的路，底盤已經在這裡了」

視覺建議：
一張學習路線圖，七個模組節點已點亮，第七個模組節點（本模組）位於圖的末端，後方分叉出五條路線箭頭，

過渡： 第一站，打開 ML 的黑盒子。

S02：ML 不是魔法，是函數逼近

核心訊息：
機器學習的本質是從資料中找到一個函數，把輸入映射到可預測的輸出——沒有魔法，只有最佳化。

講師要點：

- 很多人對 ML 的第一個印象是「AI 自己學會了」，這個說法雖然直覺，但容易讓人把它神秘化
- 更精確的說法是：給定一批 (input, output) 配對，ML 在找一個函數 f，使得 f(input) 盡可能接近 output
- 這個「找函數」的過程，就是訓練（training）；用來衡量「接近程度」的指標，就是損失函數（loss）
- 訓練結束後，我們用沒看過的資料（測試集）來確認這個函數能不能推廣（generalize），而不只是記住
- 從這個角度來看，線性迴歸、決策樹、神經網路，本質上都在做同一件事——只是函數的形式和複雜度不

視覺建議： 一個最簡化的示意圖：左側是一批散點（X, y），右側是一條曲線（模型）；箭頭標示「訓練」的方向；旁邊附一個公式欄，只寫 `f(X)` ≈

y，加上損失函數的文字說明，不展開數學推導。

過渡： 知道了「在找什麼」，接下來看監督式學習的四個核心概念。

S03：監督式學習的一張圖

核心訊息：
一張圖包含學員需要知道的六個術語：特徵、目標、訓練集、測試集、輸入、輸出。理解這張圖，就能讀懂 90% 的 ML 文章。

講師要點：

- 特徵（feature / X）：輸入給模型的資料欄位，例如「年齡、收入、消費頻率」
- 目標（target / label / y）：希望模型預測的值，例如「是否流失（分類）」或「下個月消費金額（迴歸）」
- 訓練集（train set）：拿來讓模型「看」並調整參數的資料，通常佔總資料的 70–80%
- 測試集（test set）：模型「沒看過」的資料，用來評估真實泛化能力；測試集的表現才是我們真正關心的
- 常見錯誤：在測試集上做決策再重新訓練，等於「考試前先看考題」——結果看起來好，但模型其實不可
- pandas DataFrame 裡已經有 X 和 y 的概念：X 是你選的欄位，y 是你指定的目標欄位

視覺建議： 一張清晰的流程圖，左側是原始 DataFrame，中間用虛線切分成 train / test 兩塊，右側分別接到「訓練模型」和「評估模型」兩個方塊；六個術語用標籤箭頭標注在對應位置。

過渡： 有了概念圖，在進入工具之前，先看看業界怎麼組織一個完整的資料科學專案。

S03-B：CRISP-DM — 資料科學專案的業界標準流程

核心訊息： CRISP-DM（Cross-Industry Standard Process for Data Mining）是資料科學專案最廣泛使用的流程框架。它不是線性的，而是迭代循環的。

講師要點：

- CRISP-DM 六階段：

- Business Understanding（商業理解）：定義問題。「我們要預測什麼？為什麼？成功的標準是什麼？」— 85% 的資料專案失敗不是因為技術，而是因為問題定義不清
- Data Understanding（資料理解）：EDA。看資料長什麼樣、有什麼品質問題、和業務問題的關聯是什麼 — 這就是 M5 教你做的事
- Data Preparation（資料準備）：清理、轉換、特徵工程 — 這就是 M4 教你做的事，通常佔整個專案 60-80% 的時間
- Modeling（建模）：選模型、訓練、調參 — 接下來 S04-S06 要教的
- Evaluation（評估）：模型好不好？不只是看數字，還要回到 Business Understanding 檢查：它真的解決了最初的問題嗎？
- Deployment（部署）：模型上線、監控、維護 — 屬於進階課程範疇
- 關鍵洞察：這不是直線，而是循環。評估結果可能讓你回到資料準備甚至商業理解重新定義問題
- 回顧本課程：M1-M6 其實已經走過了 CRISP-DM 的前三個階段。M7 是第四和第五階段的入門

視覺建議：CRISP-DM 的經典六角形循環圖，六個階段用箭頭連接成環形，中間標示「資料」。在每個階段旁邊標注對應的課程模組（對應 Business Understanding 思維、M4 對應 Data Preparation、M5 對應 Data Understanding 等）。

過渡：知道了完整流程，現在聚焦在建模這一步——需要一個工具把概念變成程式碼。

S04：scikit-learn：Python ML 的標準入口

核心訊息：scikit-learn 是 Python 傳統機器學習（非深度學習）的標準實作，1.0 版本的發布是這個生態成熟度的里程碑。

講師要點：

- scikit-learn 誕生於 2007 年，最初是 Google Summer of Code 專案，現在是全球最廣泛使用的 ML 函式庫之一

- 版本里程碑：scikit-learn 1.0（2021 年）代表這個函式庫正式進入穩定期，API 設計被視為業界標準參考
- 核心設計哲學：一致的 API 介面——分類器、迴歸器、聚類器全部遵循 `.fit()` / `.predict()` / `.score()` 三個方法，學一個等於學會框架
- scikit-learn 的邊界很清楚：它處理結構化資料（表格）的傳統 ML，不處理影像、語音、序列這類深度學習場景——邊界清楚是優點，不是缺陷
- 對學過 pandas 的你來說，scikit-learn 的輸入就是 DataFrame 或 NumPy array，沒有新的資料概念需要學

視覺建議： 一個「Python ML 生態地圖」，中央是 scikit-learn，周圍標示它的上游（NumPy / pandas）和下游（特徵工程、模型選擇、管線化），右側單獨標示「scikit-learn 1.0 里程碑：2021 年」的時間軸。

過渡： 有了工具，看完整的建模流程——不只是 `.fit()` 這一行。

S05：建模工作流：不只是一行 `.fit()`

核心訊息： 建模是一個迭代循環，包含分割資料、訓練、評估、診斷、調整五個步驟。只知道 `.fit()` 是最危險的半吊子狀態。

講師要點：

- 步驟一：分割資料——`train_test_split()` 是第一行，不是最後一行。分割的策略（比例、隨機種子、stratify）本身就是工程決策
- 步驟二：訓練模型——`.fit(X_train, y_train)`，這一行很短，但前面的特徵選擇、缺值處理、標準化才是大工作
- 步驟三：評估模型——`.score()` 或 `classification_report()`，看的是測試集，不是訓練集；訓練集表現好但測試集差 = 過擬合（overfitting）
- 步驟四：診斷與迭代——評估結果告訴你去哪裡：更多資料？換模型？調超參數？回頭看特徵工程？這是
- 建模不是「一次跑對」的過程，而是「每輪評估驅動下一輪改進」的工程循環——這個心態才是正確的

視覺建議：
一個循環箭頭示意圖，四個節點分別標示「分割」「訓練」「評估」「診斷/調整」，並用虛線標示「如果評估通
→
部署」的出口；在「診斷」節點下方列出三個分支：「過擬合」「欠擬合」「資料問題」，各自指向對應的處理

過渡： 概念講完，動手做一次。

S06：動手做：scikit-learn 分類 / 迴歸實作

核心訊息： 在熟悉的資料集上跑完完整的建模工作流，從 DataFrame 到評估報告，感受每一步的實際意義。

講師要點：

- 使用前幾個模組用過的真實資料集（例如客戶資料或公開資料集），而非使用 iris 或 titanic——熟悉的資料讓學員聚焦在流程而非資料理解
- 走完的程式碼骨架：`X, y = df[features], df[target] X_train, X_test, y_train, y_test = train_test_split(X, y) model = LogisticRegression() # ■ LinearRegression() model.fit(X_train, y_train) print(model.score(X_test, y_test)) print(classification_report(y_test, model.predict(X_test)))`
- 重點觀察：訓練集 score vs 測試集 score 的差距，讓學員親眼看見過擬合的現象
- 延伸挑戰（選做）：換一個模型（RandomForest），比較結果；試著調整 `test_size` 比例，觀察對評估指標的影響
- 這個練習的目的不是得到最好的模型，而是把工作流走熟一次

視覺建議： 在投影片中嵌入一個 Jupyter Notebook 截圖，顯示完整的程式碼骨架和一個真實的 `classification_report` 輸出；用紅框標示「測試集評估」的那幾行，強調這是最重要的輸出。

過渡： 傳統 ML 走完了。為什麼接下來深度學習對你來說不是從頭開始？

S07：為什麼深度學習在這裡順理成章

核心訊息： 深度學習不是一門全新的學科，它的基礎概念——tensor、矩陣乘法、梯度下降——你在 NumPy 和 PyTorch 的前導介紹中已經接觸過。

講師要點：

- 你已經知道的東西在 DL 裡叫什麼：NumPy array → tensor；矩陣乘法 → 前向傳播（forward pass）；找最小值 → 梯度下降（gradient descent）；`.grad` → autograd
- 深度學習的核心創新不是「發明了新數學」，而是「用多層函數組合，自動學習特徵」——這是對傳統 ML 手工特徵工程的替代
- 學傳統 ML 先，學 DL 後，這個順序是有意義的：因為 scikit-learn 的工作流（train/test split、評估、迭代）在 DL 裡完全一樣，只是模型結構不同
- 深度學習的門檻不在數學，在計算資源與資料量——你需要更大的資料集、GPU，以及更長的訓練時間；
- 最重要的直覺：DL 的「層」（layer）就是一系列函數的組合，每一層做一次「特徵變換」，多層堆疊讓模型能學到更複雜的模

視覺建議： 一個對照表，左欄是「你已學過的概念」，右欄是「在 DL 中的對應名稱」，共五行，用箭頭連接；下方附一個最簡單的神經網路示意圖（輸入層 → 隱藏層 × 2 → 輸出層），每層用方塊表示。

過渡： DL 的主流框架是 PyTorch——而 2.0 版本代表它進入了一個新的階段。

S08：PyTorch 2.0：從寫模型到效能工程

核心訊息： PyTorch 2.0 是深度學習框架的一個轉折點：核心創新 `torch.compile()` 讓 DL 從「只要能跑」進入「要跑得快、要在生產環境穩定」的工程化階段。

講師要點：

- PyTorch 2.0 里程碑（2023 年）：這不只是版本號的升級，而是框架設計哲學的轉移
- 過去的 PyTorch：以「動態計算圖」為核心優勢——Debug 方便、研究靈活，但生產部署效能相對弱
- `torch.compile()`：在保留動態圖靈活性的前提下，加入編譯式優化——在 A100 GPU 上的實測加速可達 30–200%，且不需要修改模型結構

- 這代表什麼轉變：DL 工程師的職責不再只是「寫出能收斂的模型」，也包括「讓模型在生產環境中跑得快、跑得穩」
- 對初學者的意義：現在學 PyTorch，不只是學 `nn.Linear` / `nn.ReLU`，而是要有「這段程式碼未來要跑在什麼環境」的效能直覺。系統層面的細節（GPU 執行模型、分散式訓練架構）在進階課程「系統設計與架構思維」中有完整介紹

視覺建議： 一個時間軸，標示 PyTorch 各主要版本的里程碑（1.0 穩定 API → 1.8 分散式 → 2.0 compile 優化）；旁邊附一個最小的程式碼對比：「PyTorch 1.x 的訓練迴圈」vs「加上 `torch.compile()` 的 2.0 版本」，只改一行，效能差異用數字標示。

過渡： ML 和 DL 都在處理「建模」問題。但如果資料本身的規模就超出單機記憶體，連建模前的預處理都跑不完，怎麼辦？

S09：當 pandas 不夠用時

核心訊息： pandas 是單機記憶體內的工具，當資料量超過機器 RAM，或需要高效能 SQL 式查詢，需要不同工具——Arrow / Polars / DuckDB / PySpark 是對應不同規模的解法。

講師要點：

- pandas 的邊界：實測上，超過記憶體 30–50% 的 DataFrame 就會開始變慢甚至 OOM（Out of Memory）；這不是 pandas 的缺陷，而是它的設計邊界
- Apache Arrow：一個記憶體格式標準，讓不同工具之間的資料交換不需要複製——pandas 2.0 的 Arrow backend 讓它在某些場景快上數倍
- Polars / DuckDB：同樣在單機運行，但用更現代的執行引擎（lazy evaluation、多核並行、向量化查詢），可以處理遠超 pandas 容量的資料，且語法更接近 SQL
- PySpark：當資料需要分散到多台機器才能處理時，PySpark 是 Python 進入分散式計算（Hadoop / Spark 生態）的橋樑
- 選哪個工具不是品味問題，是資料規模問題：100MB 用 pandas、10GB 用 Polars/DuckDB、100GB+ 用 PySpark——這是工程判斷，不是偏好

視覺建議： 一個「資料規模梯度」示意圖，X 軸是資料量（MB → GB → TB），Y 軸是工具，每個工具對應一個有效範圍的色塊：pandas（< 1GB）、Polars/DuckDB（1GB–100GB）、PySpark（100GB+）；圖下方加一句說明：「沒有最好的工具，只有

過渡： 在這個梯度的末端，PySpark 是 Python 工程師進入大數據世界的主要橋樑。

S10：PySpark：Python 進入大數據分散式的橋樑

核心訊息： PySpark 讓 Python 工程師能用熟悉的 DataFrame API 操作分散在多台機器上的資料，是資料工程與大數據分析職涯的關鍵入口。

講師要點：

- Apache Spark 是大數據分散式計算的主流框架，PySpark 是它的 Python API——你用 Python 寫，Spark 把工作分派到整個叢集
- 最重要的直覺：PySpark 的 DataFrame API 在語法上和 pandas 非常相似，但執行模型完全不同——pandas 是立即執行（eager），Spark 是延遲執行（lazy），只有在觸發 action 時才真正計算
- 這個「lazy evaluation」設計是有意義的：Spark 的優化器（Catalyst）在你下達所有指令後，一次性規劃最有效率的執行計畫，而不是一行一行跑
- 進入 PySpark 的前置條件：理解 Python、熟悉 DataFrame 概念、有基本的 SQL 查詢直覺——你在本課程全部建立了
- PySpark 背後連接著整個 Hadoop 生態（HDFS、Hive、YARN）和雲端資料倉儲（BigQuery、Databricks、AWS EMR），進入這裡等於進入企業級資料工程的核心。分散式執行模型的硬體層細節，在進階課程「系統設計

視覺建議： 一個分散式執行示意圖，左側是「Driver（你的 Python 程式）」，右側是一個叢集方塊，內含三個 Worker 節點，Driver 和叢集之間有雙向箭頭；圖下方用兩個方塊對比「pandas（單機，立即執行）」vs「PySpark（分散式，延遲執

過渡： 技術前導結束。現在你站在五條路的交叉口——讓我們把每條路說清楚。

S11：五條職涯路線地圖

核心訊息：
這門課的底盤通往五條不同的技術路線，每條路都是真實的職涯選項，選擇哪條取決於你對「資料」的哪個面向

講師要點：

- 這五條路線不是等級高低的排序，而是方向的分叉——沒有哪條「更好」，只有哪條更適合你現在的目標
- 每條路線都有清楚的「起點技能」（本課程已建立）和「下一步要學的東西」（課程結束後的方向）
- 最重要的決策依據：你對什麼問題感到好奇？是「為什麼這個模型比那個好？」還是「資料從哪裡來、怎 LLM 幫我做任務？」——問題的方向就是路線的方向
- 不必現在決定，但要現在想：完成這門課的第一週，試著選一條路，找一個入門專案動手做

五條路線詳細說明：

路線	方向	核心技能堆疊	適合誰
Route A	統計分析	機率、抽樣分布、假設檢定、迴歸分析、A/B test 設計與解讀	對「推論」與「因果判斷」感興趣；想 BI / 策略分析 / 產品數據分析
Route B	機器學習工程	scikit-learn、特徵工程、模型評估指標、交叉驗證、集成學習（Ra Forest, XGBoost）	對「建立預測系統」感興趣；想進 DS / ML Engineer / 推薦系統
Route C	深度學習	線性代數、tensor 計算圖、PyTorch、MLP / CNN / RNN / Transformer	對「學習表示」感興趣；想進 CV / NLP / 研究導向職位
Route D	資料工程	SQL、PySpark、ETL pipeline、Arrow / DuckDB / Polars、workflow orchestration（Airflow / Prefect）	對「資料如何流動、如何可靠地移動」 DE / Data Platform / MLOps 基礎設施

Route E	AI 應用 / LLM 工程	向量資料庫、embedding 概念、prompt workflow、RAG（Retrieval-Augmented Generation）、agent pipeline	對「讓 LLM 解決真實任務」感興趣；想進 AI 產品工程 / LLM Ops / GenAI 應用開發

路線選擇的判斷問題：

「你最常問的問題是哪種類型的？」

- 為什麼結果是這樣？（Route A：統計分析）
- 能不能預測得更準？（Route B：ML 工程）
- 機器怎麼看懂這張圖 / 這句話？（Route C：深度學習）
- 資料怎麼可靠地從 A 流到 B？（Route D：資料工程）
- 怎麼讓 AI 幫我完成這個任務？（Route E：LLM 應用）

視覺建議：
一張「路線地圖」示意圖，中央是一個「課程畢業點」圓圈，五條帶箭頭的路線從圓圈輻射出去，每條路線旁列

過渡： 路線地圖確定了。最後，讓我們把這 24 小時走過的東西說清楚。

S12：課程終章：你搭了一條底盤

核心訊息：
你沒有只學一套工具，你建立了一個底盤——讓你能在五條路上繼續走下去，而不需要從頭開始的那種底盤。

講師要點：

- 課程回顧：這門課從第一天就圍繞著一個核心目標——讓你能獨立完成從原始資料到有意義洞察的完整工

- 里程碑式進步：從「能讀 CSV」到「能清理資料」到「能做統計分析」到「能建一個分類模型」到「知道當資料規模大了要換什麼工具
- 心法一：「先懂資料，再懂模型；先搭底盤，再碰 AI。」——這個順序不是保守，而是讓你後面走得更快的投資
- 心法二：「You didn't learn a tool. You built a foundation.」——工具會換，但資料思維、統計直覺、程式設計能力不會換。你學的是那個不換的部分
- 下一步建議：選一條路線，找一個真實問題，用你現在的底盤去碰它。碰壁了，你會知道哪裡需要補——

視覺建議： 一張「Before / After」對比投影片，左側列出「開始前你知道什麼」（Python 基礎、可能一點點 pandas），右側列出「現在你能做什麼」（完整 EDA 流程、統計分析、ML 建模工作流、大數據工具選型判斷、五條職涯路線的第一步）；最下方單獨一行大字引用最終心法：

> "Machine learning turns patterns into prediction. You just learned how to prepare the patterns."

過渡： 課程結束。但你的學習才剛剛開始。

四、動手練習區

Part A 實作練習：完整建模工作流

目標： 在一個熟悉的資料集上走完一次完整的 scikit-learn 建模流程，從 DataFrame 到評估報告。

練習骨架：

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

# ═══════════════════════════════
# Step 1: 載入資料
df = pd.read_csv("data/customer_data.csv")

# Step 2: 定義特徵
features = ["age", "monthly_spend", "tenure_months", "num_products"]
target = "churn"
```

```
# 
X = df[features].dropna()
y = df.loc[X.index, target]
# 
# Step 3: 
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# 
# Step 4: 
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# 
# Step 5: 
model = LogisticRegression(max_iter=1000)
model.fit(X_train_scaled, y_train)
# 
# Step 6: 
train_score = model.score(X_train_scaled, y_train)
test_score = model.score(X_test_scaled, y_test)
# 
print(f"Train accuracy: {train_score:.4f}")
print(f"Test  accuracy: {test_score:.4f}")
print()
print(classification_report(y_test, model.predict(X_test_scaled)))
```

觀察重點： 1. `train_score` 和 `test_score` 的差距有多大？超過 5% 就值得懷疑是否過擬合 2. `classification_report` 中，precision 和 recall 哪個對你的問題更重要？ 3. 試著把 `LogisticRegression` 換成 `RandomForestClassifier`，結果如何變化？

延伸挑戰（選做）：

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
import numpy as np
# 
# 5-fold cross validation
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
cv_scores = cross_val_score(rf_model, X, y, cv=5, scoring="f1")
# 
print(f"CV F1 scores: {cv_scores}")
print(f"Mean F1: {np.mean(cv_scores):.4f} (+/- {np.std(cv_scores):.4f})")
```

Part B 實作練習：工具規模判斷

目標： 根據資料大小，選擇正確的工具，並比較執行時間。

```
import pandas as pd
import time

# ═══════════════════
def benchmark_pandas(filepath: str) -> float:
    start = time.time()
    df = pd.read_csv(filepath)
    result = df.groupby("category")["value"].sum()
    return time.time() - start

# ══════ polars
import polars as pl
def benchmark_polars(filepath: str) -> float:
    start = time.time()
    df = pl.scan_csv(filepath).group_by("category").agg(pl.col("value").sum()).collect()
    return time.time() - start

# ═══
1. ═══════════ pandas ═══
2. ═══════════ 100 ═══════
3. ═══════ 50 ═══ HDFS ═══
```

Part C 反思練習：職涯路線選擇

這不是程式練習，是思考練習。

用一張紙（或一個 Notion 頁面），回答以下三個問題：

> 我最感到好奇的問題是哪種類型的？ 把你的答案對應到五條路線中的一條。
>
> 我的下一個 30 天學習計畫是什麼？
> 根據選擇的路線，列出三個具體的行動（例如：讀某本書的第 X 章、做某個 Kaggle 競賽、複製某個開源專案的一部分）。
>
> 我的「不知道」清單是什麼？
> 把你現在知道自己不懂的東西列出來，這比「知道什麼」更有價值——因為它是你的學習地圖。

五、五條職涯路線詳細說明

Route A：統計分析路線

核心問題：「這個現象是真實的，還是隨機誤差造成的？」

技術堆疊（按學習順序）：

層次	技術內容
基礎統計	機率分布（常態、二項、泊松）、期望值、變異數
推論統計	抽樣分布、中央極限定理、信賴區間
假設檢定	t-test、chi-square test、ANOVA、p-value 的正確解讀
迴歸分析	線性迴歸診斷、多元迴歸、交互效應、殘差分析
實驗設計	A/B test 設計（樣本數計算、顯著水準、統計功效）
應用工具	Python `scipy.stats`、`statsmodels`、R（選修）

典型職位： 商業分析師、策略分析師、產品數據分析師、BI 工程師、研究分析師

本課程已建立的前置底盤： pandas 資料清理（M4）、統計基礎（M6）、視覺化呈現（M5）

Route B：機器學習工程路線

核心問題：「我能不能讓預測系統在真實環境中可靠地運行？」

技術堆疊（按學習順序）：

層次	技術內容

特徵工程	缺值處理策略、類別編碼（one-hot、target encoding）、特徵縮放
模型評估	ROC-AUC、PR curve、F1 score、confusion matrix 解讀
交叉驗證	k-fold、stratified fold、time-series split
集成學習	Random Forest、Gradient Boosting（XGBoost、LightGBM）
超參數調整	Grid Search、Random Search、Bayesian Optimization
管線化	scikit-learn Pipeline、ColumnTransformer
模型部署基礎	joblib 序列化、FastAPI 包裝、Docker 打包

典型職位：機器學習工程師、資料科學家、推薦系統工程師、風險模型工程師

本課程已建立的前置底盤：scikit-learn 工作流（本模組 Part A）、pandas 特徵處理（M4）、統計評估直覺（M6）

Route C：深度學習路線

核心問題：「機器如何從原始資料中自動學到有用的表示？」

技術堆疊（按學習順序）：

層次	技術內容
數學基礎	線性代數（矩陣乘法、特徵值）、微積分（鏈式法則）、機率論

Tensor 與 Autograd	PyTorch tensor 操作、計算圖、`.backward()` 機制
基礎架構	全連接層（MLP）、激活函數、損失函數、優化器（Adam、SGD）
卷積神經網路	CNN 結構、影像特徵提取、ResNet / EfficientNet
序列模型	RNN、LSTM、GRU、時序資料建模
Transformer	注意力機制、self-attention、BERT / GPT 架構直覺
效能工程	`torch.compile()`、混合精度訓練、分散式訓練概念

典型職位： 深度學習工程師、電腦視覺工程師、NLP 工程師、AI 研究員

本課程已建立的前置底盤： NumPy tensor 概念（M3）、PyTorch 前導（本模組 Part B）、Python 基礎（M2）

Route D：資料工程路線

核心問題：「資料怎麼可靠地從原始來源流動到可用的狀態？」

技術堆疊（按學習順序）：

層次	技術內容
SQL 基礎	SELECT / JOIN / GROUP BY / Window Functions、查詢優化直覺
分散式計算	PySpark DataFrame API、RDD 概念、執行計畫（explain()）
ETL 設計	資料管線設計原則、idempotency、錯誤重試、資料品質驗證

現代資料工具	Apache Arrow、DuckDB（嵌入式 OLAP）、Polars（Rust-based DataFrame）
工作流編排	Apache Airflow DAG 設計、Prefect、任務依賴管理
資料格式	Parquet、ORC、Avro、Delta Lake、Iceberg
雲端資料倉儲	BigQuery / Snowflake / Redshift 基礎查詢模式

典型職位： 資料工程師、Data Platform 工程師、MLOps 工程師、Analytics Engineer

本課程已建立的前置底盤： pandas 資料處理（M4）、PySpark 前導（本模組 Part B）、Python 基礎（M2）

> 資料管線的系統底層設計（process 模型、I/O 架構、分散式儲存原理）在進階課程「系統設計與架構思維」中有完整介紹。

Route E：AI 應用 / LLM 工程路線

核心問題：「怎麼讓大型語言模型可靠地解決真實的業務問題？」

技術堆疊（按學習順序）：

層次	技術內容
Embedding 基礎	向量表示的直覺、相似度計算（cosine similarity）、embedding 模型
向量資料庫	Chroma / Pinecone / Weaviate 的基本操作、索引與查詢
Prompt 工程	系統提示設計、few-shot learning、prompt 模板管理

RAG 架構	Retrieval-Augmented Generation 流程設計、文件分塊策略
LLM API 整合	OpenAI / Claude / Gemini API 使用、token 計算與成本控制
Agent 管線	Tool use、function calling、multi-step 推理流程
評估與監控	LLM 輸出評估指標、幻覺（hallucination）偵測、生產監控

典型職位： AI 應用工程師、LLM Ops 工程師、GenAI 產品工程師、Conversational AI 工程師

本課程已建立的前置底盤： Python 基礎（M2）、pandas 資料處理（M4）、統計思維（M6）

六、課程收束

單一路線回顧

這 24 小時的課程以「讓初學者能獨立完成資料分析工作流」為核心目標，一步一步往前推進：

```
M0 [redacted]
[redacted]
M1 [redacted]
[redacted]
M2 Python [redacted]
[redacted]
M3 NumPy [redacted] tensor [redacted]
[redacted]
M4 pandas [redacted] EDA [redacted]
[redacted]
M5 [redacted]
[redacted]
M6 [redacted]
[redacted]
M7 ML [redacted]
[redacted]
[redacted] +-- Route A: [redacted]
[redacted] +-- Route B: [redacted]
[redacted] +-- Route C: [redacted]
```

```
+-- Route D: [redacted]
+-- Route E: AI [redacted] / LLM [redacted]
```

每一個模組都建立在前一個之上，也為後面打開了新的可能性：pandas 的 DataFrame 背後是 NumPy array；統計思維讓你知道模型評估指標的意義；視覺化讓你能診斷資料問題而不只是跑程式碼。

最終心法

> 先懂資料，再懂模型；先搭底盤，再碰 AI。

這個順序不是在叫你慢下來，而是讓你後面走得更快的投資邏輯。很多人在碰 AI 工具的時候卡住，不是因為 AI 太難，而是底盤還沒搭穩。

這門課做的事情只有一件：幫你把底盤搭穩。

> "You didn't learn a tool. You built a foundation."

工具會換，框架會更新，API 會改版，但資料思維、統計直覺、程式設計能力不會換。你在這 24 小時裡學的，是那個不換的部分。

參考文獻與引用來源

機器學習框架

1. scikit-learn Official Documentation — Python 傳統 ML 的標準入口。始於 2007 年 David Cournapeau 的 Google Summer of Code 專案，1.0 版於 2021 年 9 月 24 日發布。scikit-learn.org
2. scikit-learn 1.0 Release Highlights — scikit-learn.org/stable/auto_examples/release_highlights/plot_release_highlights_1_0_0.html

深度學習

1. PyTorch 2.0 Release Blog (March 2023) — `torch.compile()` 在 163 個開源模型上測試，93% 成功編譯，A100 GPU 上平均訓練加速 43%（Float32 精度 21%、AMP 精度 51%）。pytorch.org/blog/pytorch-2-0-release
2. PyTorch Official Documentation — pytorch.org/docs

CRISP-DM 框架

1. Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C. & Wirth, R. (2000). CRISP-DM 1.0: Step-by-step Data Mining Guide. — CRISP-DM 的原始規格文件，由 NCR、DaimlerChrysler、SPSS、OHRA 聯合制定。kde.cs.uni-kassel.de/lehre/ws2012-13/kdd/files/CRISPWP-0800.pdf
2. Wirth, R. & Hipp, J. (2000). "CRISP-DM: Towards a Standard Process Model for Data Mining." — CRISP-DM 的學術論文。

大數據工具

1. PySpark Official Documentation — Python 操作 Apache Spark 分散式運算。spark.apache.org/docs/latest/api/python
2. Polars Documentation — 高效能 DataFrame 引擎。docs.pola.rs
3. DuckDB Documentation — 嵌入式 OLAP 資料庫。duckdb.org/docs
4. Apache Arrow — 跨語言欄式記憶體格式。arrow.apache.org

產業數據

1. 「85% 的大數據專案失敗」 — 原始來源：Gartner 分析師 Nick Heudecker 於 2017 年表示 Gartner 最初估計的 60% 失敗率「太保守」，實際接近 85%。另見 VentureBeat (2019) 報導：87% 的資料科學專案未能進入生產環境。參見 designingforanalytics.com/resources/failure-rates-for-analytics-bi-iot-and-big-data-projects-85-yikes
2. 「資料準備佔專案 60-80% 時間」 — CrowdFlower (2016). Data Science Report：約 60% 時間用於清理，19% 用於收集。此數據在 Forbes、HBR 等被廣泛引用。Kaggle (2018) State of Data Science Survey 報告為 15-25%。實際比例因組織成熟度而異。

3. 「訓練/測試分割通常為 70-80% / 20-30%」 — 此為 ML 社群廣泛接受的經驗法則。Hastie, T., Tibshirani, R. & Friedman, J. (2009). The Elements of Statistical Learning, 2nd Edition. Springer. 第七章討論了交叉驗證與資料分割策略。免費版本：hastie.su.domains/ElemStatLearn

延伸閱讀

1. Hastie, T., Tibshirani, R. & Friedman, J. (2009). The Elements of Statistical Learning, 2nd Edition. Springer. — ML 理論的標準教科書。
2. Geron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Edition. O'Reilly Media. — 實務導向的 ML 入門書。

下一步

1. 選一條路線——不用現在就確定一輩子，只要選一個 30 天的方向
2. 找一個真實問題——不是教材上的資料集，是你自己遇到的、想解決的問題
3. 用你的底盤去碰它——碰壁了，你會知道哪裡需要補；那才是真正的學習起點

> "Machine learning turns patterns into prediction. You just learned how to prepare the patterns."

模組資訊

項目	內容
模組編號	M7（最終模組）
時長	3 小時
課程位置	24 小時課程第 22–24 小時

前置模組	M0–M6（全部模組）
後續方向	五條職涯路線（Route A–E）